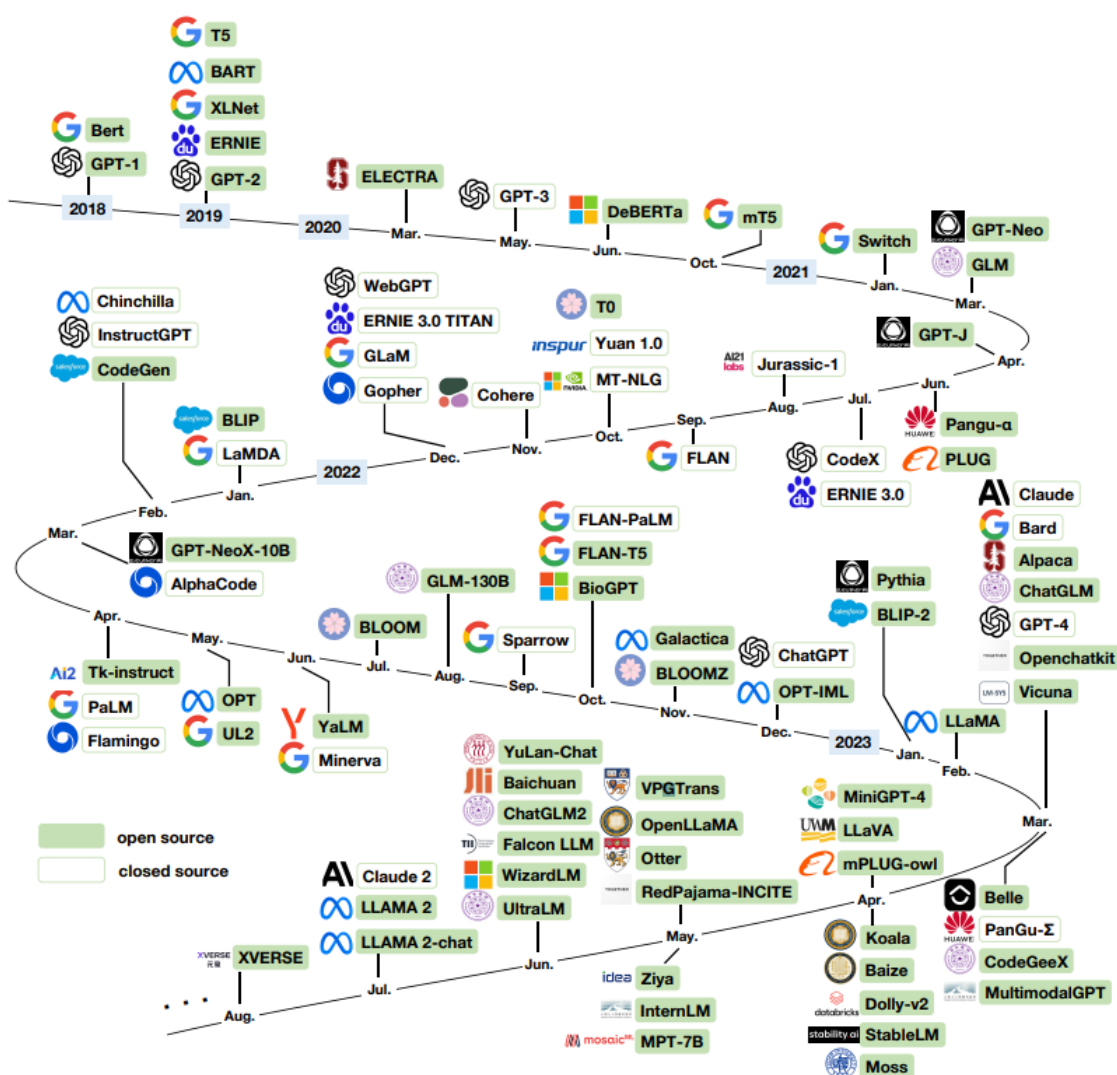


自然语言处理理论和应用

前言

自然语言是人类智慧的结晶，自然语言处理是人工智能中最为困难的问题之一，而对自然语言处理的研究也是充满魅力和挑战的。



第一章 自然语言处理简介

1.1 什么是自然语言？

什么是自然语言？

- 以语音为物质外壳，由词汇和语法两部分组成的符号系统。文字和声音是语言的两种属性。
- 语言是人类交际的工具，是人类思维的载体；人类历史上以语言文字形式记载和流传的知识占人类知识总量的80%以上。
- 是约定俗成的，有别于人工语言，比如Java、C++等程序设计语言。

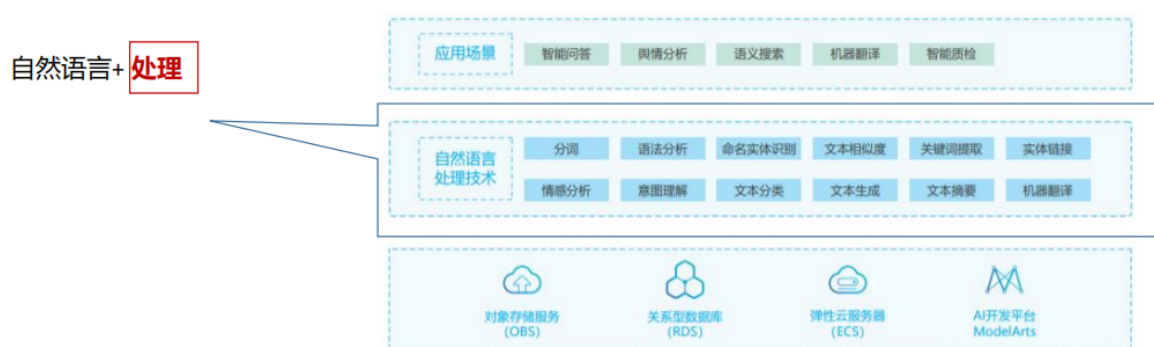
1.2 什么是自然语言处理？(1)

● 什么是自然语言处理（Natural Language Processing, NLP）？

□ 自然语言处理就是，利用计算机为工具对人类特有的书面形式和口头形式的自然语言的信息，进行各种类型处理和加工的技术。----冯志伟

□ 自然语言处理可以定义为研究在人与人交际中以及在人与计算机交际中的语言问题的一门学科。自然语言处理要研制表示**语言能力**和**语言应用**的模型，建立计算框架来实现这样的语言模型，提出相应的方法来不断完善这样的语言模型，根据这样的语言模型设计各种实用系统，并探讨这些实用系统的评测技术。----Bill Manaris

1.3 什么是自然语言处理？(2)



- 自然语言处理是人工智能的一个分支，旨在使计算机能够理解、处理和生成人类语言。
- 主要组成部分：
 - 自然语言理解（NLU）：
 - 目标：使计算机理解人类语言，提取有用信息。
 - 应用：情感识别、意图识别、信息检索（个性化搜索、语义搜索、文本匹配）、信息抽取（命名实体提取、关系抽取、事件抽取）。
 - 自然语言生成（NLG）：
 - 目标：将结构化数据转换为人类可理解的自然语言文本。
 - 分类：
 - 文本到文本（text-to-text）：翻译、摘要等。
 - 文本到其他（text-to-other）：文本生成图片等。
 - 其他到文本（other-to-text）：视频生成文本等。

1.4 自然语言处理的重要性

● 在人工智能出现之前，机器智能处理结构化的数据，例如表格。但是生活中大部分的数据都是非结构化的，例如：文章、图片、音频、视频等。

● 其中文本数据，数据量和信息量巨大。为了让机器帮助我们处理问题，则需要分析利用这些数据，让机器读懂我们的语言，从而搭建人类和机器之间沟通的桥梁。



• 语言理解是人工智能领域皇冠上的明珠。---比尔盖茨

1.5 自然语言处理应用

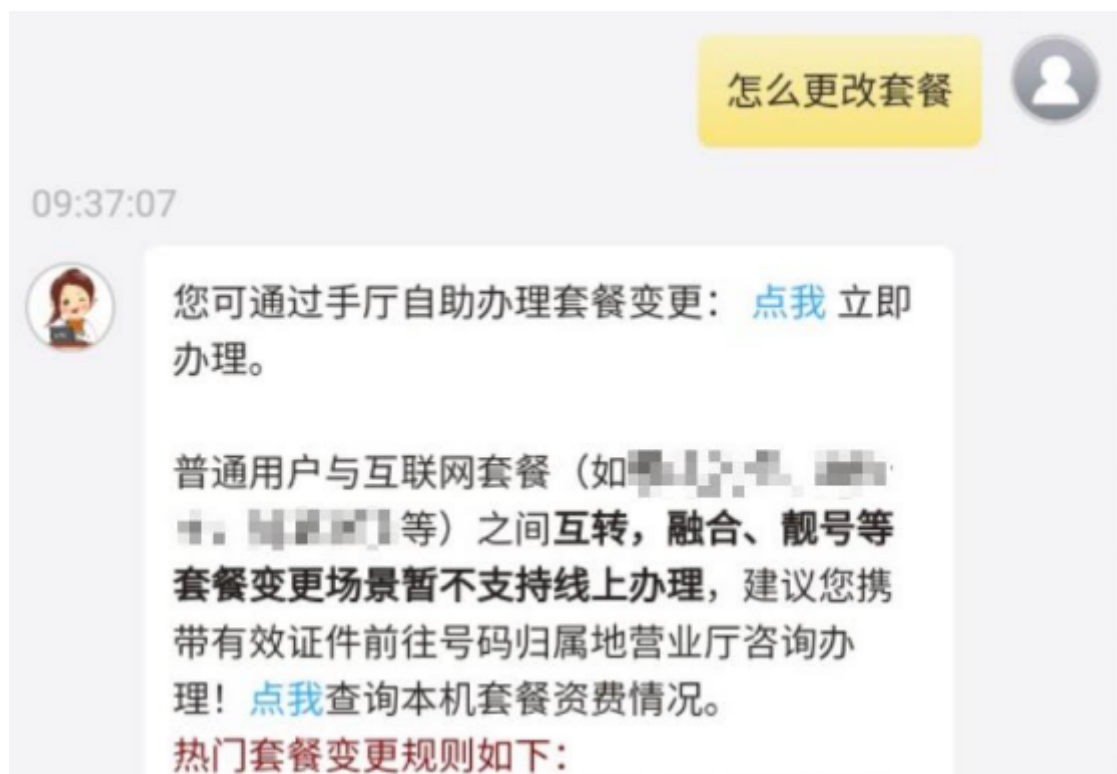
● 自然语言处理技术被广泛应用于搜索、对话、翻译、推荐等场景中：

- ☐ 智能问答
- ☐ 对话助手
- ☐ 语义搜索
- ☐ 信息抽取
- ☐ 推荐系统
- ☐ 机器翻译
- ☐ 文本创作
- ☐

1.6 对话理解



1.7 智能客服



1.8 情感分析

- 对商品评论自动过滤，筛选负面评论供人工审核。

买没几天就降价一点都不开心，闪存跑分就五百多点点 --- 😞

外观漂亮音质不错，现在电子产品基本上都是华为的了 --- 😄

● 属性情感分析：

评论	动力	外观	空间	油耗
2.0T涡轮增压发动机动力强，各种路况表现都不错，高速120超车没压力；外观是我和老婆都比较喜欢的款；后排空间有点小；有点费油啊。	😄	😄	😞	😞

1.9 内容审核





用户01

★★★★★

9月20日 15:10

外观惊艳！国产手机的巅峰之作！！！！



用户02

★★★★★

9月20日 16:12

这款手机真*****

 违规信息

1.10 词语联想



1.11 机器翻译



1.12 自然语言理解层次

● 自然语言的理解和分析是一个层次化的过程，具体可分为以下五个层次：

- 语音分析，是要根据音位规则，从语音流中区分出一个个独立的音素，再根据音位形态规则找出音节及其对应的词素或词。
- 词法分析，是找出词汇的各个词素，从中获得语言学的信息。

□ 句法分析，是对句子和短语的结构进行分析，目的是要找出词、短语等的相互关系以及各自在句中的作用。

□ 语义分析，是找出词义、结构意义及其结合意义，从而确定语言所表达的真正含义或概念。

□ 语用分析，是研究语言所存在的外界环境对语言使用者所产生的影响。



1.13 自然语言处理难点 (1)

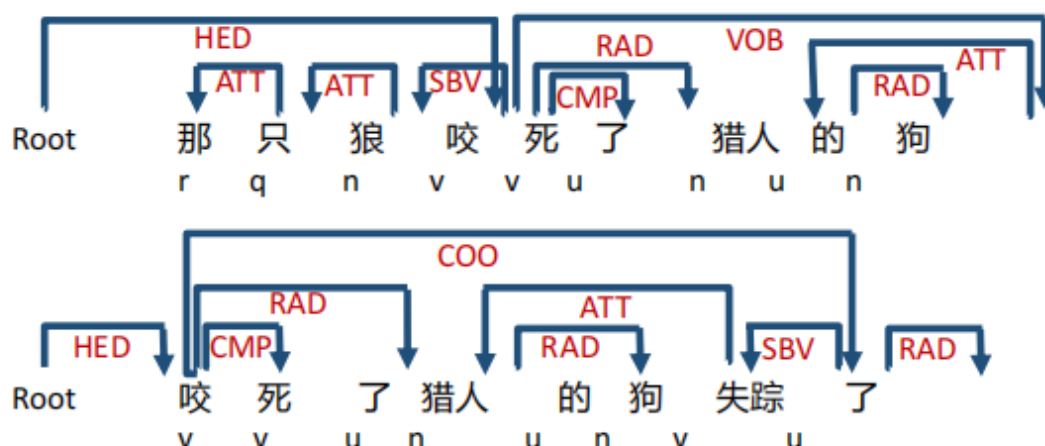
- 词法歧义：
 - 分词：词语的切分边界比较难确定。
 - 南京/市长/江大桥
 - 南京市/长江/大桥
 - 词性标注：同一个词语在不同的上下文中词性不同。
 - 我/计划/v 考/研/
 - 我/完成/了/计划/n
 - 命名实体识别：人名、专有名称、缩略词等未登录词的识别困难。
 - 高超/nr/a 华明/nr/nt 移动/nt/v

1.14 自然语言处理难点 (2)

● 句法歧义：句法层面上的依存关系受上下文的影响。

□ 咬死了猎人的狗

- 那只狼咬死了猎人的狗
- 咬死了猎人的狗失踪了



- **HED (核心词)**：指句子的核心词语，通常是动词或名词。
- **ATT (定语)**：指修饰名词的词语，如形容词或名词。
- **SBV (主谓关系)**：指主语和谓语之间的关系。
- **VOB (动宾关系)**：指动词和宾语之间的关系。
- **RAD (附加关系)**：指附加的修饰关系。
- **CMP (补语)**：指补充说明动词或形容词的词语。

- **COO (并列关系)**：指两个或多个词语之间的并列关系。

1.15 自然语言处理难点 (3)

● 语义歧义

□ At last, a computer understands you like your mother.

- 含义1：计算机像你的母亲那样很好的理解你。
- 含义2：计算机理解你喜欢你的母亲。
- 含义3：计算机像理解你母亲那样去理解你。

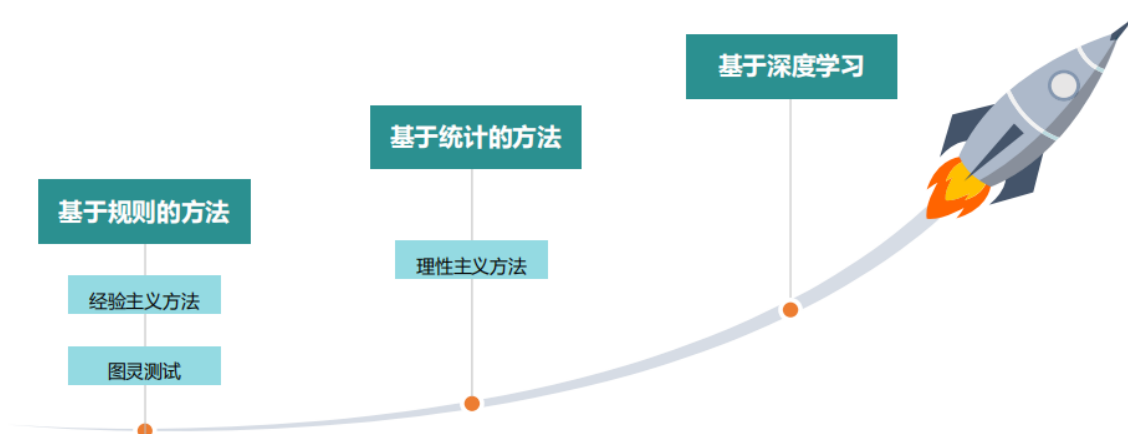
1.16 自然语言处理难点 (4)

● 语用歧义

□ “你在干什么？”

- 纯粹的询问：朋友之间的简单询问，想知道对方正在做些什么事情。
- 生气的质疑：当妈妈对淘气的儿子说时，实际表达的是儿子可能干了一些坏事。
- 疑惑的怀疑：做了一些不太合适或不可告人的事情，带有一种调查或质疑的意味。

1.17 自然语言理解的发展历史



• 基于规则的方法: 具有不可避免的缺点，首先规则不可能覆盖所有语句，其次这种方法对开发者的要求极高，开发者不仅要精通计算机还要精通语言学，因此，这一阶段虽然解决了一些简单的问题，但是无法从根本上将自然语言理解实用化。

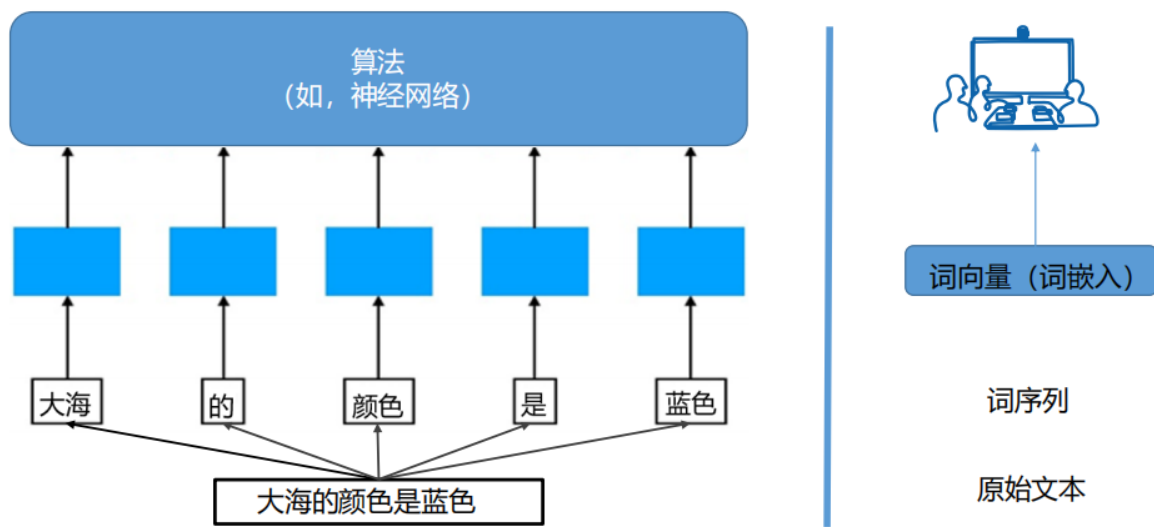
• 基于统计的方法: 随着互联网的高速发展，丰富的语料库成为现实以及硬件不断更新完善，自然语言处理思潮由经验主义向理性主义过渡，基于统计的方法逐渐代替了基于规则的方法。

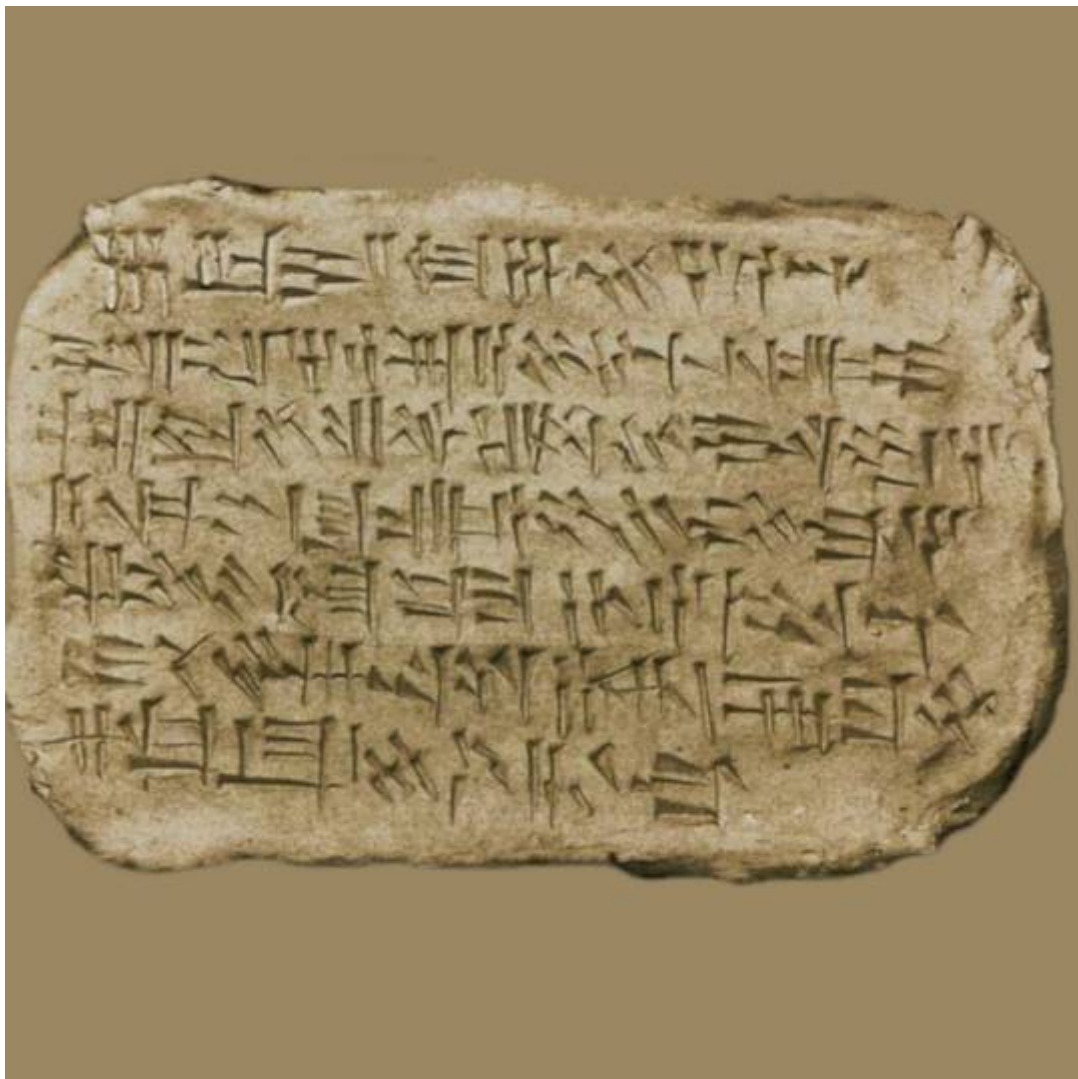
- 基于深度学习的方法：从输入到输出做端到端的训练。把输入到输出对的数据准备好，设计并训练一个神经网络，即可执行预想的任务。RNN已经是自然语言处理最常用的方法之一，GRU、LSTM等模型相继引发了一轮又一轮的热潮。

第二章 自然语言处理任务与方法

2.1 词向量

2.1.1 为什么需要词向量？





楔形文字，为苏美尔人所创，是世界上最古老的文字。

计算机正如人类一样遇到不懂文字无法被直接理解。

1. 计算机无法直接处理文本：

- 计算机只能处理数值数据，词语本身是符号，无法直接被算法理解。

2. 捕捉语义关系：

- 词向量能够将词语映射到一个高维空间中，在这个空间中，词语之间的距离和方向可以反映它们的语义关系。
- 这使得算法能够理解词语之间的相似性和差异性。

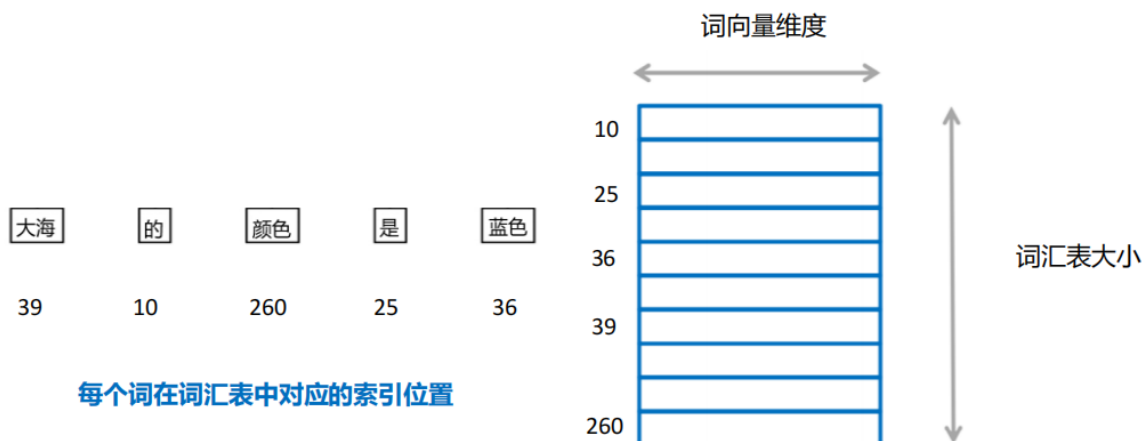
3. 提高算法性能：

- 通过使用词向量，算法能够更好地理解文本数据，从而提高NLP任务的性能。

简单来说：

- 词向量是连接自然语言和计算机算法的桥梁。
- 它们将词语转换成计算机能够理解的数值形式，并保留了词语的语义信息。

2.1.2 如何使用词向量？



2.1.3 如何表示词？ - 离散符号表示

- OneHot, 独热编码，是一种用离散符号来表示词向量的方法。向量的长度为词汇表中总的词数。



- OneHot的缺点

One-Hot编码是一种常用的分类特征编码方式，但它也存在一些明显的缺点：

1. 高维度和稀疏性：

- 当分类特征的类别数量非常多时，One-Hot编码会产生高维度的向量。例如，一个拥有1000个不同类别的特征，经过One-Hot编码后，每个样本都会被表示为一个1000维的向量。
- 这些向量中，绝大部分元素都是0，只有对应类别的位置是1，因此数据变得非常稀疏。
- 高维度和稀疏性会增加存储空间的需求，并且在训练模型时会消耗更多的计算资源。

2. 无法表示类别之间的关系：

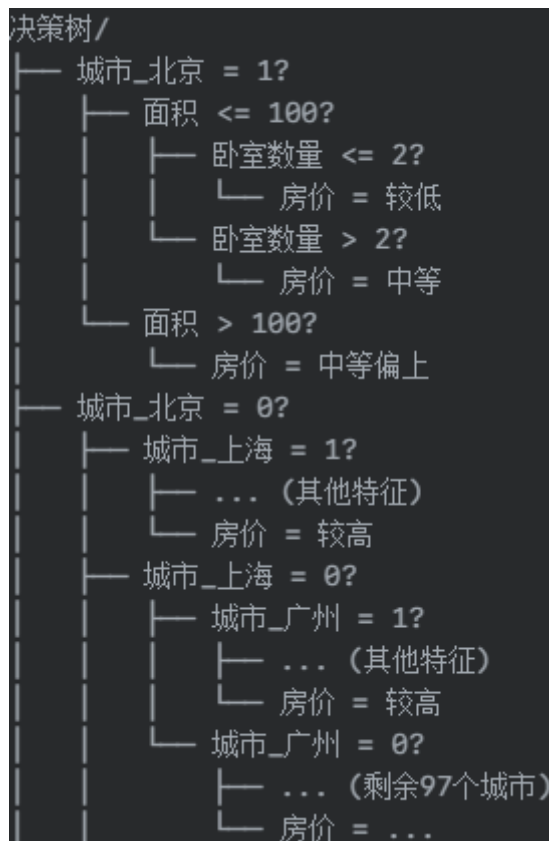
- One-Hot编码将每个类别都视为独立的，无法表达类别之间的任何内在联系或相似性。例如，它无法表达“苹果”和“香蕉”都属于“水果”这一概念。
- 这种缺乏关系表示的特性，可能导致模型在处理某些任务时效果不佳。

3. 对于树模型不友好：

- 在树模型（如决策树、随机森林、梯度提升树等），One-Hot编码会产生大量的分裂点，增加树的深度，影响模型的效率和泛化能力。

4. 可能导致维度灾难：

- 在机器学习中，当数据的维度过高时，会出现“维度灾难”现象，即模型性能随着维度增加而下降。
- One-Hot编码可能加剧维度灾难问题，尤其是在处理高基数（类别数量多）的分类特征时。



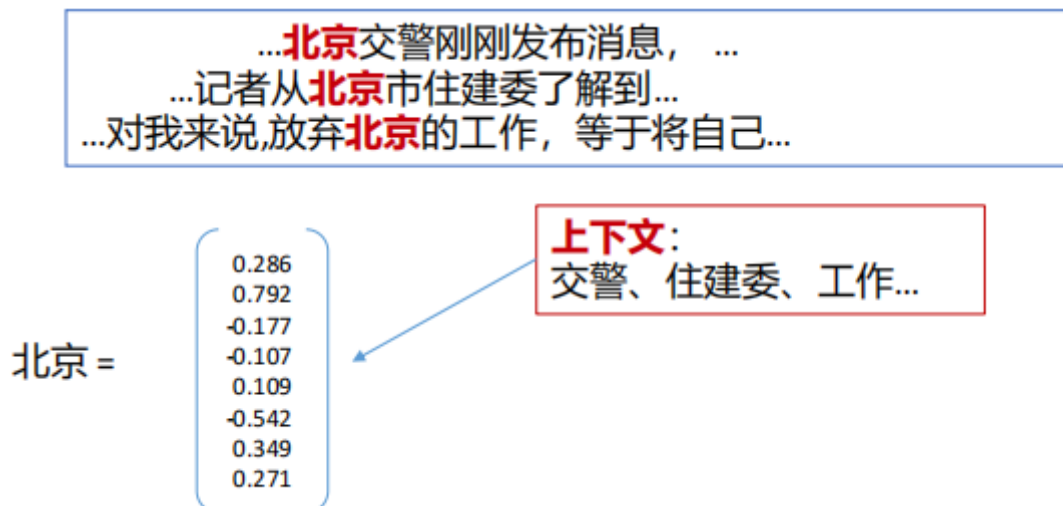
总的来说，One-Hot编码虽然简单易用，但在处理高基数分类特征时，其缺点会变得尤为突出。为了克服这些缺点，可以考虑使用其他编码方式，例如：

- 目标编码 (Target Encoding)：比如使用50分-差 80分-中 90分-高
- 词嵌入 (Word Embedding)
- 特征哈希 (Feature Hashing)

2.1.4 如何表示词？ - 分布式语义表示

● **分布式语义表示**：每个单词不再是完全正交的 0-1 向量，而是在多维实数空间中的一个点，具体表现为一个实数向量。很多时候，也会把单词的这种分布式表示叫做 **词嵌入 (Word Embedding)**。

● **分布式假设**：某个单词的含义由它周围的单词形成。



• 分布式语义表示解决了One-hot无法表示近似词的问题。

- 一个单词的真正意思应该由上下文决定。

在数学和线性代数中，**正交 (Orthogonal)** 是一个描述向量之间关系的概念。简单来说，如果两个向量是正交的，它们在几何意义上是“垂直”的，或者更准确地说，它们之间的夹角是 90 度。

更准确的定义：

- 在线性代数中，两个向量被称为正交，当且仅当它们的**内积 (点积)** 为零。
- 内积是一种将两个向量映射到一个标量 (数值) 的运算。

为什么“正交”很重要？

- **独立性：** 正交向量在某种程度上是“独立”的。这意味着它们之间没有直接的线性关系。
- **简化计算：** 在许多数学和工程问题中，使用正交向量可以简化计算。
- **基向量：** 在向量空间中，一组相互正交的基向量可以提供一個简洁且高效的坐标系统。

在词向量的上下文中：

- 在早期的自然语言处理中，人们有时会使用“one-hot 编码”来表示单词。
- “one-hot 编码”为每个单词创建一个单独的向量，该向量的长度等于词汇表的大小。
- 在这个向量中，只有一个元素是 1，其余元素都是 0。
- 如果使用 one-hot 编码，那么每个单词的向量，和其他单词的向量之间的内积都是 0，所以它们之间是正交的。
- 但是，这种编码方式无法捕捉单词之间的语义关系。
- Word2Vec 等词嵌入技术将单词映射到低维稠密向量空间中，使得语义相似的单词在向量空间中距离较近。
- 在这些词向量中，单词不再是完全正交的，而是通过向量之间的距离和方向来表示语义关系。

简单来说：

- 正交意味着“垂直”或“独立”。
- 在词向量中，使用 Word2Vec 等技术时，单词不再是完全独立的，而是通过向量之间的关系来表示语义。

2.1.5 如何表达词的语义？

● “北京” vs “上海”？

...北京交警刚刚发布消息, ...
...记者从北京市住建委了解到...
...对我来说,放弃北京的工作,等于将自己...

...上海交警刚刚发布消息...
...记者从上海市住建委了解到...
...我在上海工作, ...

● 思考：为什么“北京”和“上海”的词向量的相似度高？

1. 语义相似性：

- “北京”和“上海”都是中国的特大型城市，都是直辖市，也是重要的经济、文化和政治中心。

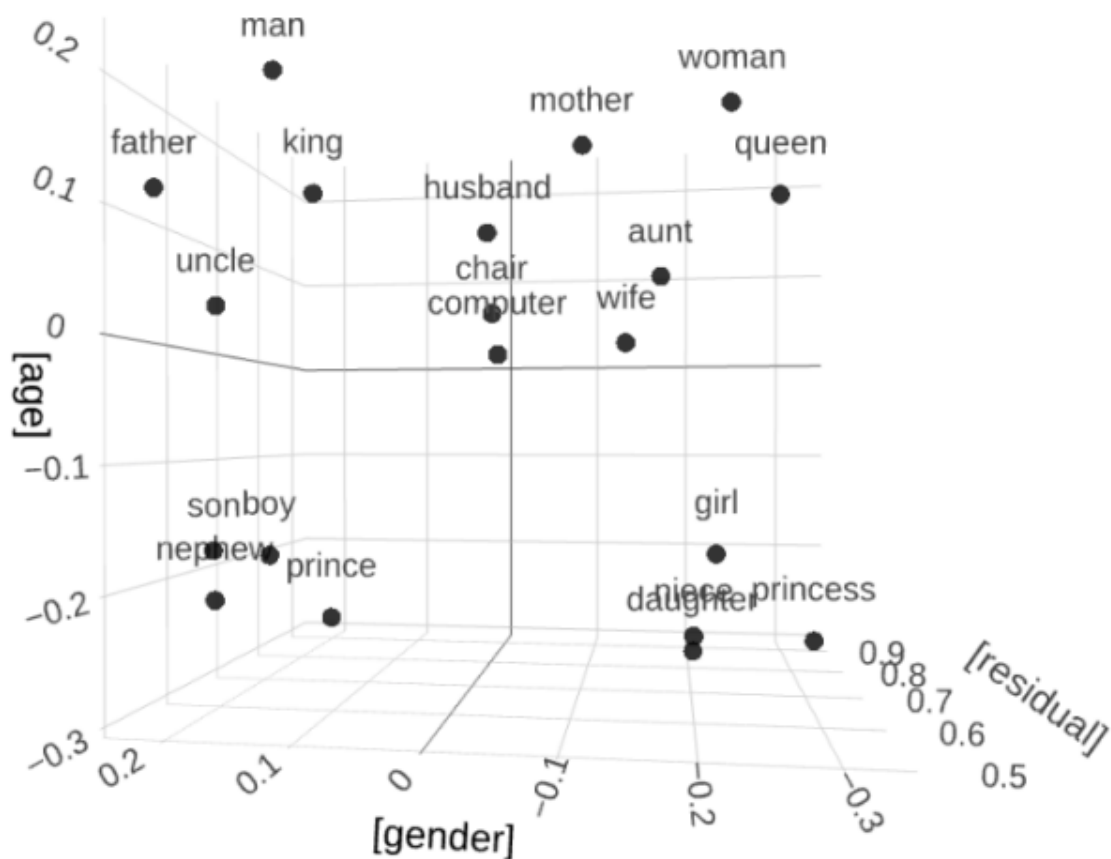
- 在大量文本数据中，它们经常出现在相似的上下文中，例如：
 - “...记者从**北京/上海市**住建委了解到...”
 - “...**北京/上海**交警刚刚发布消息...”
 - “...我在**北京/上海**工作...”
- 这些相似的上下文使得模型在训练过程中，将它们映射到相近的向量空间中。

2. 上下文相似性：

- 在训练词向量时，模型会考虑词语的上下文信息。
- 由于“北京”和“上海”经常出现在类似的语境中，例如与“城市”、“经济”、“交通”、“政府”等相关的词语，因此它们的上下文向量也比较相似。
- 这种上下文的相似性进一步拉近了它们在向量空间中的距离。

3. 词向量的特性：

- 词向量旨在捕捉词语之间的语义关系。
- 对于具有相似属性或功能的词语，词向量会倾向于将它们表示在相近的位置。
- “北京”和“上海”作为同类型的城市，自然会被认为具有较高的语义相似性。



2.1.6 如何体现上下文？ - 共现矩阵

窗口为2的共现矩阵:

语料库:

我 爱 上海
我 爱 杭州
我 爱 北京 天安门



	我	爱	上海	杭州	北京	天安门
我	0	3	0	0	0	0
爱	3	0	1	1	1	0
上海	0	1	0	0	0	0
杭州	0	1	0	0	0	0
北京	0	1	0	0	0	1
天安门	0	0	0	0	1	0

2.1.7 共现矩阵 + SVD奇异值分解

● X 为共现矩阵, SVD矩阵分解为低维稠密矩阵:

$$\begin{pmatrix} \hat{X} \\ x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & \\ \vdots & \vdots & \ddots & \\ x_{m1} & & & x_{mn} \\ m \times n \end{pmatrix} \approx \begin{pmatrix} U \\ u_{11} & \dots & u_{1r} \\ \vdots & \ddots & \\ u_{m1} & & u_{mr} \\ m \times r \end{pmatrix} \begin{pmatrix} S \\ s_{11} & 0 & \dots \\ 0 & \ddots & \\ \vdots & & s_{rr} \\ r \times r \end{pmatrix} \begin{pmatrix} V^T \\ v_{11} & \dots & v_{1n} \\ \vdots & \ddots & \\ v_{r1} & & v_{rn} \\ r \times n \end{pmatrix}$$

● 思考: 共现矩阵+SVD方法的缺点?

1. 共现矩阵的高维稀疏性:

- 共现矩阵记录了词汇在文本中共同出现的次数, 当词汇表很大时, 矩阵的维度会非常高。
- 由于大部分词汇不会同时出现在同一个上下文中, 因此矩阵中会存在大量的零值, 导致数据非常稀疏。
- 高维稀疏矩阵会占用大量内存, 并且在计算过程中效率低下。

2. 共现矩阵忽略词序信息:

- 共现矩阵只统计词汇共同出现的次数, 不考虑它们在句子或文本中的顺序。
- 词序对于表达语义至关重要, 忽略词序会导致模型无法捕捉到词汇之间的细微关系。
- 例如, “猫追老鼠”和“老鼠追猫”的共现矩阵是相同的, 但它们的语义完全不同。

3. 计算复杂度高, 新增词汇需重新计算:

- 构建共现矩阵需要统计整个语料库中所有词汇的共现次数, 计算量非常大。
- 每当语料库中新增一个词汇时, 都需要重新计算整个共现矩阵, 这使得模型难以适应动态变化的语料库。

4. SVD 学习的语义表示较浅:

- SVD 是一种线性降维方法, 它只能捕捉到词汇之间浅层的线性关系。
- 词汇之间的语义关系往往是非线性的, SVD 无法有效地捕捉这些复杂的语义关系。
- 因此, SVD 得到的词向量可能无法很好地表示词汇的深层语义信息。

2.1.8 为什么不直接学习词的分布式表达呢?

1. 克服共现矩阵的缺点:

- 稀疏性问题:

- 共现矩阵非常庞大且稀疏，导致存储和计算效率低下。
- 直接学习词向量的方法（如 Word2Vec 和 GloVe）通过神经网络等技术，生成低维稠密的词向量，大大降低了数据稀疏性。
- 忽略词序：
 - 共现矩阵无法捕捉词序信息，而词序对于语义理解至关重要。
 - 直接学习词向量的模型，例如 Word2Vec 中的 CBOW 和 Skip-gram 模型，能够考虑词语的上下文信息，从而更好地捕捉语义关系。
- 计算效率：
 - 共现矩阵的计算量很大，且新增词汇需要重新计算。
 - 直接学习词向量的模型通常采用在线学习的方式，能够高效地处理大规模语料库，并适应动态变化的词汇。

2. 更好地捕捉语义信息：

- 深层语义：
 - SVD 只能捕捉浅层的线性关系，而词语的语义关系往往是非线性的。
 - 神经网络模型能够学习更复杂的非线性关系，从而生成更准确的词向量，更好地表示词语的深层语义。
- 上下文学习：
 - 直接学习词向量的模型通过分析词语的上下文信息，能够更好地理解词语的用法和含义。
 - 例如，Word2Vec 能够学习到“国王”和“女王”等词语之间的关系，因为它们经常出现在相似的上下文中。

3. 更广泛的应用：

- 直接学习的词向量在各种自然语言处理任务中表现出色，例如：
 - 文本分类
 - 情感分析
 - 机器翻译
 - 问答系统
- 这些任务需要对词语的语义有深入的理解，而直接学习的词向量能够提供更丰富的语义信息。

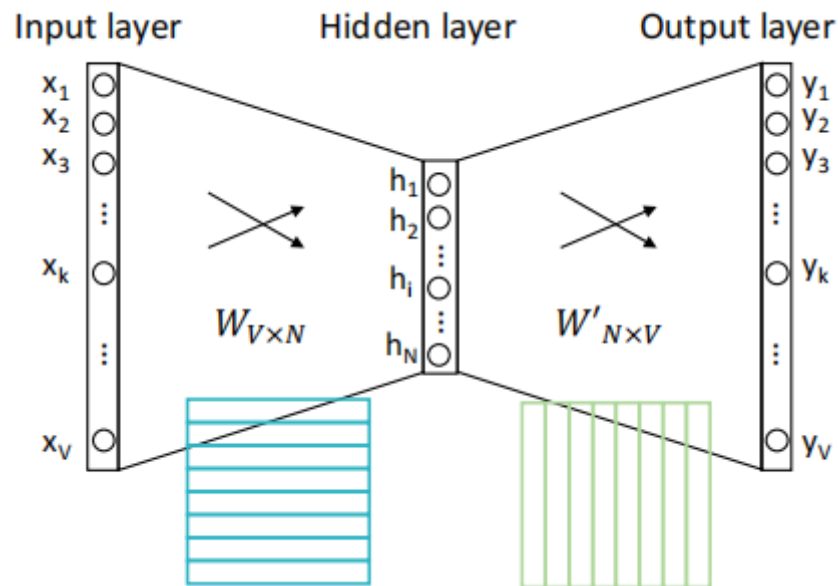
2.1.9 Word2Vec

Word2Vec 模型的基本步骤

1. 准备一个大的语料库：
 - Word2Vec 模型需要大量的文本数据进行训练，以便学习词语之间的语义关系。
2. 词汇表中的每个词都有对应的向量表示：
 - 模型的目标是为每个词汇生成一个低维的稠密向量，这个向量能够捕捉词语的语义信息。
3. 遍历语料库中每篇文章的每个位置，每个位置都有中心词 c 和对应的一些上下文词 o ：
 - Word2Vec 模型通过分析词语的上下文来学习词向量。
 - 对于语料库中的每个词，模型会考虑其周围的词语作为上下文。
4. 用 c 和 o 的词向量的内积来计算给定 c 预测出 o (或相反) 的概率：
 - Word2Vec 模型使用词向量的内积来衡量词语之间的相似度。
 - 模型的目标是最大化给定中心词预测上下文词的概率，或者反过来。
5. 持续优化调整词向量使得概率最大化：
 - 模型通过梯度下降等优化算法，不断调整词向量，使得预测概率最大化。
 - 经过训练，词向量能够更好地捕捉词语之间的语义关系。

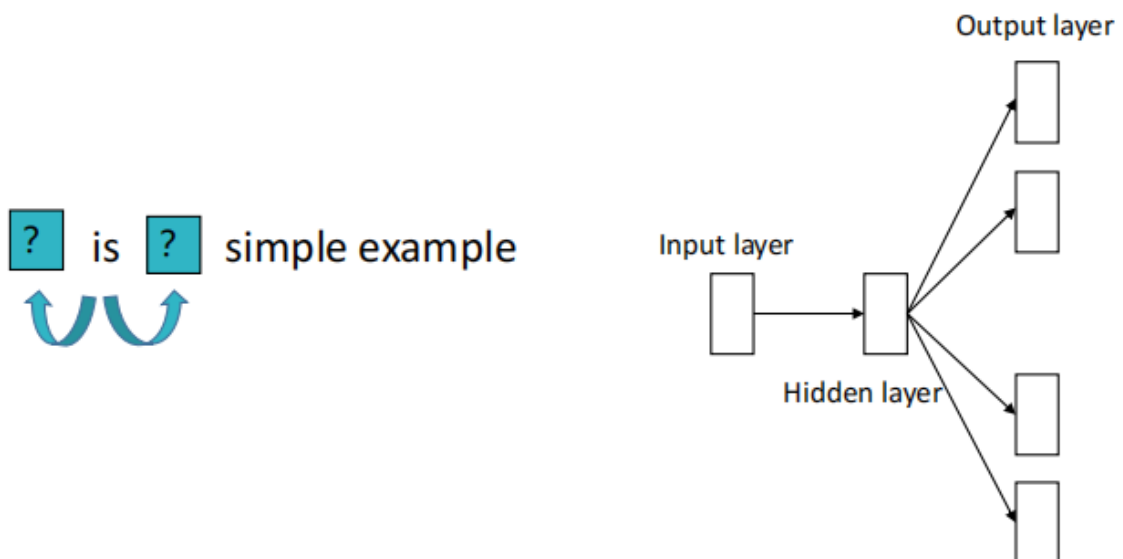
图示解释

- Input layer (输入层):
 - 输入层接收中心词或上下文词的 One-Hot 编码。
- Hidden layer (隐藏层):
 - 隐藏层将 One-Hot 编码转换为低维的词向量。
- Output layer (输出层):
 - 输出层预测上下文词或中心词的概率分布。
- $W_{V \times N}$ 和 $W'_{N \times V}$:
 - 这两个矩阵是模型的权重矩阵，它们在训练过程中被不断调整，以优化词向量。



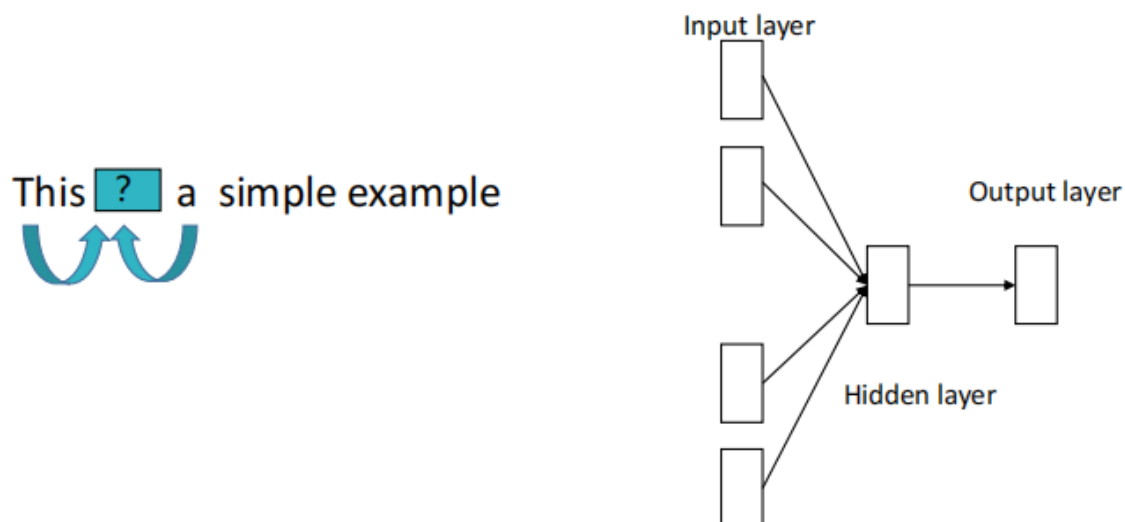
2.1.10 Word2Vec: Skip-gram (SG)

● 用中心词来预测周围的词。在skip-gram中，会利用周围的词的预测结果情况，使用梯度下降来不断的调整中心词的词向量，最终所有的文本遍历完毕之后，也就得到了文本所有词的词向量。

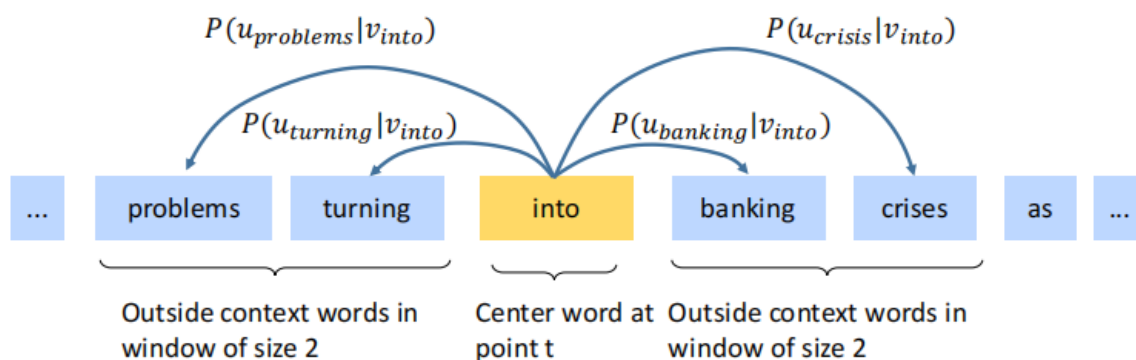


2.1.11 Word2Vec: Continuous Bag of Words (CBOW)

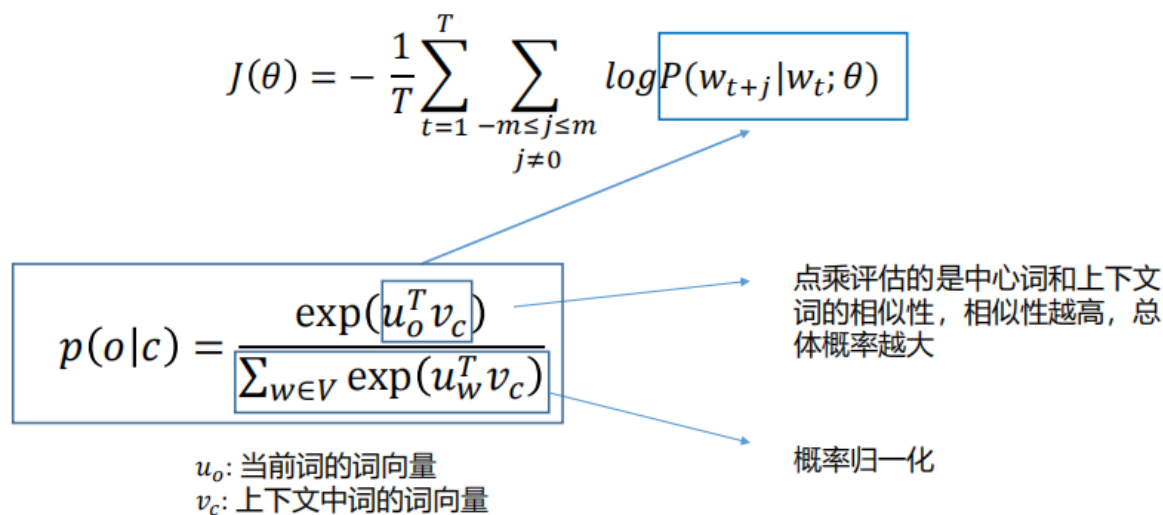
● 用周围词预测中心词，利用中心词的预测结果情况，使用梯度下降等方法，不断的去调整周围词的向量。当训练完成之后，每个词都会作为中心词，把周围词的词向量进行了调整，这样也就获得了整个文本里面所有词的词向量。



2.1.12 Word2Vec - Skip-gram 示例



2.1.13 Word2Vec: 损失函数



这张图片展示了 Word2Vec 模型（特别是 Skip-gram 模型）的目标函数和概率计算方法。

以下是对图片内容的详细解释：

1. 目标函数 $J(\theta)$:

- $J(\theta)$:
 - 表示模型的损失函数或目标函数。
 - 目标是最小化这个函数，从而优化模型的参数 θ (θ 代表词向量)。
- $-(1/T) \sum \sum \log P(w_{t+j} | w_t; \theta)$:
 - 这是一个求平均负对数似然的表达式。
 - T : 表示语料库中的总词数。
 - $\sum (t=1 \text{ to } T)$: 表示对语料库中的每个词进行求和。
 - $\sum (-m \leq j \leq m, j \neq 0)$: 表示对中心词 w_t 周围窗口内的每个上下文词 w_{t+j} 进行求和。
 - $\log P(w_{t+j} | w_t; \theta)$: 表示给定中心词 w_t ，预测上下文词 w_{t+j} 的概率的对数。
 - θ : 表示模型的参数，即词向量。
- 简单来说，目标函数 $J(\theta)$ 的目的是最大化给定中心词预测上下文词的概率。

2. 概率计算 $p(o|c)$:

- $p(o|c)$:
 - 表示给定中心词 c ，预测上下文词 o 的概率。
- $\exp(u_o^T v_c) / \sum_{w \in V} \exp(u_w^T v_c)$:
 - 这是一个 Softmax 函数，用于计算概率。
 - u_o : 表示上下文词 o 的词向量。
 - v_c : 表示中心词 c 的词向量。
 - u_w : 表示词汇表 V 中任意词 w 的词向量。
 - $u_o^T v_c$: 表示 u_o 和 v_c 的点积，用于评估中心词和上下文词的相似性。
 - $\sum_{w \in V} \exp(u_w^T v_c)$: 表示对词汇表 V 中所有词的相似性进行求和，用于概率归一化。
- 点积评估相似性:
 - 点积 $u_o^T v_c$ 评估的是中心词 v_c 和上下文词 u_o 的相似性。
 - 相似性越高， $p(o|c)$ 的值就越大。
- 概率归一化:
 - 分母 $\sum_{w \in V} \exp(u_w^T v_c)$ 用于将概率归一化，使得所有上下文词的概率之和为 1。

• 负对数似然（交叉熵）

• 概率越大，损失越小

2.1.14 Word2Vec 问题与改进

$$p(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

词汇表中每个词都要算一遍，导致训练太慢

● 优化方法：

- Hierarchical softmax
- Negative sampling

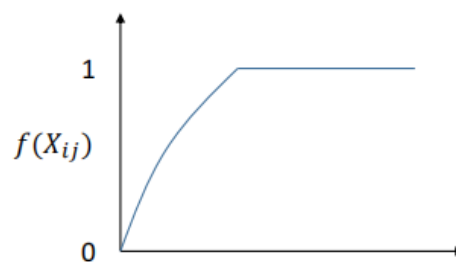
2.1.15 GloVe

● GloVe (Global Vectors for Word Representation)：通过将全局的词共现信息和局部上下文信息结合起来，通过矩阵分解的方式学习词向量。

$$J(\theta) = \frac{1}{2} \sum_{i,j=1}^W f(X_{ij})(u_i^T v_j - \log X_{ij})^2$$

$$X_{final} = U + V$$

x 是共现矩阵



2.1.16 FastText

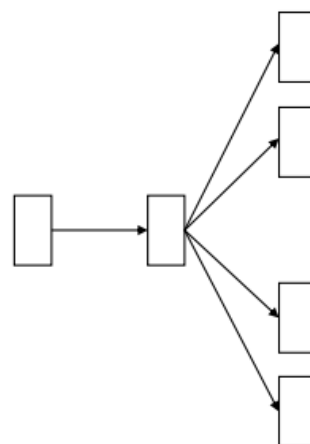
● FastText：基于词袋模型 (Bag of Words) 的文本表示方法，通过将词分解为子词 (subwords) 并将子词的向量表示相加来学习词向量，充分利用字符N-gram信息。

Apple (2-gram) = <ap, pp, pl, le>



训练过程中学习到的

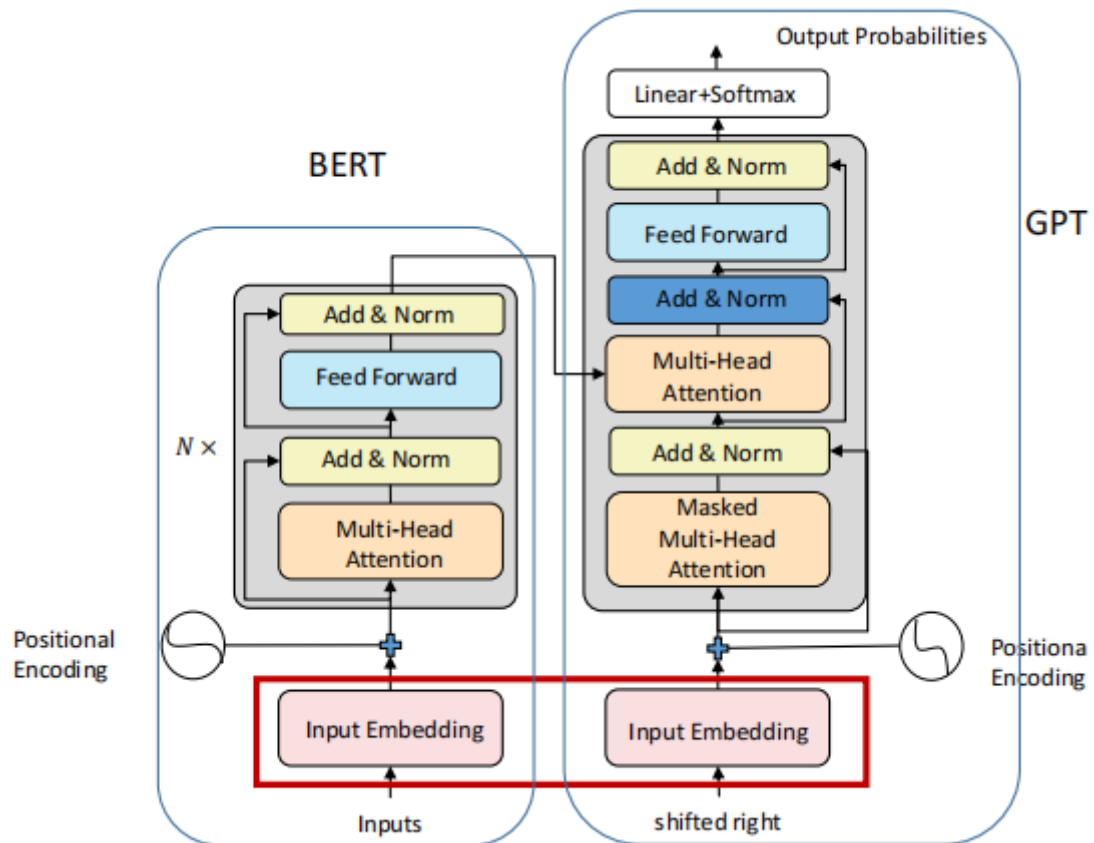
ap	[-0.3, 0.2, 0.5]
pp	[0.4, -0.1, 0.6]
pl	[0.2, 0.3, -0.4]
le	[-0.1, 0.5, 0.2]
apple	[0.2, 0.9, 0.9]



• 假设Apple前后不包含开始和结束，若包含则为<a, a, pp, pl, le, e>

2.1.17 Transformer-based模型

- Transformer-based 模型：如 BERT、GPT等，从大规模文本数据中学习词向量表示。



- K,V 来自编码器的输出， Q解码器的输出
- 预训练（Word2vec）和随机初始化（Transformer-based）会随着任务学习的过程而不断变化

2.1.18 词向量评估

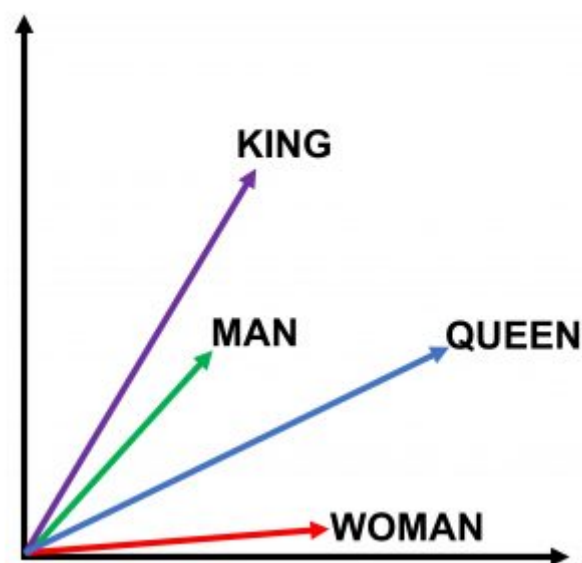
- 内部评估

- 相似度任务：计算两个词的相似度。比如数据集 wordsim353。
- 类比任务：经典的 King - Queen = Man - Woman 。

- 外部评估

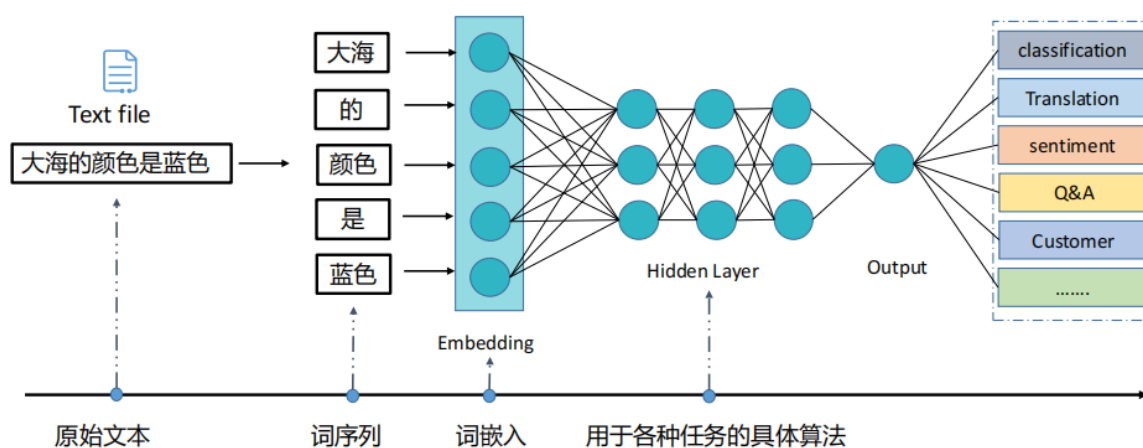
- 以**实际任务的指标**来评价词向量的好坏。如命名实体识别、机器翻译、文本分类等。

$$\text{King} - \text{Man} + \text{Woman} = \text{Queen}$$



2.1.19 下游任务

- 有了词向量后，能做更多下游任务



- 好的词向量可以助力下游任务，使得下游任务做得更加出色。

2.2 文本分类

2.2.1 文本分类

定义：

- 输入：
 - 一篇文章 d
 - 固定的类别集合 $C = \{C_1, C_2, \dots, C_j\}$
- 输出：
 - 预测出的该文章所属的类别 $c \in C$

应用：

- 情感分析
- 意图识别
- 内容审核
-

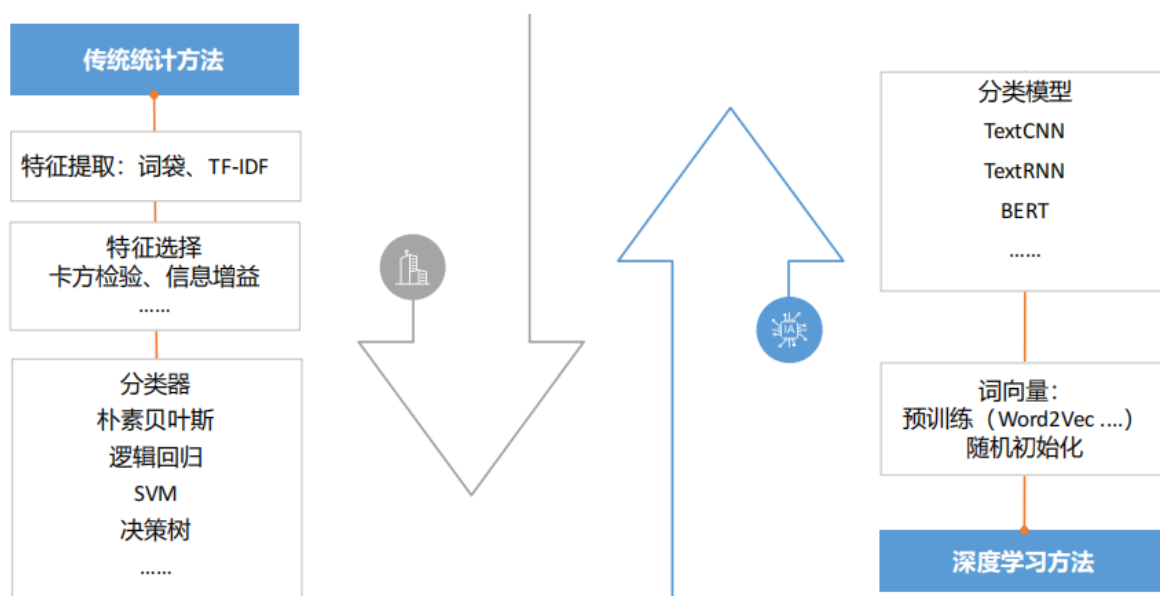
input: 今天发了工资，我很开心！



$C = \{0: \text{positive}, 1: \text{negative}\}$

output: 0 (positive)

2.2.2 文本分类方法



- 文本分类的方法包括传统统计方法和深度学习方法，是大模型的IP

2.2.3 特征提取 - 词袋

- 词袋模型 (Bag of Words, BOW) 是一种简单的文本向量化的一种方法，它不考虑句子中单词的顺序，只考虑词表 (vocabulary) 中单词在这个句子中的出现次数。

语料库:

我 爱 上海
我 爱 杭州
我 爱 北京 天安门



	上海	北京	天安门	我	杭州	爱
d1	1	0	0	1	0	1
d2	0	0	0	1	1	1
d3	0	1	1	1	0	1

2.2.4 特征提取 - TF-IDF

- TF-IDF (词频-逆文档频率) 向量化相比于词袋向量化，更好的考虑了句子中每个词的重要程度。

$$w_{i,d} = tf_{i,d} \times \log\left(\frac{n}{1 + df_i}\right)$$

语料库:

我 爱 上海
我 爱 杭州
我 爱 北京 天安门



	上海	北京	天安门	我	杭州	爱
d1	0.767	0.0	0.0	0.453	0.0	0.453
d2	0.0	0.0	0.0	0.453	0.767	0.453
d3	0.0	0.608	0.608	0.359	0.0	0.359

TF-IDF (Term Frequency-Inverse Document Frequency, 词频-逆文档频率) 是一种用于信息检索与文本挖掘的常用加权技术。它的主要目的是评估一个词语在一个文档集或语料库中的重要程度。

TF-IDF 的核心思想:

- 词频 (TF):
 - 表示一个词语在给定文档中出现的频率。
 - 如果一个词语在文档中出现的次数越多, 则认为它在该文档中越重要。
- 逆文档频率 (IDF):
 - 表示一个词语在整个文档集中出现的普遍程度。
 - 如果一个词语在很多文档中都出现, 则认为它很常见, 不具有区分度, 其 IDF 值会较低。
 - 如果一个词语只在少数文档中出现, 则认为它比较稀有, 具有区分度, 其 IDF 值会较高。
- TF-IDF 值:
 - 将词频 (TF) 和逆文档频率 (IDF) 相乘, 得到 TF-IDF 值。
 - TF-IDF 值越高, 表示该词语在给定文档中越重要, 同时在整个文档集中又相对稀有。

TF-IDF 的作用:

- 关键词提取:
 - 通过计算 TF-IDF 值, 可以找出文档中的关键词, 这些关键词能够代表文档的主要内容。
- 文本分类:
 - TF-IDF 可以作为文本分类的特征, 将文本转换为数值向量, 用于训练分类模型。
- 信息检索:
 - 在搜索引擎中, TF-IDF 可以用于评估文档与用户查询之间的相关程度, 从而提高搜索结果的质量。

2.2.5 特征选择

● 词袋向量化或TF-IDF向量化后的特征通常都有高维稀疏的问题, 直接给分类器训练通常效果并不好, 过高的数据维度也影响训练和预测的速度, 因此, 通常会使用特征选择的方法过滤一些无效特征。文本分类中常见的特征选择方法有:

- ☐ 卡方检验
- ☐ 互信息
- ☐ 信息增益
- ☐

2.2.6 卡方检验

● 源于统计学的卡方分布 (chi-square), 从 (类, 词项) 相关表出发, 考虑每一个类和每一个词项的相关情况, 度量两者 (特征和类别) 独立性的缺乏程度, 卡方越大, 独立性越小, 相关性越大。

2.2.7 卡方值计算

	<i>Term = jaguar</i>	<i>Term ≠ jaguar</i>	
<i>Class = auto</i>	2 (0.25)	500 (502)	expected: f_e
<i>Class ≠ auto</i>	3 (4.75)	9500 (9498)	observed: f_o

Expected value, for instance in the up-left cell is:

$P(c=auto) * P(T=jaguar) * \text{\#of cases} = 502/10005 * 5/10005 * 10005 = 0.2508$

$$\chi^2(j,a) = \sum (O - E)^2 / E = (2 - .25)^2 / .25 + (3 - 4.75)^2 / 4.75 + (500 - 502)^2 / 502 + (9500 - 9498)^2 / 9498 = 12.9 (p < .001)$$

• O是观测值，E是期望值

2.2.8 分类器

● 在完成特征提取后，传统的机器学习分类算法都可以用来对文本进行分类：

- ☐ 朴素贝叶斯
- ☐ 逻辑回归
- ☐ SVM
- ☐ 决策树
- ☐ 集成算法：随机森林，XGBoost.....

2.2.9 朴素贝叶斯

- 朴素贝叶斯分类方法 (Naïve Bayes) 是利用贝叶斯定理来预测一个未知类别的样本属于各个类别的可能性，选择其中可能性最大的一个类别作为该样本的最终类别。
- 给定文档 d 和类别集合 $C = \{c_1, c_2\}$, x 是文档 d 中的单词，计算文档所属类别：

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(c|d)$$

$$= \operatorname{argmax}_{c \in C} \frac{P(d|c)P(c)}{P(d)}$$

贝叶斯公式

$$= \operatorname{argmax}_{c \in C} P(d|c)P(c)$$

$$= \operatorname{argmax}_{c \in C} P(x_1, x_2, \dots, x_n|c)P(c)$$

$$= \operatorname{argmax}_{c \in C} P(c) \prod_{x \in X} P(x|c)$$

2.2.10 垃圾邮件识别

输入: 黄金地段, 快速抢购
输出: 垃圾邮件 or 正常邮件?

$$\operatorname{argmax}_{c \in C} P(c) \prod_{x \in X} P(x|c)$$

$$p1 = p(\text{垃圾邮件}) \times p(\text{黄金}|\text{垃圾邮件}) \times p(\text{地段}|\text{垃圾邮件}) \times p(\text{快速}|\text{垃圾邮件}) \times p(\text{抢购}|\text{垃圾邮件})$$

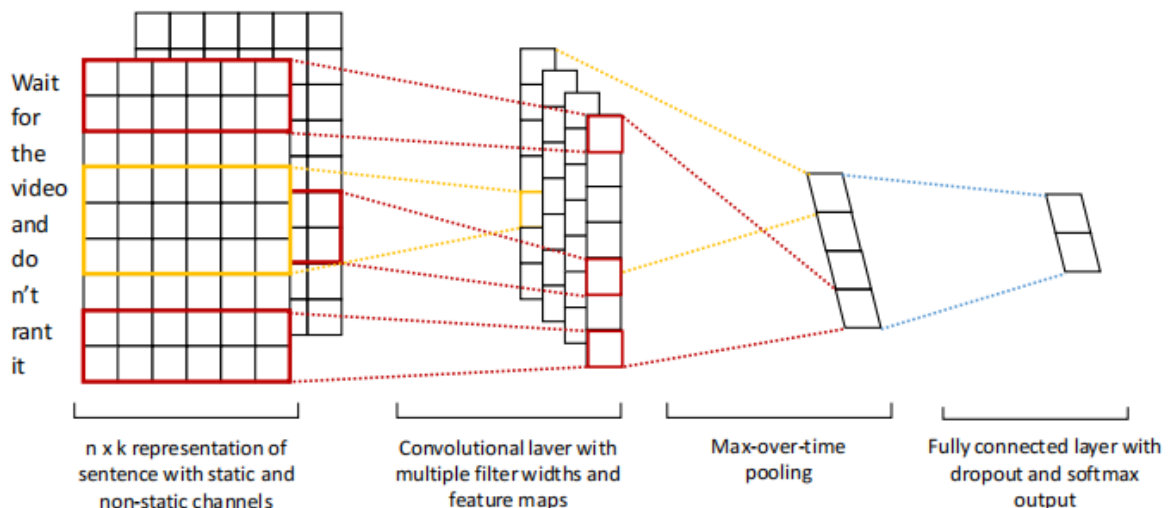
$$p2 = p(\text{正常邮件}) \times p(\text{黄金}|\text{正常邮件}) \times p(\text{地段}|\text{正常邮件}) \times p(\text{快速}|\text{正常邮件}) \times p(\text{抢购}|\text{正常邮件})$$

$$\text{class} = \begin{cases} \text{垃圾邮件} & \text{if } p1 > p2 \\ \text{正常邮件} & \text{if } p1 < p2 \end{cases}$$

$$p(\text{抢购}|\text{垃圾邮件}) = \frac{\text{count}(\text{垃圾邮件中 '抢购' 出现的次数})}{\text{count}(\text{垃圾邮件中总的词数})}$$

2.2.11 深度学习方法 - TextCNN

- TextCNN是一种使用卷积神经网络对文本进行分类的模型。
- 文本在经过词嵌入后被送入神经网络, 与CNN不同的是, TextCNN的卷积操作用在一维数据上。
- TextCNN提取的特征类似N-gram模型, 是词的局部关系。

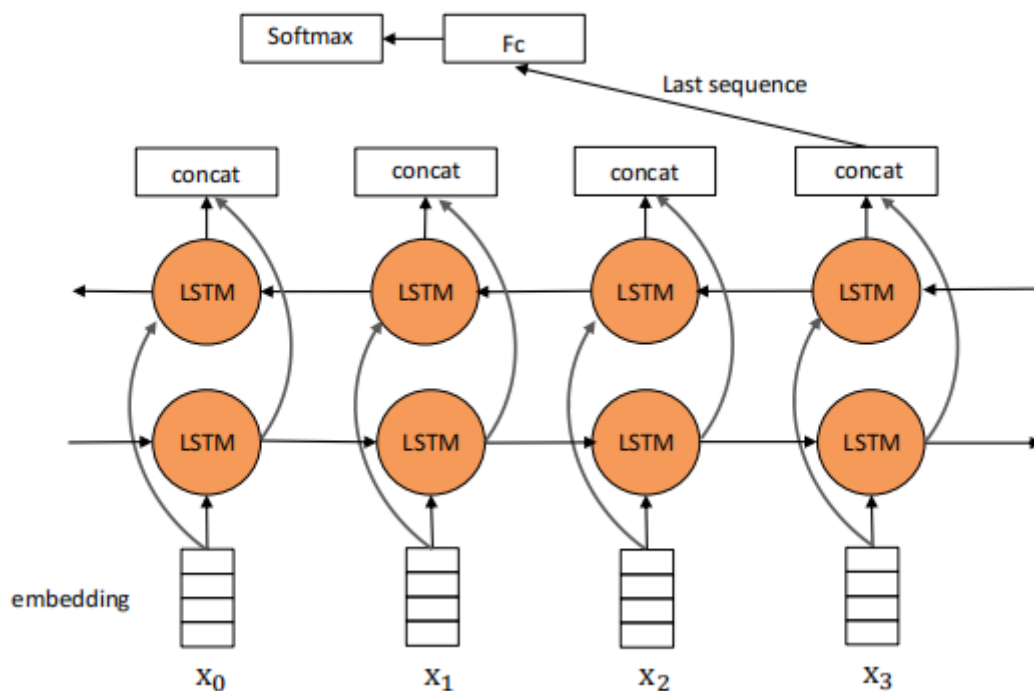


- TextCNN 概述
 - TextCNN 是一种利用卷积神经网络 (CNN) 进行文本分类的深度学习模型。
 - 它将文本经过词嵌入 (word embedding) 转换为向量表示, 然后输入到 CNN 中进行特征提取和分类。
 - 与传统的图像 CNN 不同, TextCNN 的卷积操作是在一维的文本数据上进行的。
- 特征提取
 - TextCNN 提取的特征类似于 N-gram 模型, 能够捕捉词语之间的局部关系。
 - N-gram 模型: 可以理解为连续出现的n个词的序列, 比如, 今天天气很好, 可以分成“今天天气”, “天气 很好”。
- 网络结构
 - TextCNN 的网络结构相对简单, 主要包括:
 - 一层卷积层 (Convolutional layer): 用于提取文本特征。
 - 一层最大池化层 (Max-over-time pooling): 用于提取最重要的特征。
 - 全连接层 (Fully connected layer): 用于将提取的特征映射到分类结果。
 - softmax 输出层: 用于输出分类结果。
 - 可以参考图片中的结构。
- 输入数据
 - 与图像 CNN 的二维输入不同, TextCNN 的输入是一维的文本序列。
 - 虽然词嵌入将文本转换为二维的词向量矩阵, 但卷积操作是在词向量的维度上进行的, 而不是像图像那样在空间维度上滑动。
 - 例如: 对于“今天”这个词, 即使词向量为[0, 0, 0, 0, 1], 如果用1*2的卷积核从左到右滑动, 得到的[0,0],[0,0],[0,0],[0,1]这几个向量, 表达的都是“今天”这个词, 没有实际意义。
- 参考链接
 - <https://arxiv.org/abs/1408.5882>

关键点

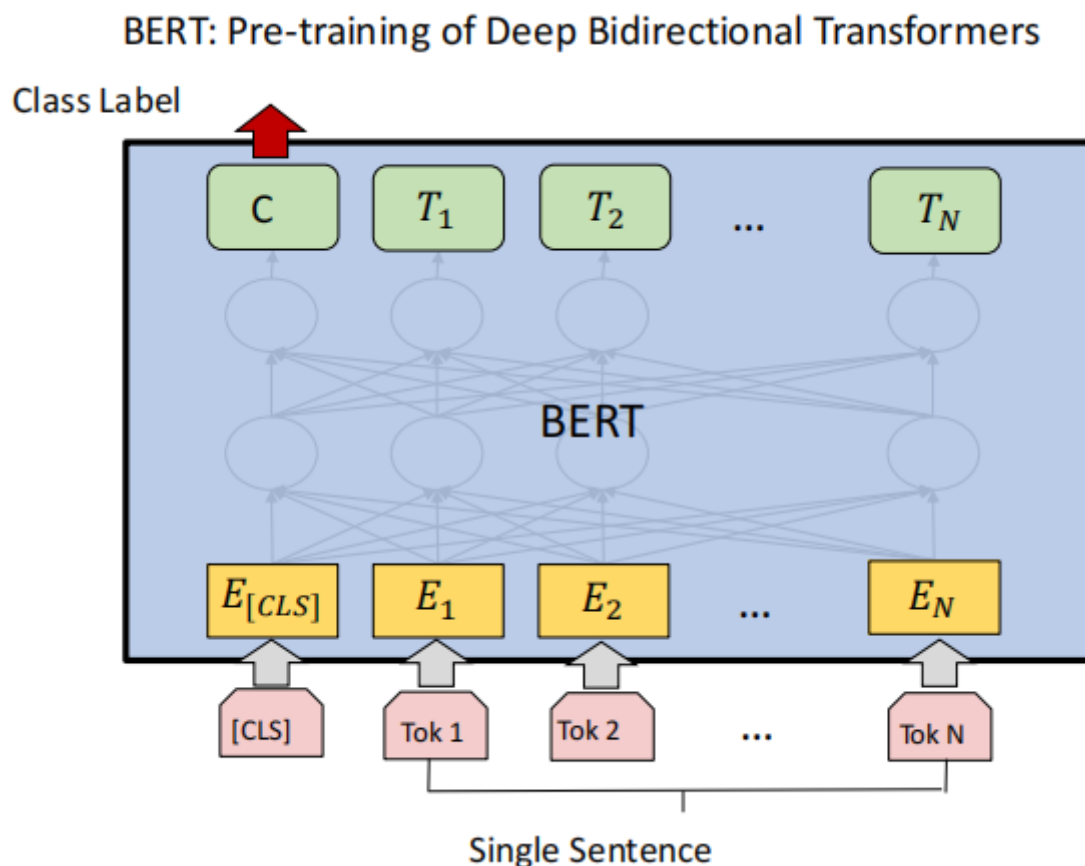
- TextCNN 借鉴了图像 CNN 的思想, 但针对文本数据的特点进行了改进。
- 它能够有效地提取文本的局部特征, 并用于文本分类任务。
- 它和传统的CNN最大的不同, 就是输入的数据, 传统CNN是二维图片, 而TextCNN是一维的文本。

2.2.12 深度学习方法 - TextRNN



- 一般取前向/反向LSTM在最后一个时间步长上隐藏状态，然后进行拼接，在经过一个softmax层(输出层使用softmax激活函数)进行一个多分类；或者取前向/反向LSTM在每一个时间步长上的隐藏状态，对每一个时间步长上的两个隐藏状态进行拼接，然后对所有时间步长上拼接后的隐藏状态取均值，再经过一个softmax层(输出层使用softmax激活函数)进行一个多分类(2分类的话使用sigmoid激活函数)。

2.2.13 深度学习方法 - BERT



- BERT $[CLS]$ 标志位对应的输出是整个句子的向量编码，可以直接softmax实现分类

2.2.14 评估指标

- **正确率 (accuracy):** 判断正确的样本数占有所有样本数的比例。
- **错误率 (error rate):** 判断错误的样本数占有所有样本数的比例。
- **召回率 (recall):**

$$Recall = \frac{\text{正确的目标类判断}}{\text{数据集中目标类的总数}}$$

- **精度 (precision):**

$$Precision = \frac{\text{正确的目标类判断}}{\text{被判断为目标类的总数}}$$

- **F1-score:**

$$F1\text{-score} = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

2.2.15 举例

● 我们训练了一个机器学习的模型用来识别垃圾邮件，现在用200篇邮件文本来验证下模型的性能指标。这200篇文档中，170张是正常邮件，30篇是垃圾邮件。模型的识别结果为160篇是正常邮件，40篇是垃圾邮件。

- **精度 (Precision):**

$$P = \frac{TP}{TP + FP} = \frac{140}{140 + 20} = 87.5\%$$

- **召回率 (Recall):**

$$R = \frac{TP}{P} = \frac{140}{170} = 82.3\%$$

- **准确率 (Accuracy):**

$$ACC = \frac{TP + TN}{P + N} = \frac{140 + 10}{170 + 30} = 75\%$$

- **F1-score:**

$$F1\text{-score} = \frac{2 \times precision \times recall}{precision + recall} = \frac{2 \times 0.875 \times 0.933}{0.875 + 0.933} = 0.848$$

预测 实际	yes	no	合计
yes	140	30	170
no	20	10	30
合计	160	40	200

2.3 序列标注

2.3.1 序列标注

- 序列标注是对输入序列中的每一个元素 x_i 分配一个对应的标签 y_i 的任务。

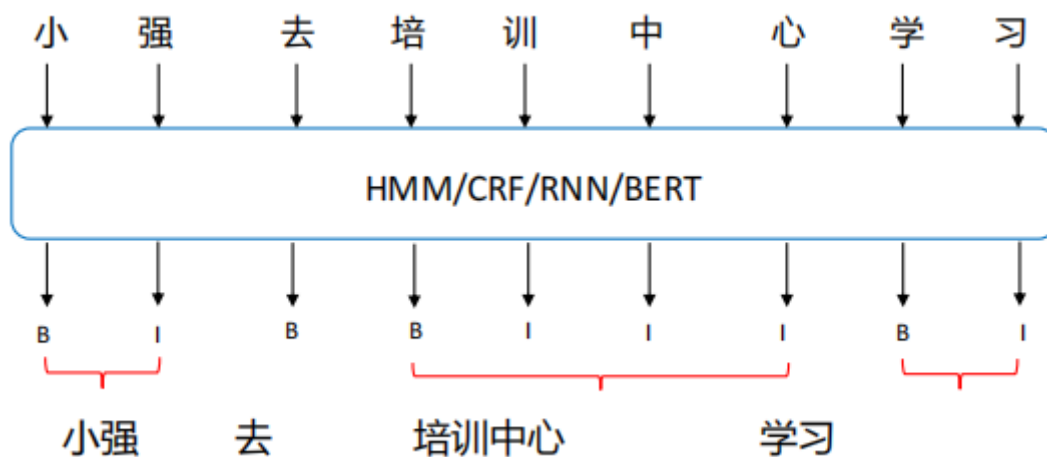
文本分类 ➡ 序列标注



2.3.2 中文分词

● **中文分词 (Chinese Word Segmentation)**：指的是将一个汉字序列切分成一个个单独的词。**分词**就是将连续的字序列按照一定的规范重新组合成词序列的过程。

- 例如：一九九八年/中国/实现/进出口/总值/达/一千零九十八点二亿/美元

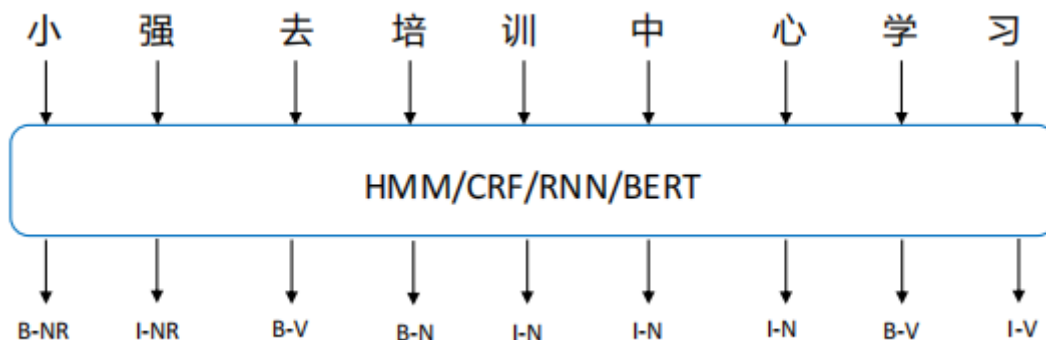


图片中的“B”和“I”是中文分词中常用的**BIO标注法**（Begin, Inside, Outside）的标签。它们用于标记一个词语的开始和中间部分，从而帮助模型区分词语的边界。

- **B (Begin)**: 表示一个词语的开始。
- **I (Inside)**: 表示一个词语的中间部分。

2.3.3 词性标注

● **词性标注 (Part-Of-Speech tagging, POS tagging)**：是指为句子的分词结果中的每个单词标注一个正确的词性的程序，也即确定每个词是名词、动词、形容词或者其他词性的过程。例如：迈向/v 充满/v 希望/n 的/u 新/a 世纪/n。词性标注是很多NLP任务的预处理步骤，如句法分析、信息抽取，经过词性标注后的文本会带来很大的便利性，但不是不可或缺步骤。

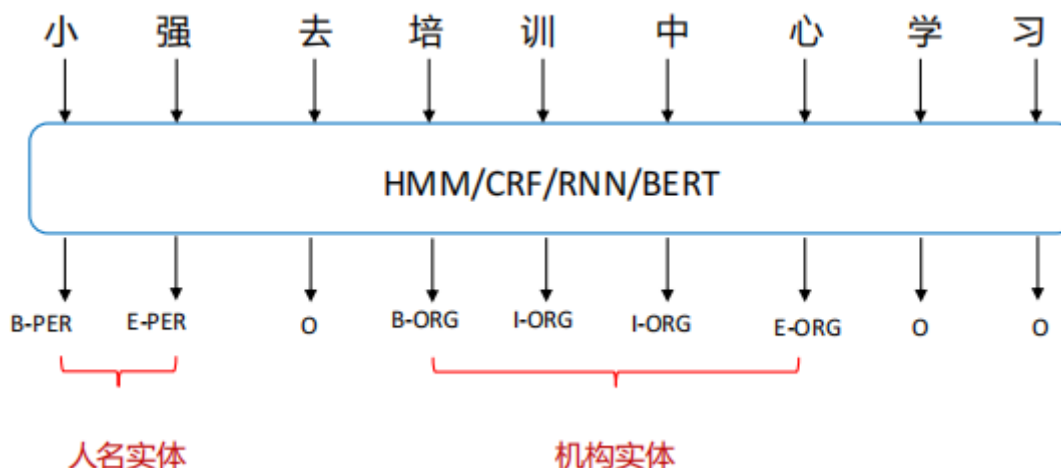


图片中的 "nr"、"n" 和 "v" 是词性标注 (Part-of-Speech tagging, POS tagging) 中常用的标签，它们分别代表：

- **nr**: 人名 (Name Recognition) 。
- **n**: 名词 (Noun) 。
- **v**: 动词 (Verb) 。

2.3.4 命名实体识别

命名实体识别 (Named Entity Recognition, NER) 为自然语言处理 (NLP) 的基础任务之一，其目标是提取文本中的命名实体并对这些实体进行分类，比如人名、地名、机构、时间、货币和百分比等。



PER (Person): 人名实体。

ORG (Organization): 组织机构实体。

B-PER: 人名实体的第一个字。

E-PER: 人名实体的最后一个字。

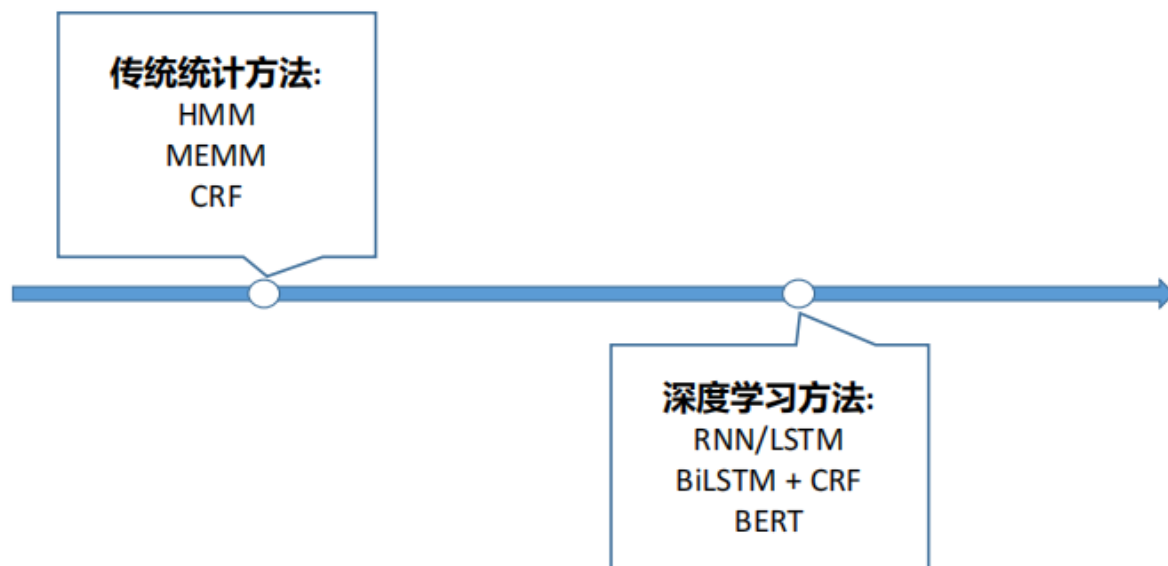
O: 非实体字。

B-ORG: 组织机构实体的第一个字。

I-ORG: 组织机构实体内部的字。

E-ORG: 组织机构实体的最后一个字。

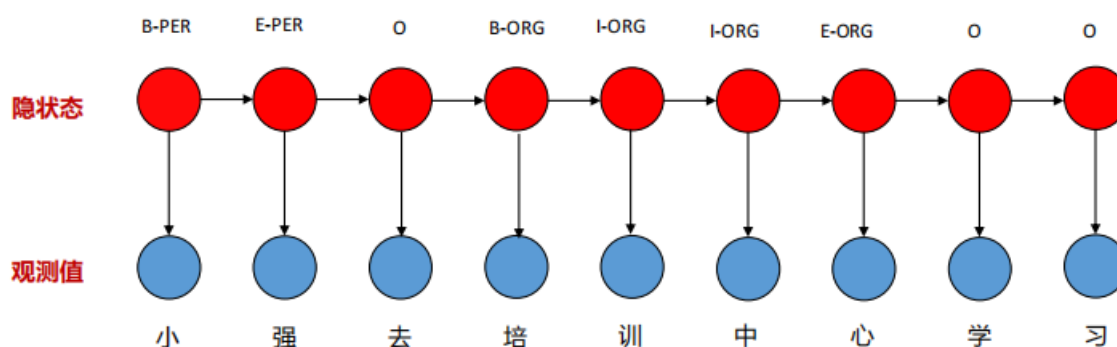
2.3.5 序列标注方法



2.3.6 HMM

● 隐马尔科夫模型（Hidden Markov Models, HMM）有两个假设：

- 隐状态独立假设（齐次马尔科夫假设）：当前状态只与上一状态有关。
- 观测独立假设：当前观测值只由当前隐状态产生，与其他观测值相互独立。



例子：天气预报

假设我们有一个简化的天气预报模型，它只考虑三种天气状态：晴天（Sunny）、阴天（Cloudy）和雨天（Rainy）。我们用 S、C 和 R 分别表示这三种状态。

马尔科夫性质：

我们假设明天的天气只取决于今天的天气，而与过去的天气无关。例如，如果今天是晴天，那么明天下雨的概率可能比较低，但如果今天是雨天，那么明天下雨的概率可能比较高。

状态转移概率矩阵：

我们可以用一个状态转移概率矩阵来表示从一个状态转移到另一个状态的概率。例如：

	明天晴天概率	今天阴天概率	今天雨天概率
今天晴天	0.6	0.3	0.1
今天阴天	0.2	0.5	0.3
今天雨天	0.1	0.4	0.5

这个矩阵表示，如果今天是晴天（S），那么明天的天气：

- 有 60% 的概率是晴天（S）。
- 有 30% 的概率是阴天（C）。
- 有 10% 的概率是雨天（R）。

齐次马尔科夫假设：

现在，我们来解释“状态转移概率在时间上是恒定的”这个概念。

- 齐次马尔科夫假设认为，无论今天是哪一天，这个状态转移概率矩阵都是不变的。
- 也就是说，如果今天是星期一，那么从晴天（S）转移到雨天（R）的概率是 10%。
- 如果今天是星期二，那么从晴天（S）转移到雨天（R）的概率仍然是 10%。
- 如果今天是任何一天，这个转移概率都是不变的。

隐马尔可夫模型（HMM）

隐马尔可夫模型（HMM）的工作原理：**天气预测**。

场景：

假设我们生活在一个小岛上，每天的天气要么是晴天（Sunny），要么是雨天（Rainy）。我们无法直接观测到天气，但我们知道每天海藻的湿度。海藻的湿度有三种状态：干燥（Dry）、潮湿（Damp）和湿透（Soggy）。

HMM 模型：

我们可以使用 HMM 来建模这个天气预测问题。

- 隐藏状态：晴天（Sunny）、雨天（Rainy）
- 可观测状态：干燥（Dry）、潮湿（Damp）、湿透（Soggy）
- 状态转移概率：
 - 表示从一种天气状态转移到另一种天气状态的概率。
 - 例如：
 - 从晴天转移到晴天的概率：0.7
 - 从晴天转移到雨天的概率：0.3
 - 从雨天转移到晴天的概率：0.4
 - 从雨天转移到雨天的概率：0.6
- 观测概率：
 - 表示在给定天气状态下，海藻出现某种湿度的概率。
 - 例如：
 - 在晴天时，海藻干燥的概率：0.6
 - 在晴天时，海藻潮湿的概率：0.3
 - 在晴天时，海藻湿透的概率：0.1

- 在雨天时，海藻干燥的概率：0.1
- 在雨天时，海藻潮湿的概率：0.4
- 在雨天时，海藻湿透的概率：0.5

HMM 的应用：

- 评估问题：
 - 给定 HMM 模型和海藻湿度的观测序列（例如，干燥、潮湿、湿透、干燥...），计算这个观测序列出现的概率。
- 解码问题：
 - 给定 HMM 模型和海藻湿度的观测序列，找到最有可能生成这个观测序列的天气状态序列（例如，晴天、雨天、雨天、晴天...）。
- 学习问题：
 - 给定海藻湿度的观测序列，估计 HMM 模型的参数（状态转移概率和观测概率）。

实际应用：

- 在这个例子中，我们可以使用 HMM 来预测未来的天气，即使我们无法直接观测到天气。
- 通过分析海藻湿度的变化，我们可以推断出最有可能的天气状态序列。

HMM 是一种强大的统计模型，能够有效地处理含有隐含未知参数的时序数据。在天气预测的例子中，HMM 可以帮助我们根据海藻湿度的观测序列，推断出最有可能的天气状态序列。

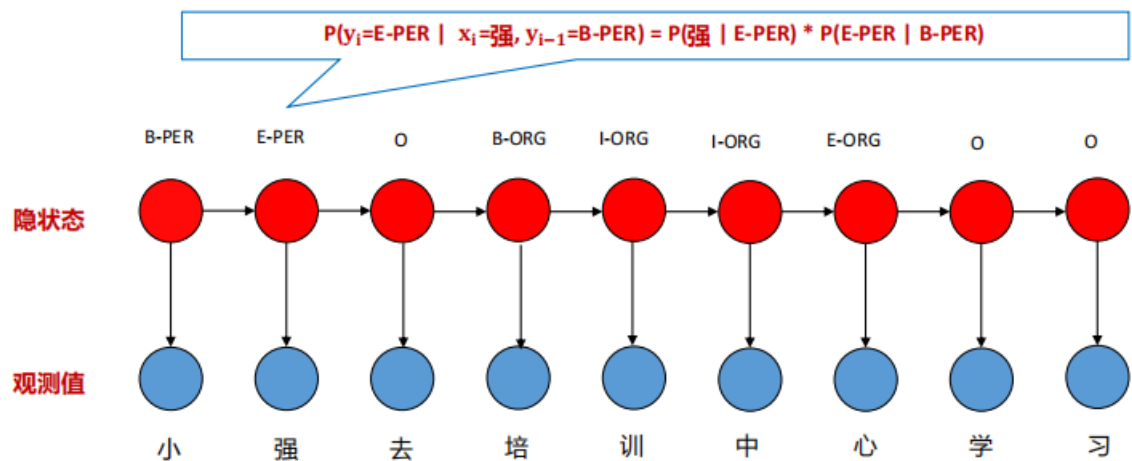
2.3.7 HMM 概率计算

- HMM 搜索使得联合概率 $P(X, Y)$ 最大的隐状态序列 $(y_1, y_2, \dots, y_n)$ 。

$$P(X, Y) = \prod_{i=1}^n p(x_i, y_i)$$

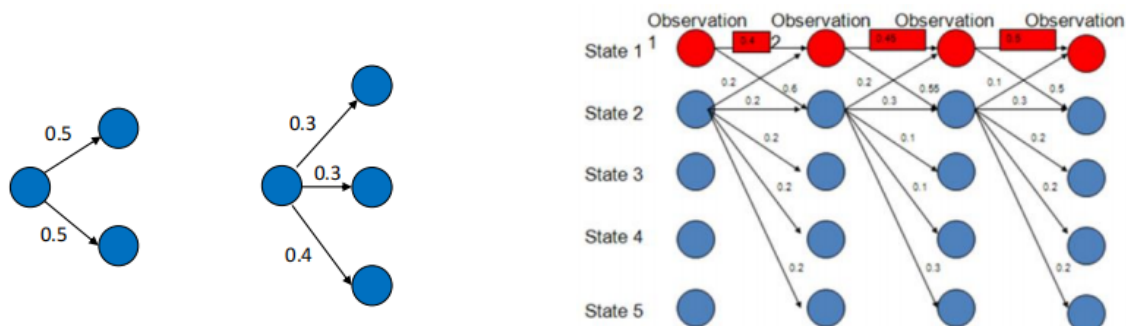
解释：

- **HMM (Hidden Markov Model)**: 隐马尔可夫模型，是一种统计模型，用于描述由隐藏的马尔可夫过程生成可观测序列的过程。
- **X**: 可观测序列，例如：词语序列、语音信号。
- **Y**: 隐状态序列，例如：词性序列、音素序列。
- $P(X, Y)$: 可观测序列 X 和隐状态序列 Y 的联合概率。
- y_i : 隐状态序列 Y 中的第 i 个隐状态。
- x_i : 可观测序列 X 中的第 i 个观测值。
- $p(x_i, y_i)$: 第 i 个观测值 x_i 和第 i 个隐状态 y_i 的联合概率。
- $\prod$: 连乘符号，表示将所有 $p(x_i, y_i)$ 相乘。



2.3.8 HMM问题 - 标签偏置

● HMM的状态独立假设，使得在计算标签转移概率时在局部归一化转移概率，这使得分支少的状态比分支多的状态有着更大的概率，使得算法倾向于选择分支少的状态。

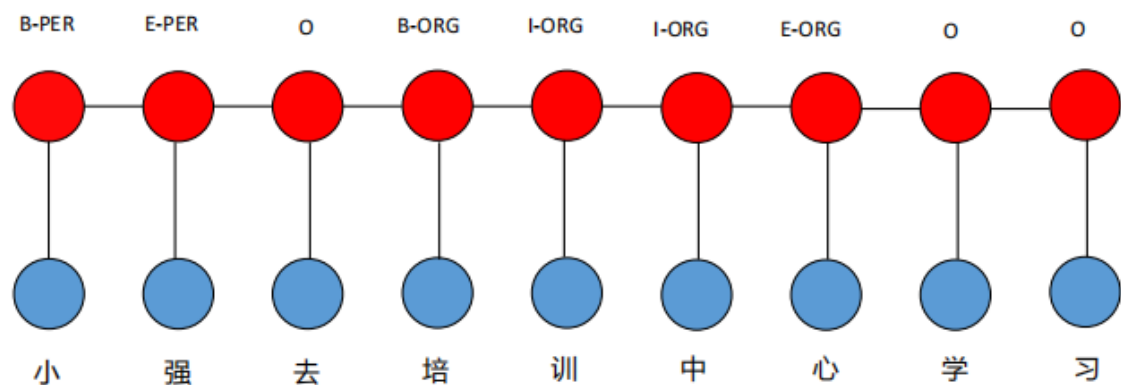


2.3.9 HMM问题 - 上下文缺失

● HMM的观测独立假设，使得算法当前时间步的概率只与当前观测有关，无法利用周围的上下文特征，使得HMM的性能非常受限。

2.3.10 CRF

● 条件随机场 (Conditional Random Field, CRF)，取消了HMM的两个独立假设，把标签转移和上下文输入都当做全局特征之一，在全局进行概率归一化，解决了HMM的标签偏置和上下文特征缺失问题。



2.3.11 CRF - 概率计算

定义特征

$$f_k(y_{i-1}, y_i, x, i), k = 1, 2, \dots, K$$

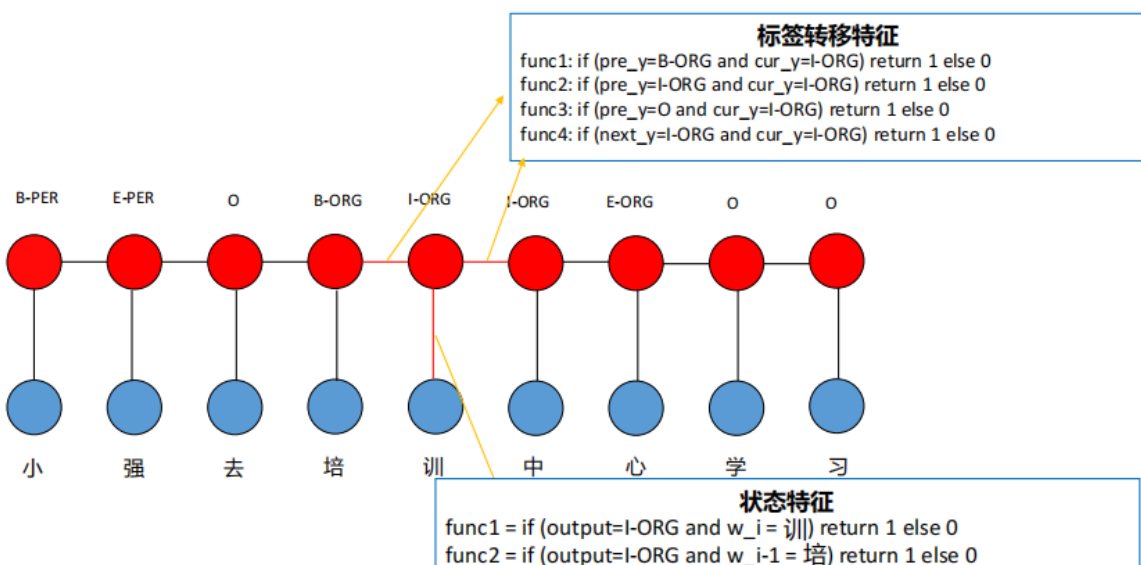
计算score

$$score(y|x) = \sum_{j=1}^m \sum_{i=1}^n \lambda_j f_j(x, i, y_i, y_{i-1})$$

概率归一化

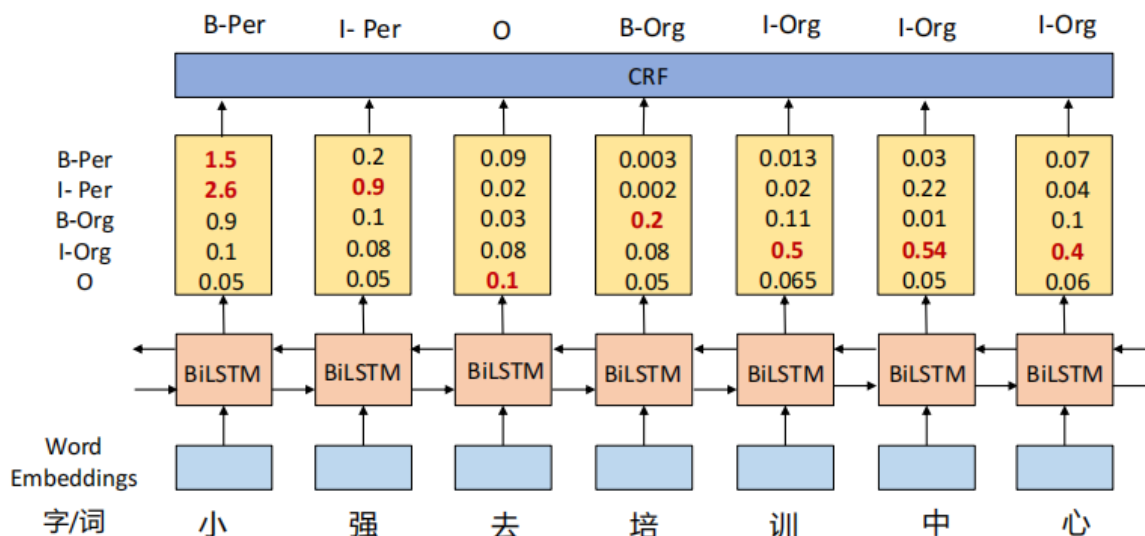
$$P(y|x) = \frac{\exp(score(y|x))}{\sum_y \exp(score(y|x))}$$

2.3.12 CRF - 特征



2.3.13 BiLSTM + CRF

● BiLSTM+CRF里的CRF并不是指传统的CRF算法，这里只是用于学习标签转移特征并进行全局维度解码的一个神经网络层。



• 因为BiLSTM针对每个单词的输出是标签得分，对于每个单词，我们可以选择最高得分的标签作为预测结果。

• 但是大多数情况下是不能获得正确标签的。

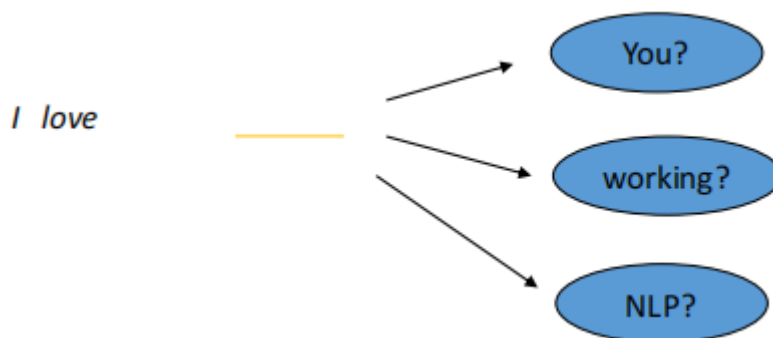
• CRF层可以对最终的约束标签添加一些约束条件，从而保证预测标签的有效性。而这些约束条件是CRF层自动从训练数据中学到。

2.4 语言模型与文本生成

2.4.1 语言模型

什么是语言模型？

□ 给定一些词序列，预测最有可能的下一个词是什么。



□ 更正式的定义：给定词序列 $w_1, w_2, \dots, w_{i-1}$ ，计算下一个词 w_i 的概率分布：

$$P(w_i | w_1, w_2, \dots, w_{i-1})$$

w_i 可能是词汇表中任何一个词。

也可认为语言模型是对一段文本进行概率计算的模型：

$$P(w_1, w_2, \dots, w_m) = P(w_1)P(w_2|w_1) \dots P(w_i|w_1, w_2, \dots, w_{i-1}) \dots P(w_m|w_1, w_2, \dots, w_{m-1})$$

解释：

- **语言模型 (Language Model)**：是一种用于计算文本序列概率的统计模型。
- w_i ：词序列中的第 i 个词。
- $P(w_i|w_1, w_2, \dots, w_{i-1})$ ：在给定前 $i-1$ 个词的情况下，第 i 个词 w_i 出现的条件概率。
- $P(w_1, w_2, \dots, w_m)$ ：词序列 $w_1, w_2, \dots, w_m$ 出现的联合概率。
- $\prod$ ：连乘符号，表示将所有条件概率相乘。

2.4.2 N-Gram语言模型

- 利用 **n 元模型** (n -gram model) 估算条件概率，即忽略距离大于等于 n 的上文词的影响。因此，若用频数计数的比例来计算 n 元条件概率可表示为：

$$\begin{aligned} P(w_i|w_1, w_2, \dots, w_{i-1}) &\approx P(w_i|w_{i-(n-1)}, w_{i-(n-2)}, \dots, w_{i-1}) \\ &= \frac{\text{count}(w_{i-(n-1)}, w_{i-(n-2)}, \dots, w_{i-1}, w_i)}{\text{count}(w_{i-(n-1)}, w_{i-(n-2)}, \dots, w_{i-1})} \end{aligned}$$

- 当 $n = 1$ 时称为**一元模型**,

$$P(w_1, w_2, \dots, w_m) = P(w_1)P(w_2) \dots P(w_m)$$

- 当 $n = 2$ 时称为**二元模型**,

$$P(w_1, w_2, \dots, w_m) = P(w_1)P(w_2|w_1) \dots P(w_m|w_{m-1})$$

解释：

- **n-gram 模型**：一种统计语言模型，用于预测给定前 $n-1$ 个词的情况下，下一个词出现的概率。
- **条件概率**：给定某个事件发生的情况下，另一个事件发生的概率。
- **count()**：统计词序列在语料库中出现的次数。
- **一元模型 (unigram model)**：假设每个词出现的概率相互独立。
- **二元模型 (bigram model)**：假设当前词出现的概率只与前一个词有关。

例如：

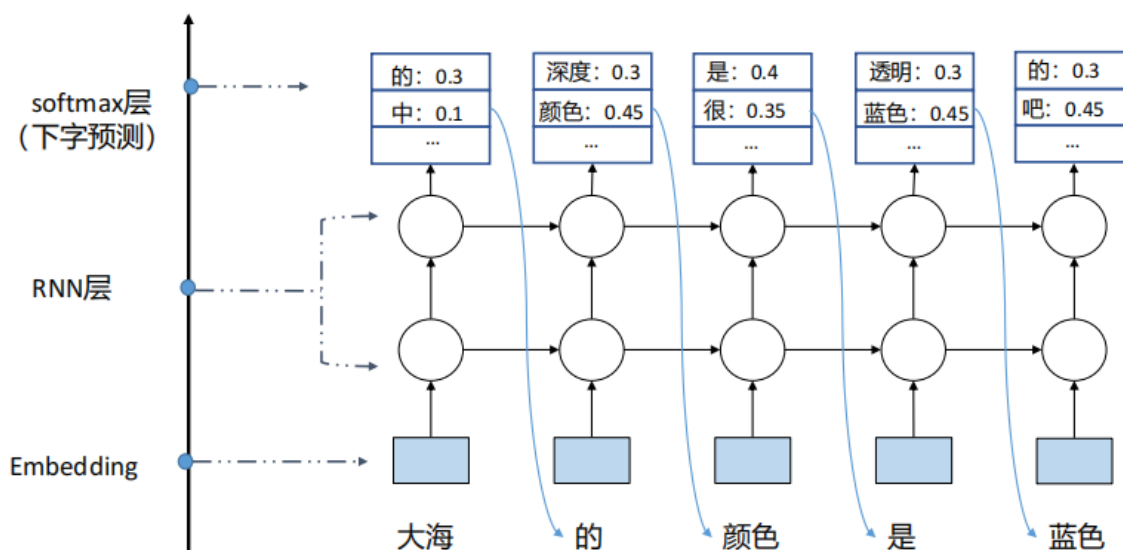
```
<s> I am Lily </s>
<s> Lily I am </s>
<s> I do not like green eggs and ham </s>
```

```
p(I | <s>) = 2/3 = 0.667
p(am | I) = 2/3 = 0.667
p(</s> | Lily) = 1/2 = 0.5
....
```

问题：

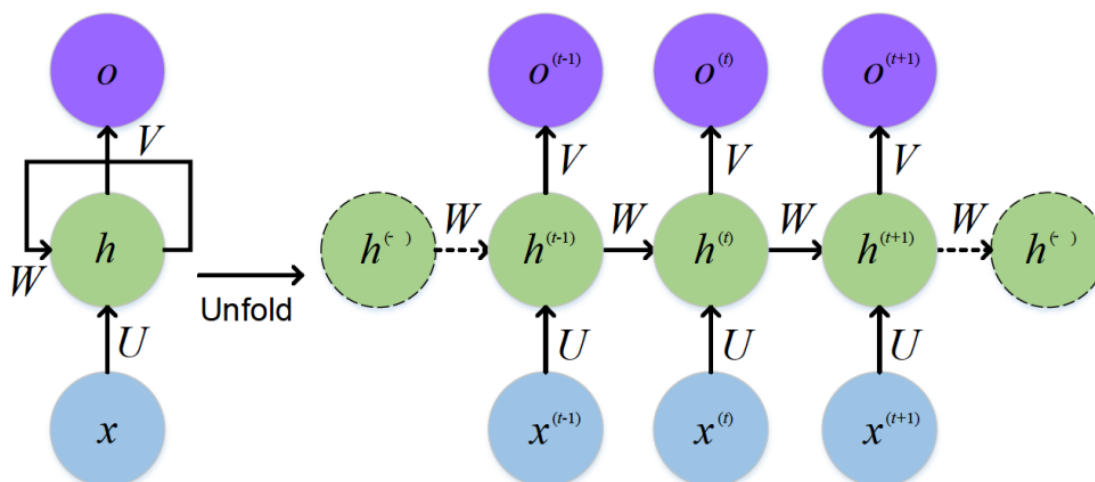
- 稀疏：窗口越大，越容易出现count为0的问题
- 存储：这些count需要预先存储下来，这个量太大了

2.4.3 RNN语言模型



- RNN语言模型优势：
 - 可以处理任意长度文本序列，而参数量不变。
 - 比n-gram能处理更长的上下文，而且没有稀疏的问题。

循环神经网络：



隐藏状态计算：

$$h_t = \sigma(U \cdot x_t + W \cdot h_{t-1} + b)$$

其中：

- h_t 是当前时间步 t 的隐藏状态。
- x_t 是当前时间步 t 的输入。
- h_{t-1} 是前一个时间步 $t - 1$ 的隐藏状态。

- U 是输入到隐藏状态的权重矩阵。
- W 是隐藏状态到隐藏状态的权重矩阵。
- b 是隐藏层的偏置向量。
- σ 是激活函数（例如 sigmoid 函数）。

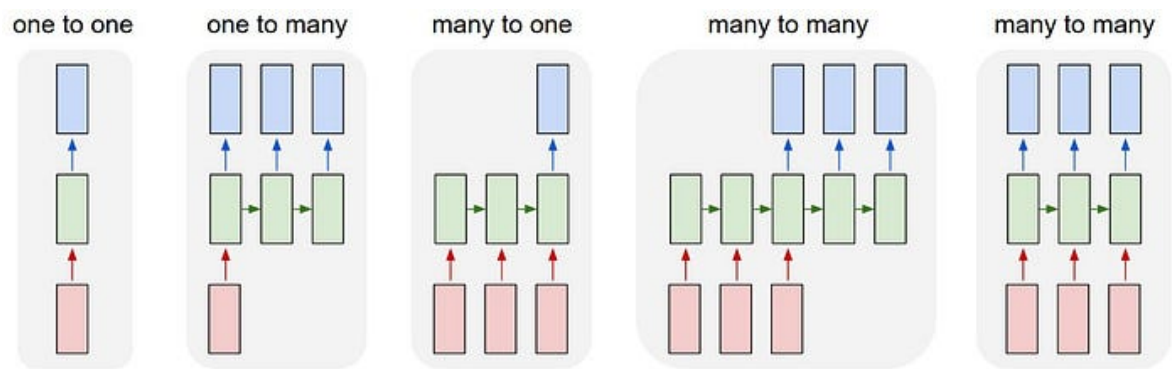
输出计算：

$$O_t = Vh_t + c$$

其中：

- y_t 是当前时间步 t 的输出。
- V 是隐藏状态到输出的权重矩阵。
- c 是输出层的偏置向量。

循环神经网络分类：



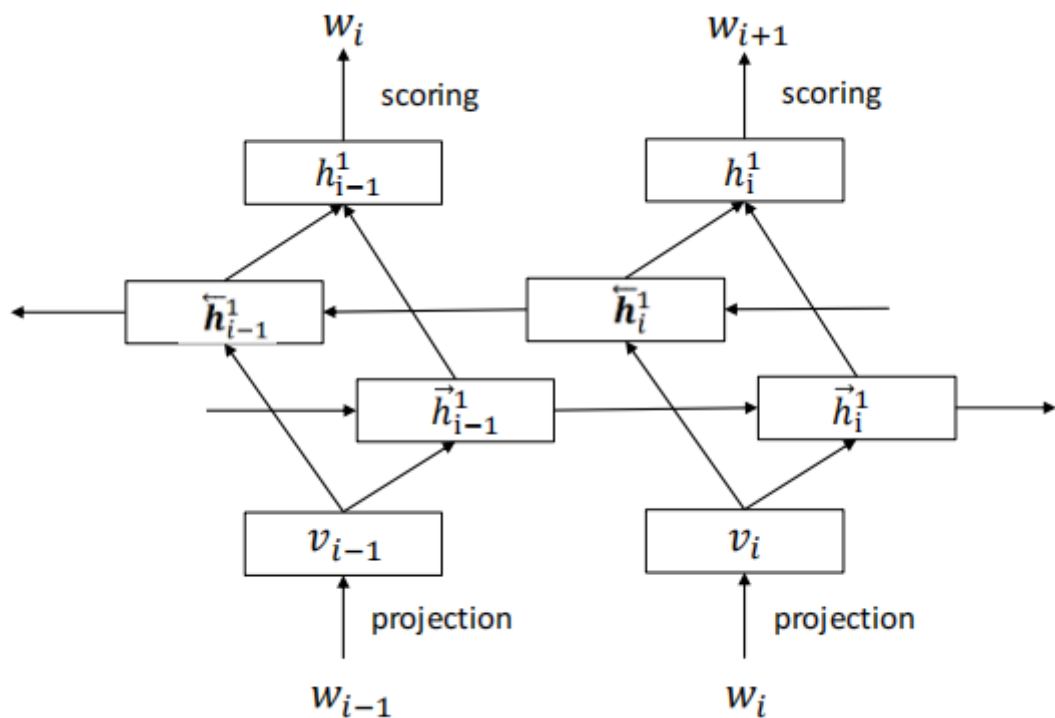
类型	描述	应用场景
一对一 (One-to-One)	一个输入对应一个输出	图像分类
一对多 (One-to-Many)	一个输入生成一个序列输出	图像描述生成
多对一 (Many-to-One)	输入一个序列生成一个输出	情感分析
多对多 (Many-to-Many)	输入一个序列生成一个序列输出	机器翻译

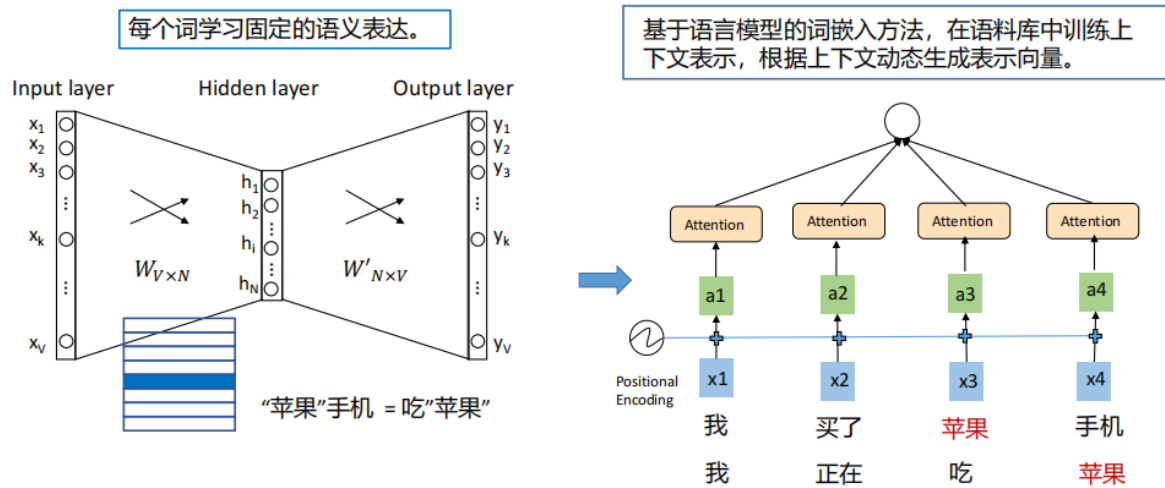
2.4.4 双向RNN语言模型

核心概念：

- **考虑了左右两边的上下文：** 双向 RNN 语言模型能够同时考虑输入序列中每个词语的左右两侧上下文信息，从而更全面地理解语义。
- **不能用于文本生成：** 由于双向 RNN 需要同时访问整个序列，因此它不适用于需要逐步生成文本的场景。

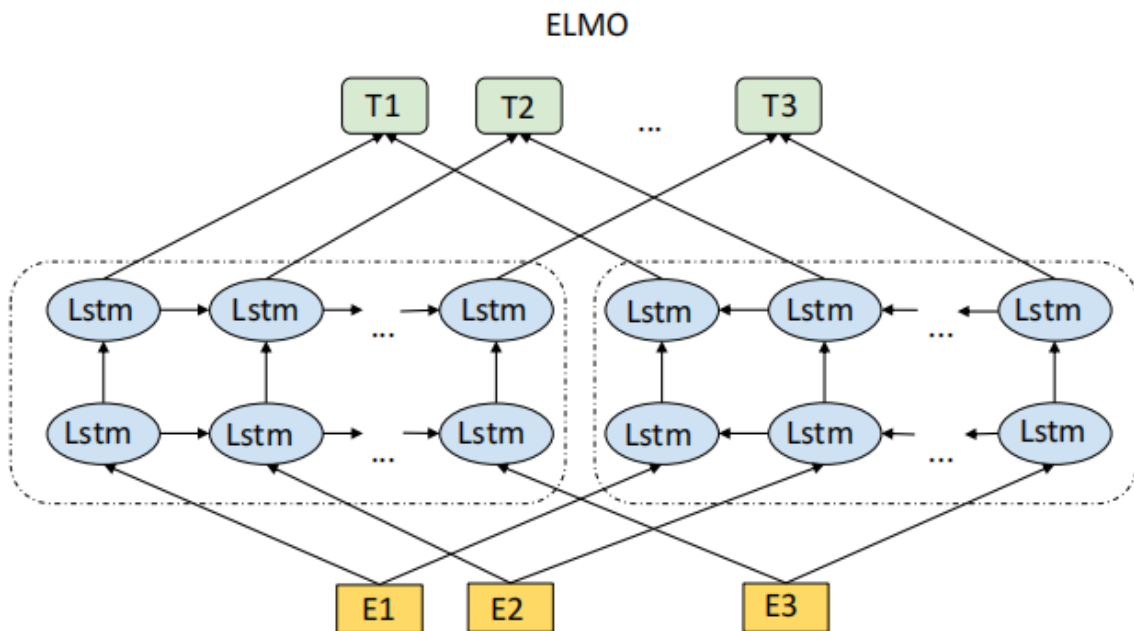
模型结构：





- 静态的词向量表达是固定的，“苹果”的词向量不变，动态词向量会跟着下游的任务不断学习变化，“苹果”的词向量会不断变化。

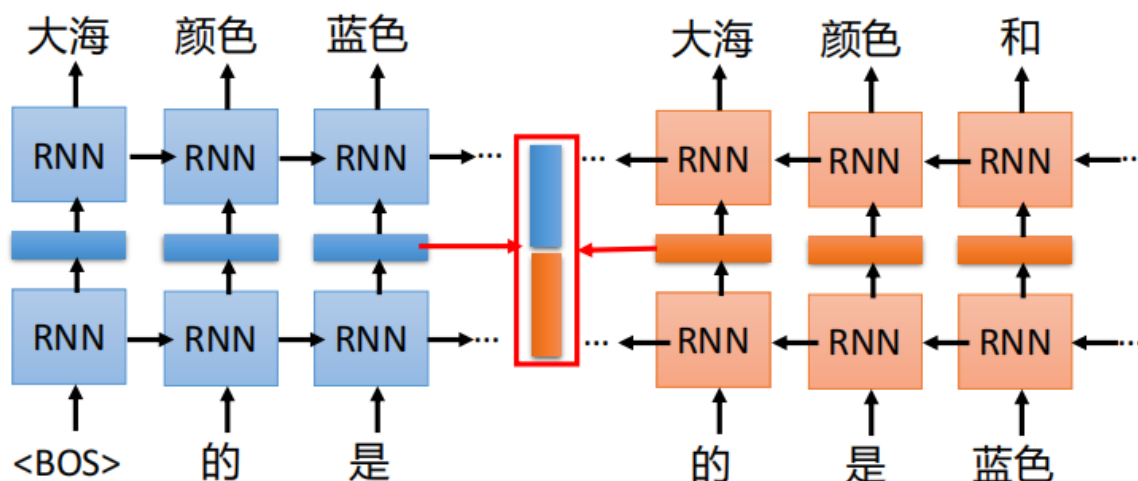
2.4.6 ELMO - Embeddings from Language Model (1)



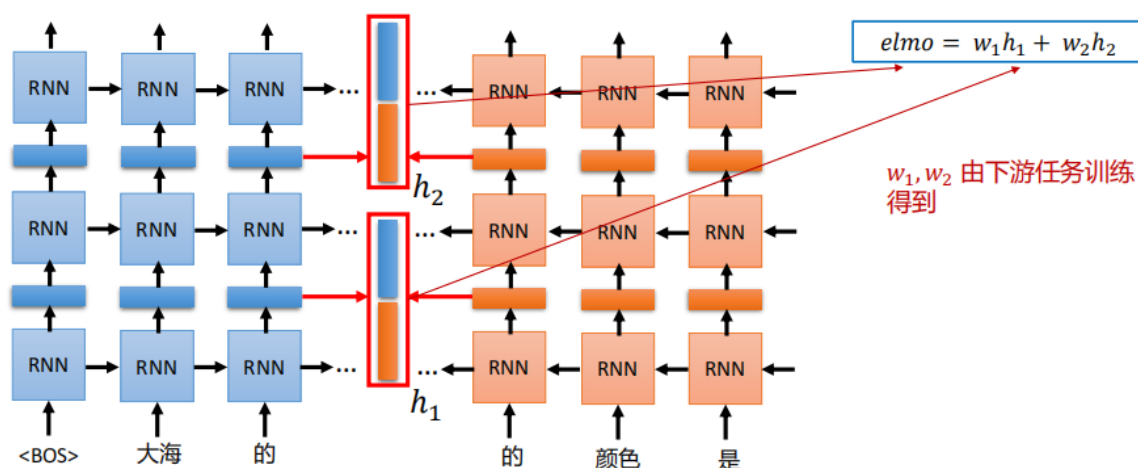
- 区别于双向RNN，两个独立的单向LSTM模型，可以做生成。

2.4.7 ELMO - Embeddings from Language Model (2)

文本：“大海的颜色是蓝色和绿色”



2.4.8 ELMO - Embeddings from Language Model (3)



• 不止一层的RNN，多层的输出也就是图中的h，因为不同的h代表不同的特征，所以不同层次输出的向量都十分重要，所以将这些向量进行加权重相加，权重也是可学习的参数，不同任务学习到的权重不同。

2.4.9 ELMo词向量

● 在双向模型预训练完成后，模型的编码部分便可以用来计算任意文本的动态词向量表示。在ELMo模型中，不同层次的隐含层向量蕴含了不同层次或粒度的文本信息。

□ 越接近顶层的LSTM隐含层表示，通常编码了更多的语义信息（对于阅读理解类语义信息需求高的任务适用）。

□ 接近底层的隐含层表示更偏重与词法句法信息（适用于命名实体识别类任务）。

2.5 Transformer

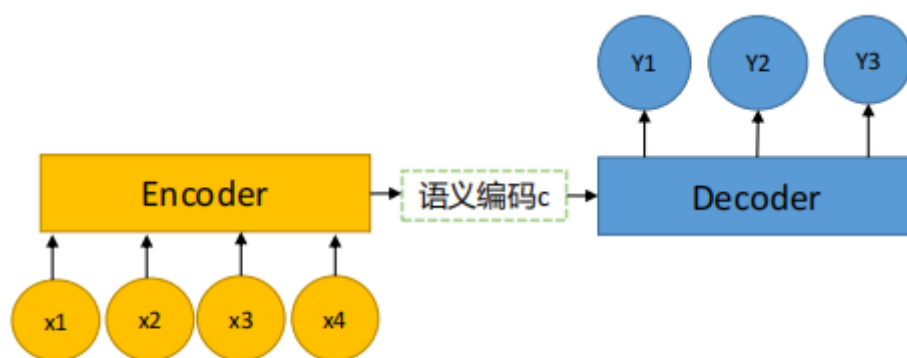
2.5.1 Seq2Seq

● Seq2Seq (Sequence to Sequence, 序列到序列) 模型, 由编码器 (Encoder) 和解码器 (Decoder) 两部分组成, 一般选用 RNN/LSTM/GRU 等网络实现。Seq2Seq的输入是一个不定长的序列, 输出也是一个不定长的序列。

● 编码器和解码器的作用:

□ 编码器将长度可变的输入序列转换成长度固定的上下文向量 (context vector), 并且将输入序列的信息在该上下文向量中进行编码。

□ 解码器将上下文向量进行解码, 转换成长度可变的输出序列。



Encoder-Decoder框架

解释:

- **Seq2Seq (Sequence-to-Sequence):** 一种模型架构, 用于将一个序列转换为另一个序列。
- **Encoder:** 将输入序列编码成一个固定长度的向量, 通常用于提取输入序列的特征。
- **Decoder:** 将编码器生成的向量解码成目标序列, 通常用于生成目标序列。
- **语义编码 c:** 连接编码器和解码器的中间状态向量, 用于传递输入序列的语义信息。
- **CNN (Convolutional Neural Network):** 卷积神经网络, 一种常用的深度学习模型, 适用于处理图像和序列数据。
- **RNN (Recurrent Neural Network):** 循环神经网络, 一种常用的深度学习模型, 适用于处理序列数据。
- **Transformer:** 一种基于自注意力机制的深度学习模型, 在自然语言处理任务中表现出色。
- **Image Caption:** 图像描述, 一种将图像转换为文本描述的任务。

2.5.2 Transformer: Attention is All You Need (1)

● Transformer是谷歌的研究人员提出的一种基于Seq2Seq结构的全新模型, 在被提出之后, 很快就席卷了整个自然语言处理领域。

● 与循环神经网络等传统模型不同, Transformer模型仅仅使用一种被称作自注意力机制的方法和标准的前馈神经网络, 完全不依赖任何循环单元或者卷积操作。自注意力机制的优点在于可以直接对序列中任意两个单元之间的关系进行建模, 这使得长距离依赖等问题可以更好地被求解。

● Transformer也可以当作一种表示模型, 因此也被大量地使用在自然语言处理的其他领域, 目前非常流行的BERT等预训练模型就是基于Transformer。

2.5.3 Transformer: Attention is All You Need (2)

	Seq2seq without attention	Seq2seq with attention	Transformer
Processing within encoder	RNN/CNN	RNN/CNN	attention
Processing within decoder	RNN/CNN	RNN/CNN	attention
decoder-encoder interaction	static fixed-sized vector	attention	attention

• 除了encoder-decoder之间的注意力机制，transformer在encoder编码器，decoder解码器内部还引入了自注意力机制（self-attention），更好的捕捉上下文语义，相比于传统的基于RNN网络的编码器和解码器，使用自注意力机制的编码器和解码器能够更好的捕捉上下文信息，此外RNN网络在时序上必须顺序执行，无法并行，效率较慢，自注意力机制可以并行处理，充分利用硬件进行加速。

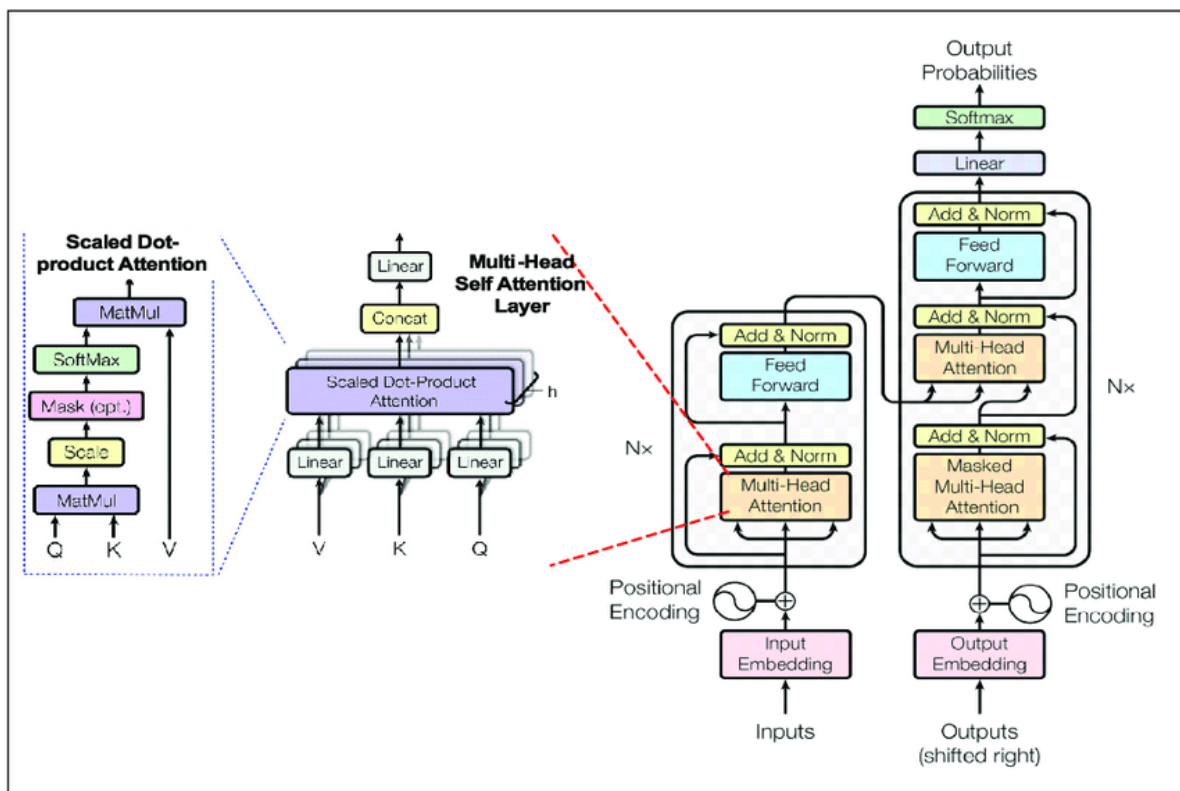
2.5.4 Transformer时间复杂度

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

RNN、CNN、Transformer时间复杂度对比
<https://arxiv.org/pdf/1706.03762>

2.5.5 Transformer架构

- Transformer由Encoder和Decoder两部分组成。
- Encoder包含N=6个相同的layer，layer指的就是图中左侧的单元。
- Decoder也包含N=6个相同的layer。



<https://arxiv.org/pdf/1706.03762>

2.5.6 Encoder Block中的组件 (1)

● Transformer中的Encoder和Decoder中包含的模块相似：

- Input Embedding：将输入的数据（如文本中的单词或标记）转化为模型可以处理的数值向量形式。
- Positional Encoding，位置编码：捕捉输入之间的顺序和位置关系。
- Multi-Head Attention，多头注意力：由多个注意力子层构成，用于扩展了模型专注于不同位置的能力。
- 自注意力（Self-attention）：用于学习到不同位置之间的关联性，从而捕捉到更丰富的上下文信息。
- Feed Forward，前馈神经网络：使用全连接的前馈神经网络对输入向量序列进行进一步变换。

• 以上操作就构成了 Transformer 的一层，各个模块执行的顺序可以简单描述为：

Self-Attention → Residual Connection → Layer Normalization → Feed Forward Network →

Residual Connection → Layer Normalization

编码器可以包含多个这样的层，比如，可以构建一个六层编码器，每层都执行上面的操作。最上层的结果作为整个编码的结果，会被传入解码器。

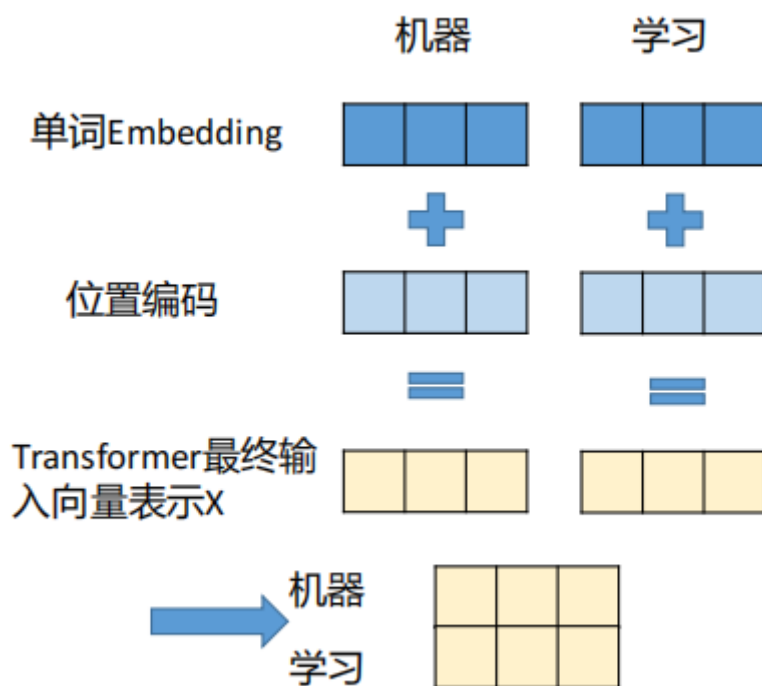
2.5.7 Encoder Block中的组件 (2)

□ 残差连接 (Residual Connection, 标记为 “Add”) : 对于自注意力子层和前馈神经网络子层, 都有一个从输入直接到输出的额外连接, 也就是一个跨子层的直连。残差连接可以使深层网络的信息传递更为有效;

□ 层正则化 (Layer Normalization, 标记为 “Norm”) : 自注意力子层和前馈神经网络子层进行最终输出之前, 会对输出的向量进行层正则化, 规范结果向量取值范围, 这样易于后面进一步的处理。

2.5.8 Transformer的输入

- 分词: 将输入文本切分成一系列标记 (Token) 。
- Embedding: 将分词后的标记转换为高维向量表示。
- 位置编码: 为输入序列中的每个元素提供一个唯一的位置标识。
- 合并位置信息和词向量 (通常采用相加的方式) 作为模型输入。



2.5.9 Transformer位置编码

- Transformer中采用的是三角函数位置编码 (绝对位置编码) 。

$$\text{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$\text{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

公式解释:

这两个公式用于计算 Transformer 模型中的位置编码 (Positional Encoding, PE)。

- $PE_{(pos, 2i)}$: 表示位置 pos 在维度 $2i$ 上的编码值。
- $PE_{(pos, 2i+1)}$: 表示位置 pos 在维度 $2i+1$ 上的编码值。
- pos: 表示输入序列中词语的位置。
- i: 表示编码向量的维度索引。
- d_{model} : 表示模型的维度。
- $\sin()$: 表示正弦函数。
- $\cos()$: 表示余弦函数。

位置编码的作用:

由于 Transformer 模型没有循环神经网络 (RNN) 的结构, 无法捕捉序列中词语的位置信息。为了让模型能够感知词语的位置, 需要将位置编码添加到输入序列的词向量中。

公式细节:

- $10000^{2i / d_{model}}$: 这是一个缩放因子, 用于控制不同维度上的波长。
- $\sin()$ 和 $\cos()$: 使用正弦和余弦函数生成周期性的编码值, 使得模型能够区分不同位置的词语。

□ 为什么是 $\sin$ 和 $\cos$: 对于每个位置 i 和每个维度 d , 使用正弦和余弦函数交替地生成位置编码, 其中波长形成一个几何级数。这种编码方式不仅与序列中每个位置的绝对值有关, 还由于三角函数的性质, 使得位置 $a+b$ 的向量可以表示成位置 a 和位置 b 的向量的组合, 从而提供了表达相对位置信息的可能性。对于固定长度的间距 k , $PE(pos + k)$ 可以用 $PE(pos)$ 计算得到。

在 Transformer 模型的位置编码公式中, 10000 是一个人为设定的常数, 它的选择并非基于某种特定的数学推导, 而是出于以下几个方面的考虑:

1. 控制波长范围:

- 位置编码的目的是为模型提供关于序列中词语位置的信息。
- 通过使用不同频率的正弦和余弦函数, 模型可以区分不同位置的词语。
- 10000 这个常数被用来调整这些正弦和余弦函数的波长, 从而确保在模型处理的序列长度范围内, 位置编码能够提供足够区分度的信息。
- 通过 10000 来控制波长的范围, 可以让模型在一定长度的序列中, 有比较好的区分度。

2. 避免波长过于接近:

- 如果波长过于接近, 模型可能难以区分相邻位置的词语。
- 10000 这个常数被选择, 使得在常见的序列长度范围内, 不同维度上的波长能够保持适当的距离, 从而提高位置编码的有效性。

3. 经验性选择:

- Transformer 模型的原始论文 ("Attention Is All You Need") 中, 作者通过实验发现, 10000 这个常数能够提供较好的性能。
- 因此, 它被沿用至今, 成为了 Transformer 模型中位置编码的默认设置。

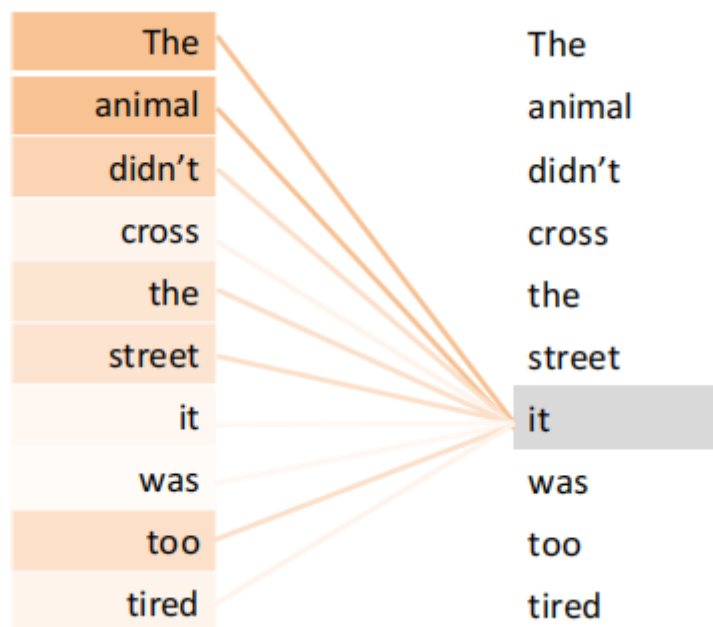
• 与传统的可学习位置嵌入相比, 三角函数位置编码不需要额外的训练参数。位置编码是通过固定的数学公式直接计算得到的, 这减少了模型的参数量。

• 由于三角函数位置编码具有显式的生成规律, 模型可以更容易地将其推广到未见过的序列长度上。

- 三角函数位置编码还具有一定的表达相对位置信息的能力。通过计算不同位置编码之间的差值或比例，模型可以推断出元素之间的相对位置关系。但在实际使用过程中，这种隐式的相对位置信息表达模型并不是总能学习到。

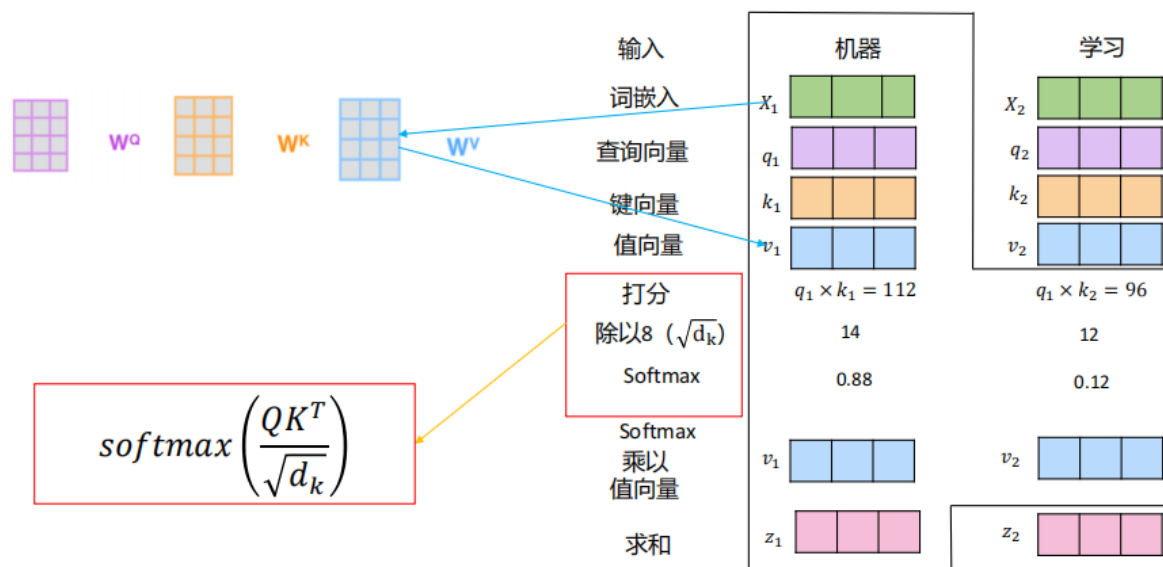
2.5.10 自注意力

- 编码器-解码器注意力 (Encoder-Decoder Attention) 关注的是输入和输出之间的关系，自注意力 (Self-Attention) 关注的是输入之间的关系。

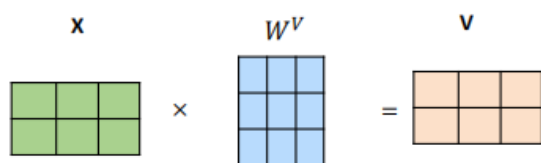
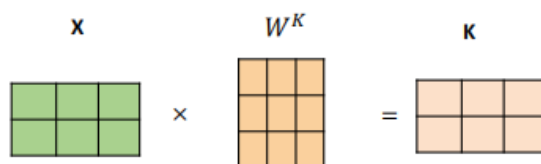
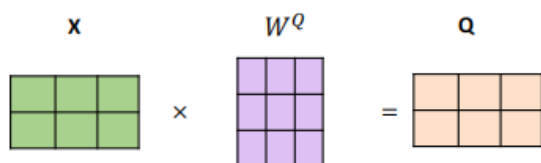


- 上图中，it和 animal 和 tired的关系权重比较大，这反映出attention能够反映输入之间的一些语义关系

2.5.11 自注意力实现 (1)

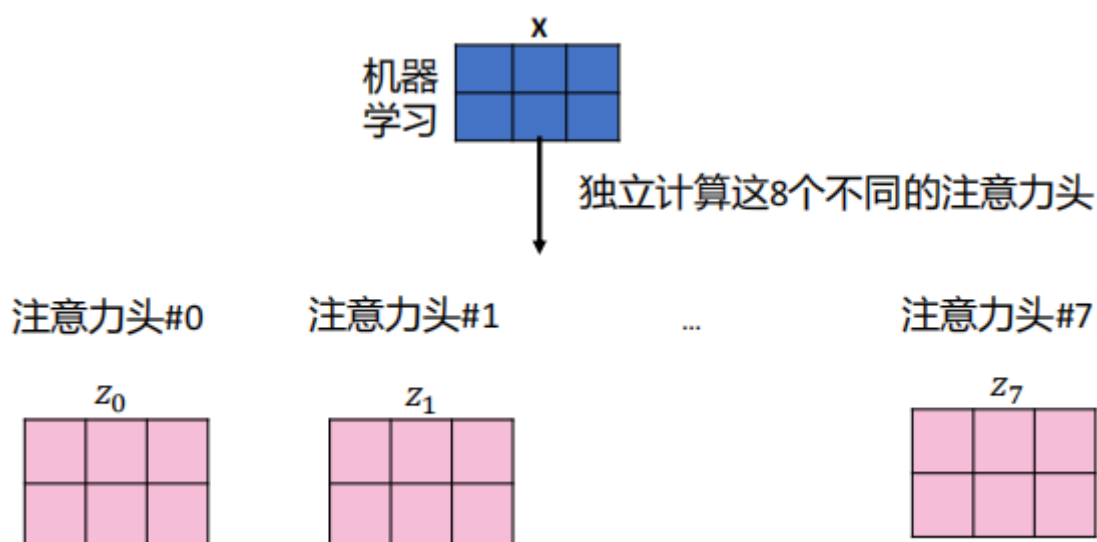


2.5.12 自注意力实现 (2)



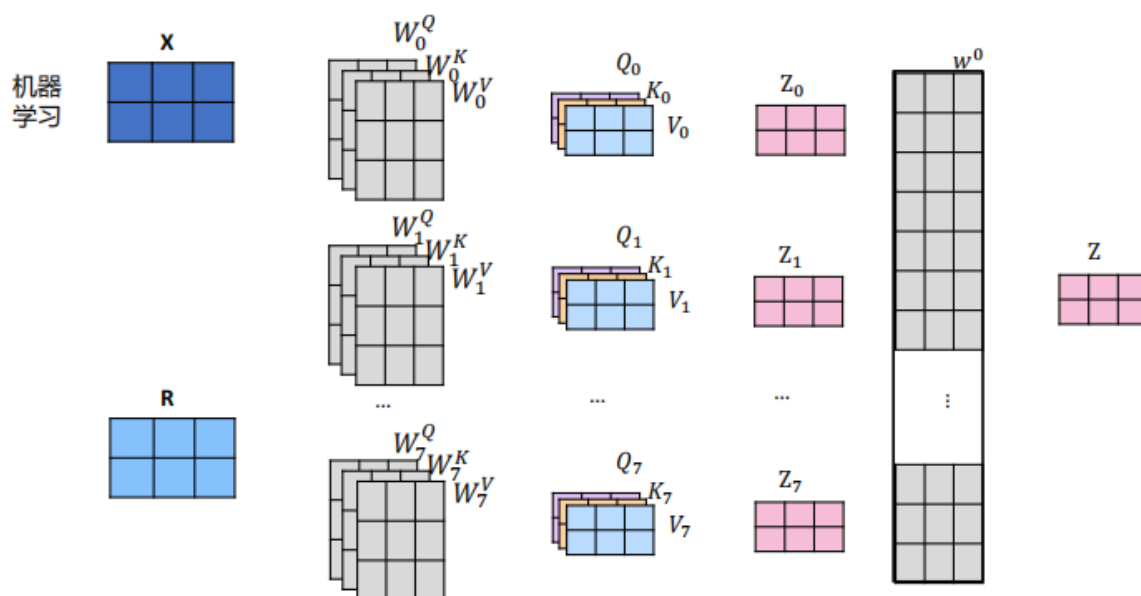
$$Attention(Q, K, V) = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

2.5.13 多头注意力 (1)



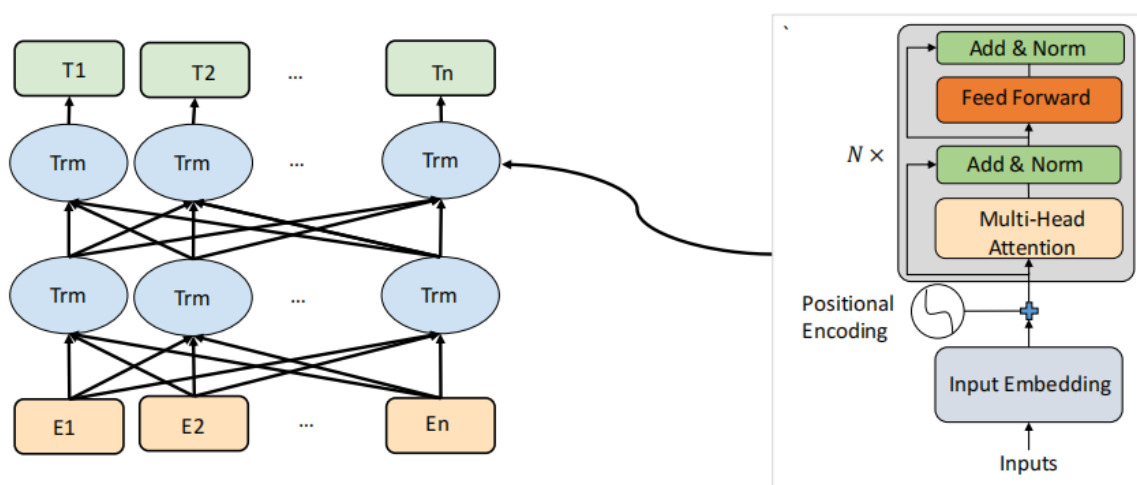
- 在“多头”注意机制下，我们为每个头保持独立的 查询/键/值 权重矩阵，从而产生不同的查询/键/值矩阵。和之前一样，我们拿 X 乘以 $W_Q / W_K / W_V$ 矩阵来产生 查询/键/值 矩阵。
- 如果我们做与上述相同的自注意力计算，只需八次不同的权重矩阵运算，我们就会得到八个不同的 Z 矩阵。
- 这给我们带来了一点挑战。前馈层不需要 8 个 矩阵，它只需要一个矩阵（由每一个单词的表示向量组成）。所以我们需要一种方法把这八个矩阵压缩成一个矩阵。那该怎么做？

2.5.14 多头注意力 (2)



2.5.15 BERT

BERT的模型架构是基于Transformer的多层双向Transformer Encoder，来实现深度双向预训练模型训练，即所有层都联合上下文语境进行预训练，较之前使用单向语言模型训练更准确，信息量更大，且语义理解更准确。

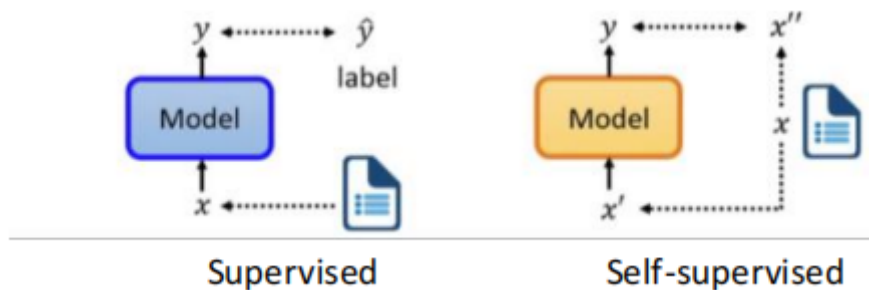


2.5.16 自监督学习

当前拥有一堆的数据，但是没有标注。

将这堆数据 x 分成两个部分：一部分作为模型的输入 x' ，一部分作为模型的标签 x'' 。

然后对模型进行训练，使模型地输出 y 尽可能地接近标签 x'' 。注意这里 x'' 不是人标注的，而是数据里本来就有的。



● BERT通过自监督学习的方式进行训练。在训练过程中，BERT通过对大量无标签的语料库进行学习，以预测上下文语境中下一个单词的概率分布。

2.5.17 BERT Input

Input	[CLS]	my	cat	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embeddings	E_{CLS}	E_{my}	E_{cat}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{\#ing}$	$E_{[SEP]}$
	+	+	+	+	+	+	+	+	+	+	+
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_A	E_A	E_A	E_A	E_A
	+	+	+	+	+	+	+	+	+	+	+
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

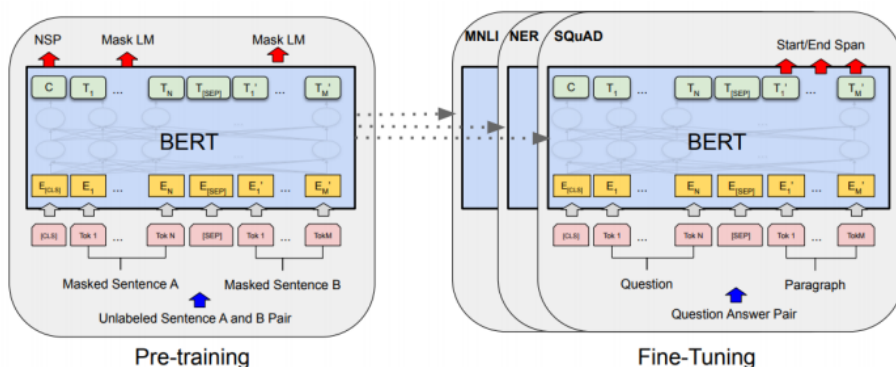
> **Token Embeddings**是词向量，第一个单词是CLS标志，可以用于之后的分类任务。

> **Segment Embeddings**用来区别两种句子，因为预训练不光做LM还要做以两个句子为输入的分类任务。

> **Position Embeddings**对位置信息编码，表示每个单词输入序列的位置信息。

2.5.18 Pre-train和Fine-tune

● BERT包含pre-train和fine-tune两个阶段。pre-train阶段模型是在无标注的标签数据上进行训练，fine-tune阶段使用预训练BERT模型在下游任务的带标签数据上对模型参数进行微调，BERT模型首先是被pre-train模型参数初始化，然后所有的参数会用下游的有标注的数据进行训练。



<https://arxiv.org/pdf/1810.04805>

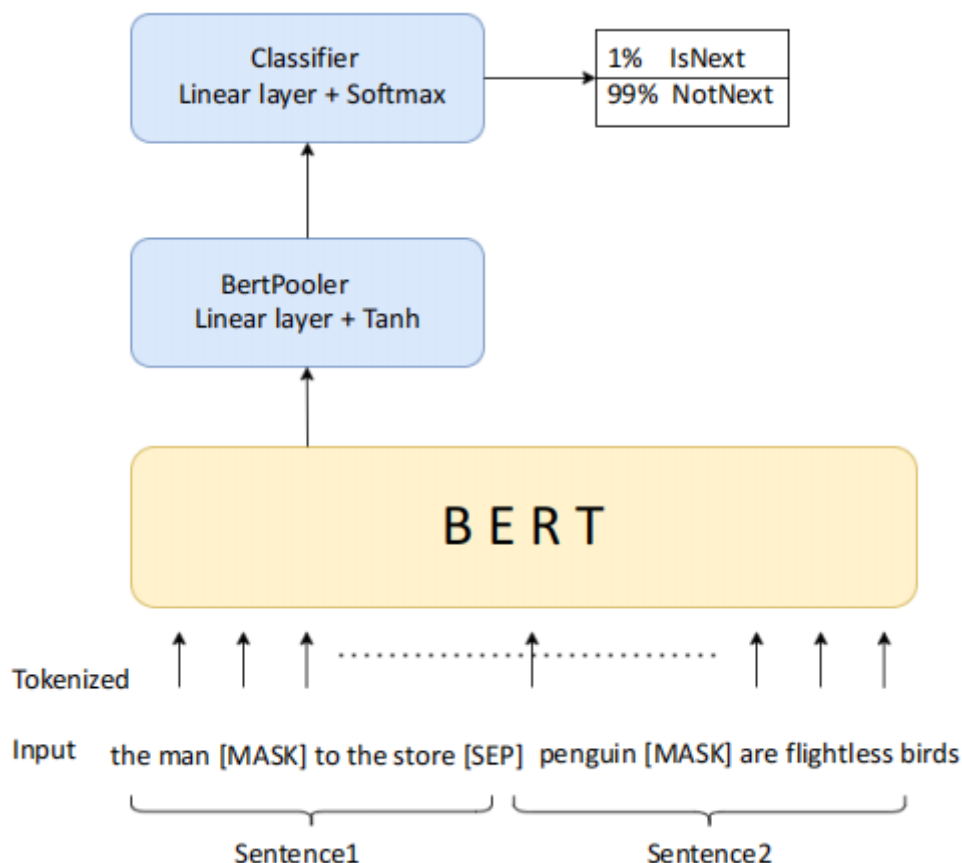
2.5.19 MLM (Masked Language Model)

- BERT 是一个多任务模型，它的预训练 (Pre-training) 任务是由之前提到的两个自监督任务组成，即 MLM 和 NSP。
- MLM 是指在训练的时候随机从输入语料上 mask 掉一些单词，然后通过该单词的上下文预测该单词，该任务非常像我们在中学时期经常做的完形填空。
 - 80% 的时候会直接替换为 [Mask]，将句子 “my dog is cute” 转换为句子 “my dog is [Mask]”。
 - 10% 的时候将其替换为其它任意单词，将单词 “cute” 替换成另一个随机词，例如 “apple”。将句子 “my dog is cute” 转换为句子 “my dog is apple”。
 - 10% 的时候会保留原始 Token，例如保持句子为 “my dog is cute” 不变。

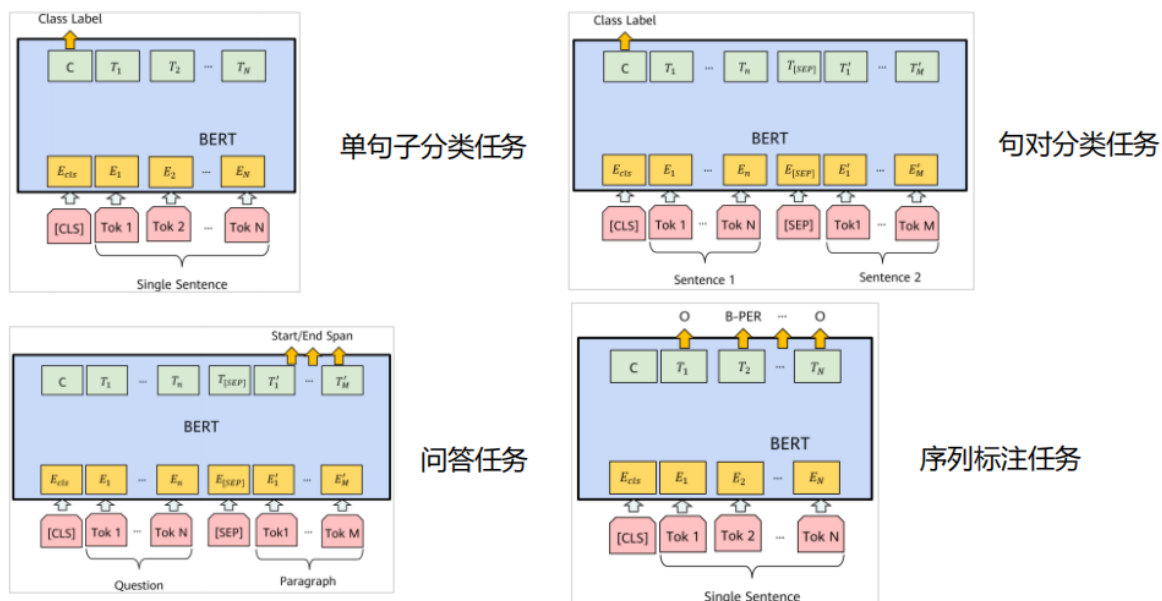
2.5.20 Next Sentence Prediction (1)

- Next Sentence Prediction (NSP) 是判断句子B是否是句子A的下文。
- 许多重要的下游任务，如问答 (QA) 和自然语言推理 (NLI) 都是基于理解两个句子之间的关系，而这并不能被语言建模直接捕获。为了训练一个理解句子关系的模型，BERT采用下面的NSP策略
 - 如果是的话输出IsNext，否则输出NotNext。
- 训练数据的生成方式是从平行语料中随机抽取的连续两句话，其中50%保留抽取的两句话，它们符合IsNext关系，另外50%的第二句话是随机从预料中提取的，它们的关系是NotNext的。这个关系保存在的[CLS]符号中。

2.5.21 Next Sentence Prediction (2)

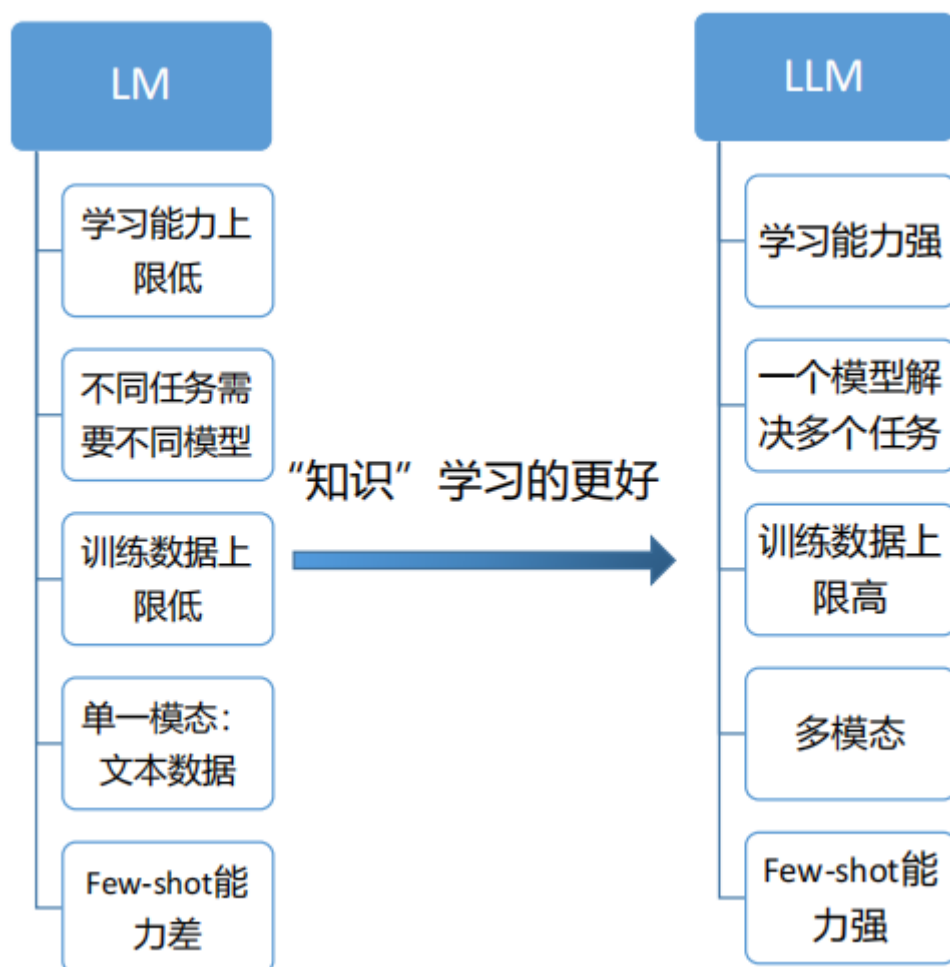


2.5.22 Fine-tuning



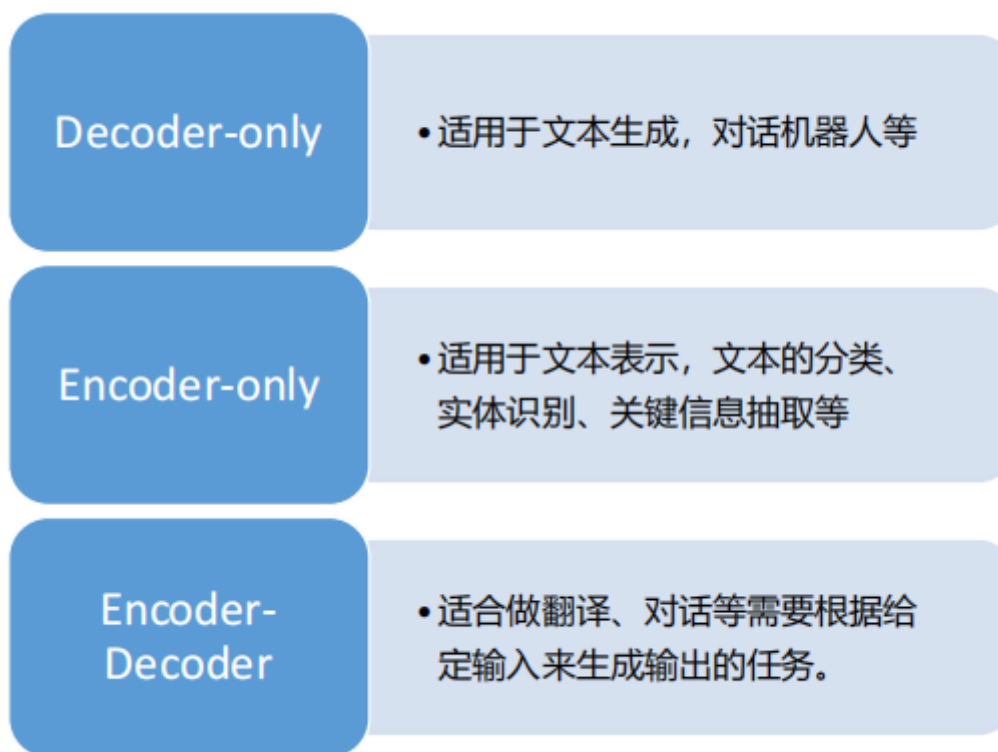
2.5.23 LLM

● 大语言模型 (Large Language Models, 简称LLM)，指那些拥有海量参数、能够处理和理解广泛语言数据集的模型。它们不仅能够进行文本生成、问答、摘要等任务，还能在特定领域展现出惊人的创造力和理解能力，比如编写代码、撰写文章甚至进行文学创作。



2.5.24 大语言模型的结构分类

当前LLM以Transformer为基础分为三个类别：



大模型进化树：

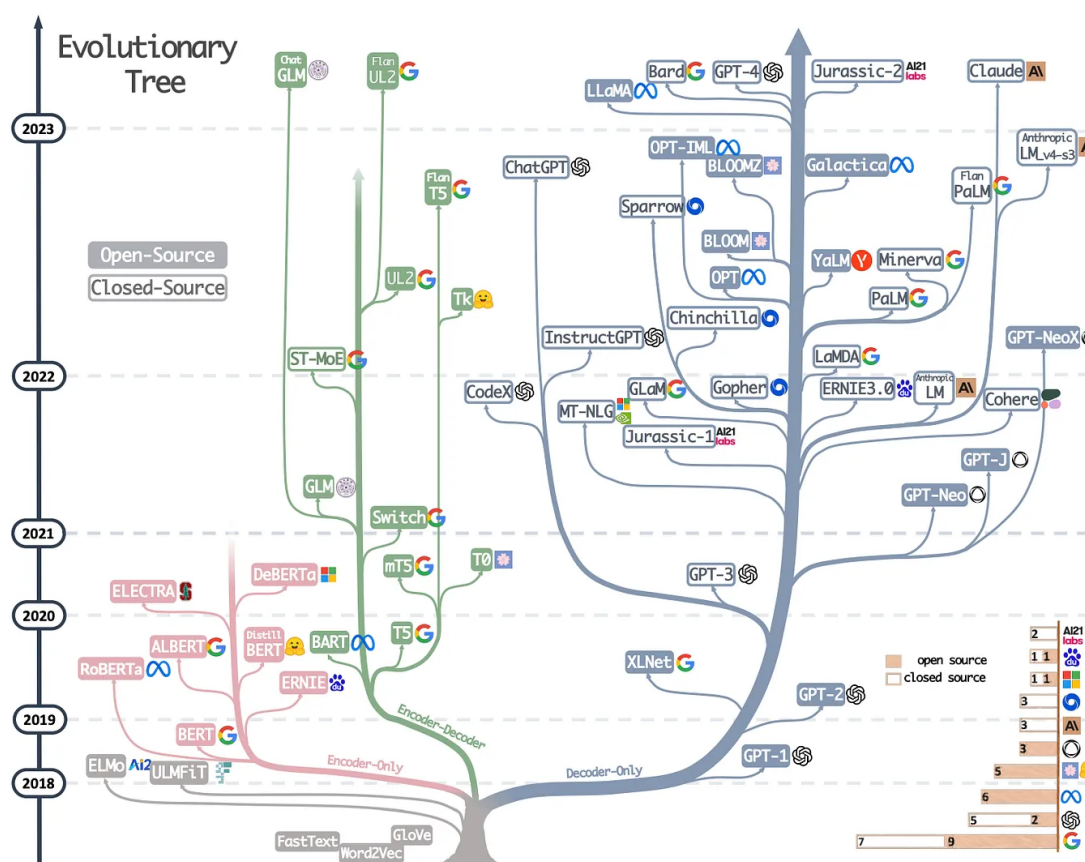
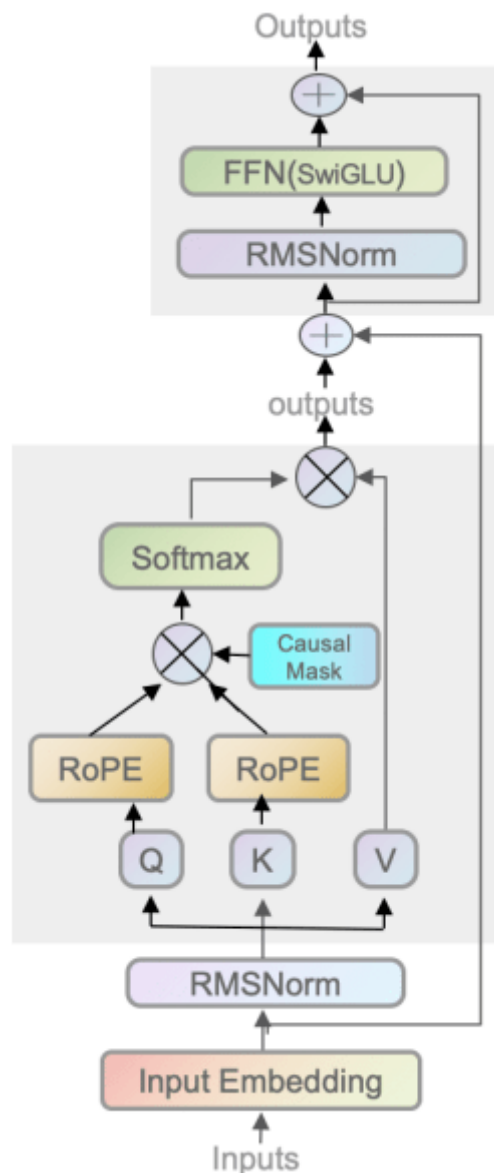


Fig. 1. The evolutionary tree of modern LLMs traces the development of language models in recent years and highlights some of the most well-known models. Models on the same branch have closer relationships. Transformer-based models are shown in non-grey colors: decoder-only models in the blue branch, encoder-only models in the pink branch, and encoder-decoder models in the green branch. The vertical position of the models on the timeline represents their release dates. Open-source models are represented by solid squares, while closed-source models are represented by hollow ones. The stacked bar plot in the bottom right corner shows the number of models from various companies and institutions.

2.5.25 LLAMA结构

● LLaMA是由Meta AI发布的大语言模型（目前为LLaMA3），模型采用了Decoder-only的架构，有7B、13B、33B、60B多个版本。在公开数据集（1.4T token）上进行训练。模型结构细节如下（以7B为例）：

- 模型整体结构由32个Block堆叠而成；
- 文本数据处理采用了BPE；
- 位置编码使用旋转位置编码；
- 在每个子层中使用了 RMSNorm 进行归一化；
- 激活函数使用SwiGLU。



RMSNorm（Root Mean Square Normalization）和LayerNorm（层归一化）都是深度学习中常用的归一化方法，尤其在Transformer架构及其变体中被广泛应用。以下是它们的区别：

LayerNorm:

给定输入向量 $\mathbf{x} = [x_1, x_2, \dots, x_n]$, LayerNorm的计算步骤如下:

- 计算均值: $\mu = \frac{1}{n} \sum_{i=1}^n x_i$
- 计算方差: $\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$
- 归一化操作: $\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}}$
- 缩放和平移: $\mathbf{y} = \mathbf{g} \odot \hat{\mathbf{x}} + \mathbf{b}$

这里 ϵ 是一个很小的常数, 用于保证数值稳定性, $\mathbf{g}$ 是可学习的缩放参数, $\mathbf{b}$ 是可学习的偏移参数。

1. RMSNorm公式

RMSNorm的计算步骤如下:

- 计算均方根 (RMS) :

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}$$

- 归一化操作:

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x}) + \epsilon}$$

- 缩放操作 (可选) :

$$\mathbf{y} = \mathbf{g} \odot \hat{\mathbf{x}}$$

与LayerNorm不同, RMSNorm没有计算均值并进行偏移的步骤, 只通过均方根进行归一化。

2. 计算复杂度

- **LayerNorm**: 需要计算均值和方差, 涉及两次对输入向量元素的遍历, 计算量相对较大。
- **RMSNorm**: 只需要计算均方根, 只需要对输入向量元素进行一次遍历, 计算量相对较小, 在计算效率上更有优势。

3. 数值稳定性

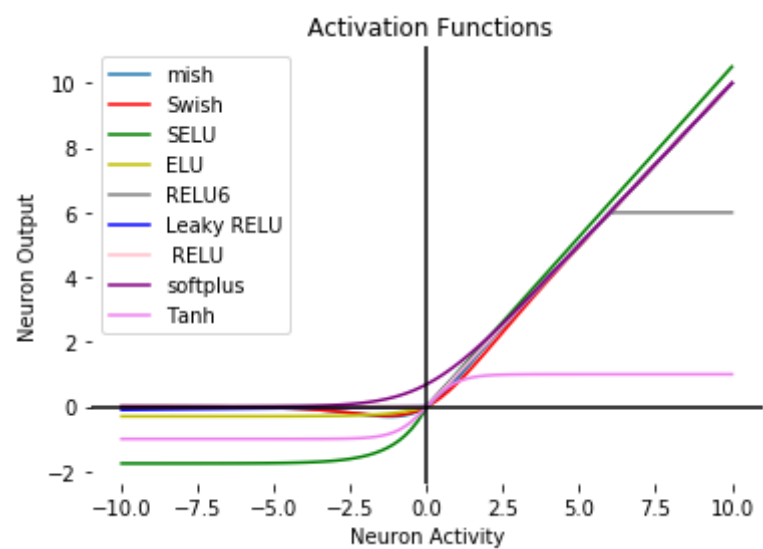
- **LayerNorm**: 减去均值的操作可以使得数据在归一化后分布在零附近, 在某些情况下有助于模型的训练稳定性, 但当数据的均值波动较大时, 可能会影响归一化的效果。
- **RMSNorm**: 由于不减去均值, 避免了均值波动对归一化的影响, 在数值稳定性上表现较好, 尤其是在处理长序列数据时。

4. 表达能力

- **LayerNorm**: 通过减去均值和方差归一化, 能够更全面地调整数据的分布, 使得模型可以学习到数据的均值和方差信息, 在一些任务中可能具有更强的表达能力。
- **RMSNorm**: 虽然没有减去均值, 但通过可学习的缩放参数 $\mathbf{g}$ 也可以在一定程度上调整数据的尺度, 并且在一些实验中显示出与LayerNorm相当的性能。

5. 应用场景

- **LayerNorm**：在许多深度学习模型中都有广泛应用，特别是在Transformer架构中，是标准的归一化方法，在自然语言处理任务中表现良好。
- **RMSNorm**：由于其计算效率和数值稳定性的优势，在一些大规模的语言模型和需要高效计算的场景中得到了应用，如GPT - NeoX等模型。



Swish 函数的定义为：

$$\text{Swish}(z) = z \cdot \sigma(z)$$

其中 σ 是 Sigmoid 函数。

SwiGLU 的公式为：

$$\text{SwiGLU}(x, W, V, b, c) = \text{Swish}(xW + b) \otimes (xV + c)$$

其中：

- x 是输入向量。
- W 和 V 是线性变换矩阵。
- b 和 c 是偏置项。
- $\otimes$ 表示逐元素乘积（Hadamard 积）。

核心优势:

- **增强非线性表达**：通过门控机制动态调节信息流，结合 Swish 的平滑性和非单调性，提升模型对复杂模式的捕捉能力。
- **缓解梯度消失**：Swish 在负区间仍有非零梯度，且门控结构通过线性部分保留信息，优化训练稳定性。
- **Transformer 中的作用**：在 Transformer 架构中，SwiGLU 通常用于替换前馈神经网络（FFN）层中的 ReLU 或其他激活函数，通过门控机制控制 FFN 层的输出，从而影响模型的整体性能。
- **高效计算**：一些模型会使用SiLU函数替换Swish函数，从而达到更好的运算速度。

第三章 DeepSeek算法解析及工程优化

3.1 稠密模型和稀疏模型

● 模型规模是提升模型性能的关键因素之一。随着基于Transformer结构的大模型不断发展，模型的参数量越来越大（提高模型性能），如果训练和推理时全部的参数均参与计算（即参与计算的矩阵大多为稠密矩阵），那么计算所需要的算力会越来越高。因此这类大参数量的稠密模型推理成本相对较高。

● 那如何提高模型的参数，但同时又不显著增加计算需求量呢？

● 混合专家模型（MoE）是一个很好的选择！它在计算过程中，均是部分参数参与计算，即只有少部分参数不为零。这种稀疏矩阵的计算可以减少计算资源的需求，模型单次训练和推理所需计算资源低，计算时间短。

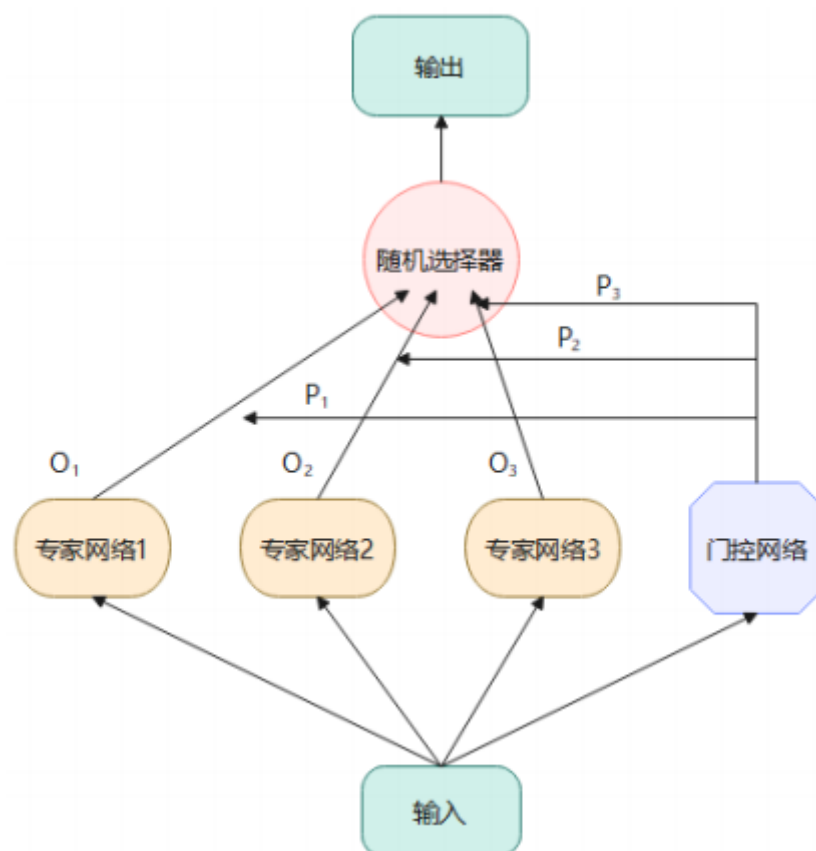
3.2 混合专家模型（MoE）

● 究竟什么是一个混合专家模型（MoE）呢？它的整体框架主要分为以下部分：

□ 专家网络：用于学习不同的数据。输入数据经过每个专家系统，分别会输出一个值，在该图中，每个输出标记为 O_i 。

□ 门控网络：决定每个专家系统输出在随机选择器里的权重。它为每个输出分配的权重为 P_i 。该网络同样可以被训练。

□ 随机选择器：获取多个专家系统的当前输出，结合权重获得最终输出结果。

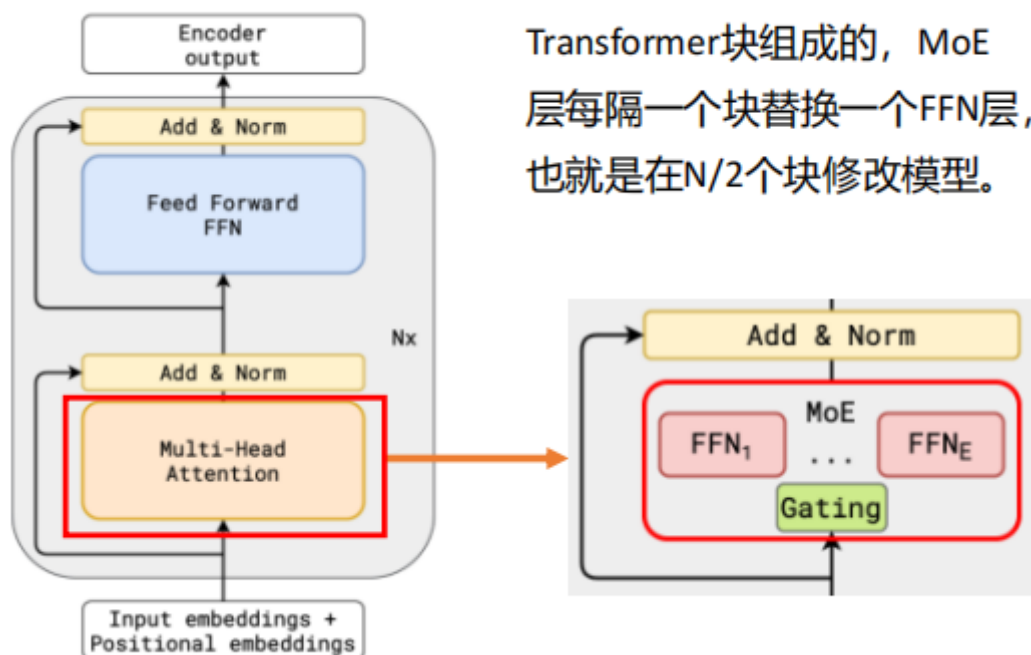


3.3 MoE模型 - GShard

● Gshard是Google发布的首个Transformer架构与MoE结合模型架构。

● Transformer 编码器层由两个连续层组成，即一个自注意力层，后面跟着一个位置前馈层。解码器添加了第三个交叉注意力层，该层关注编码器的输出。通过条件计算稀疏地缩放Transformer，将每个其他前馈层替换为Position-wise Mixture of Experts (MoE) 层。

- Encoder/Decoder是由多个Transformer块组成的，MoE层每隔一个块替换一个FFN层，也就是在 $N/2$ 个块修改模型。



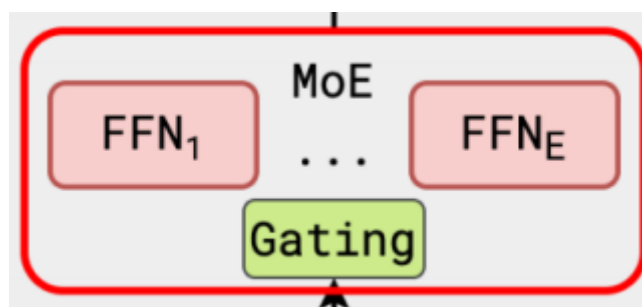
• 之前的MoE是把每个样本分到不同的expert，Transformer架构中是把每个token分到不同的expert。

3.4 GShard - Top-K gating

● 门控网络可能会使用Top-K gating机制，这意味着对于每个输入token，只有概率最高的K个专家会被选中进行计算。Gshard使用的K是2，即top-2。

● Gshard中门控函数满足两个条件：

- 平衡负载
- 可高效率的大规模并行计算



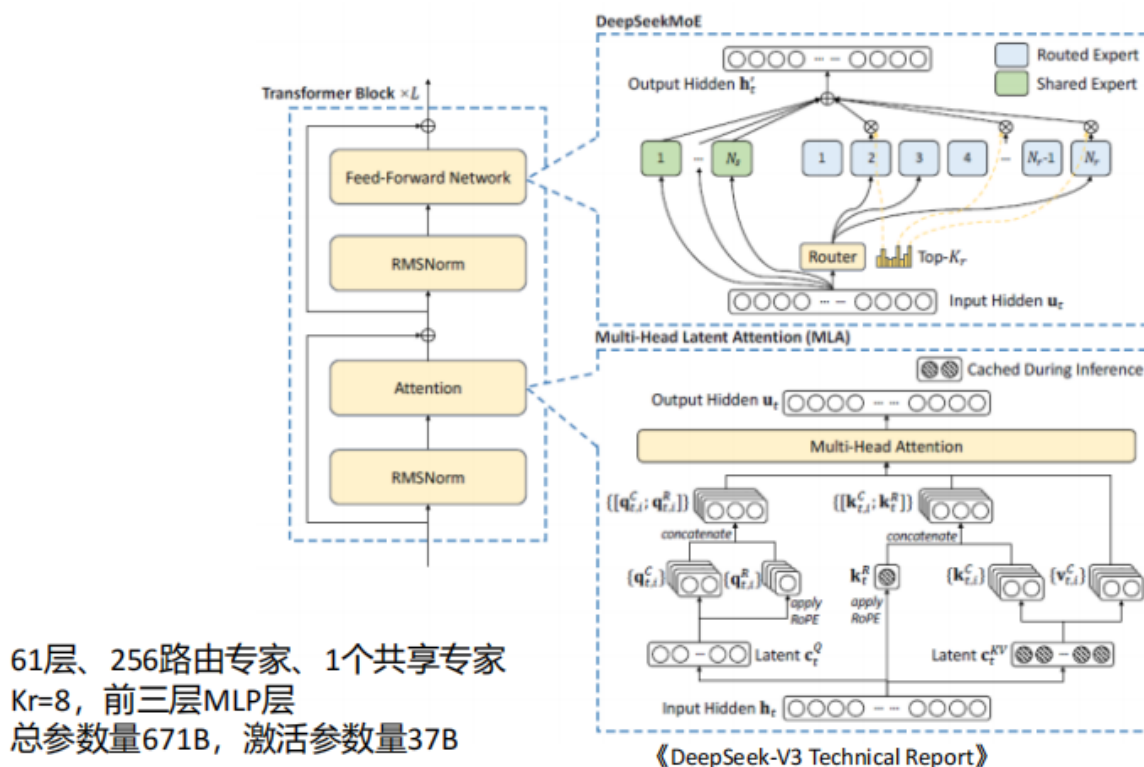
• 希望MoE层能够稀疏地激活给定令牌的专家。一个简单的解决方案是根据 softmax 概率分布选择 top-k 专家。然而，众所周知，这种方法会导致训练的负载不平衡问题。在训练过程中看到的大多数令牌将被分配给少数专家，只为少数（忙碌的）专家积累了非常大的输入缓冲区，而其他专家则未得到训练，从而减慢了训练速度。与此同时，许多其他专家根本没有得到充分的训练。一个更好的门控函数设计可以更均匀地将处理负担分配给所有专家。

• 如果按顺序执行门控功能，则实现平衡负载将是相当简单的。对于给定E个专家的输入批中所有N个令牌，单独的门控函数的计算成本至少为O。然而，在研究中，N的数量级为百万，E的数量级为千，门控函数的顺序实现将使大部分计算资源在大多数时间处于空闲状态。因此，需要一个有效的门控函数并行实现来利用许多器件。

3.5 DeepSeek-V3

● 2024 年 12 月 26 日晚，AI公司深度求索（DeepSeek）正式上线全新系列模型DeepSeek-V3首个版本并同步开源。该模型以强大的通用AI能力（尤其是在知识问答、文本生成、代码编写等任务）和较低的训练和推理成本震惊AI界。

● DeepSeek-V3网络依然采用了MoE的结构，并在此基础上进行了优化。

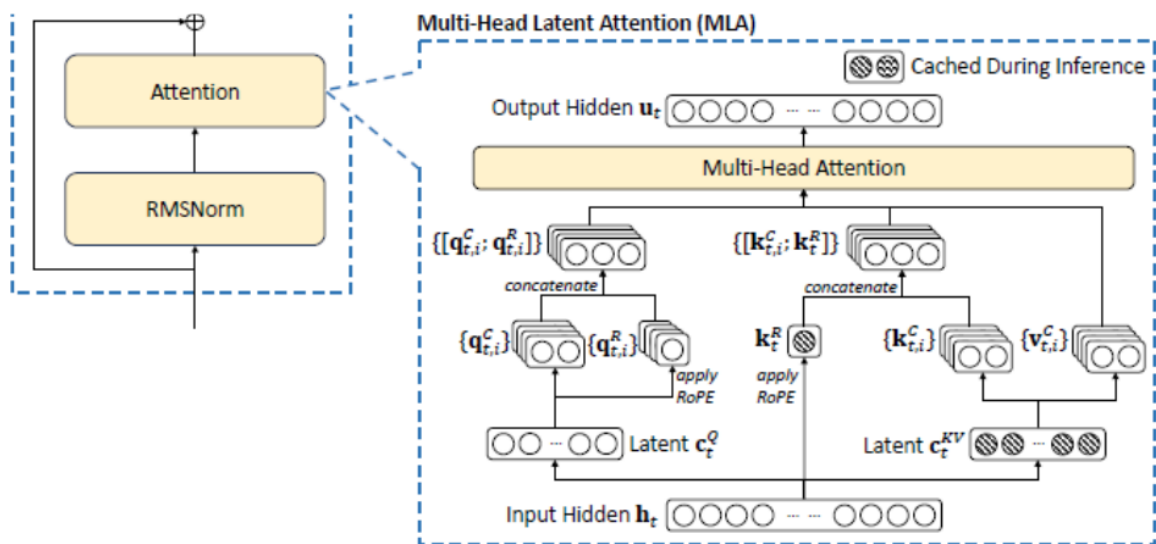


3.6 DeepSeek-V3模型结构创新 - MLA

● MLA主要作用是通过优化 KV-Cache 来减少显存占用，从而提升推理性能。

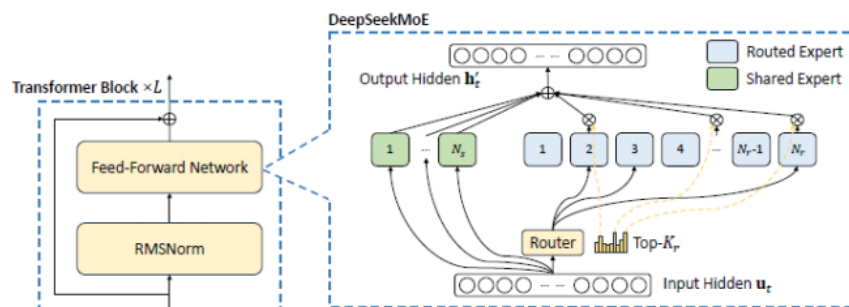
● 功能流程：

- 输入隐藏状态 h_t
- 计算Q/K/V
- 融合旋转位置编码
- 输出隐藏状态 u_t



《DeepSeek-V3 Technical Report》

3.7 DeepSeek-V3模型结构创新 - DeepSeekMoE



《DeepSeek-V3 Technical Report》

Router (路由器)

- 路由器决定输入 U_c 的每一部分将路由到哪些专家
- 每个词元最多路由到 K_r 个专家节点，减少通信开销

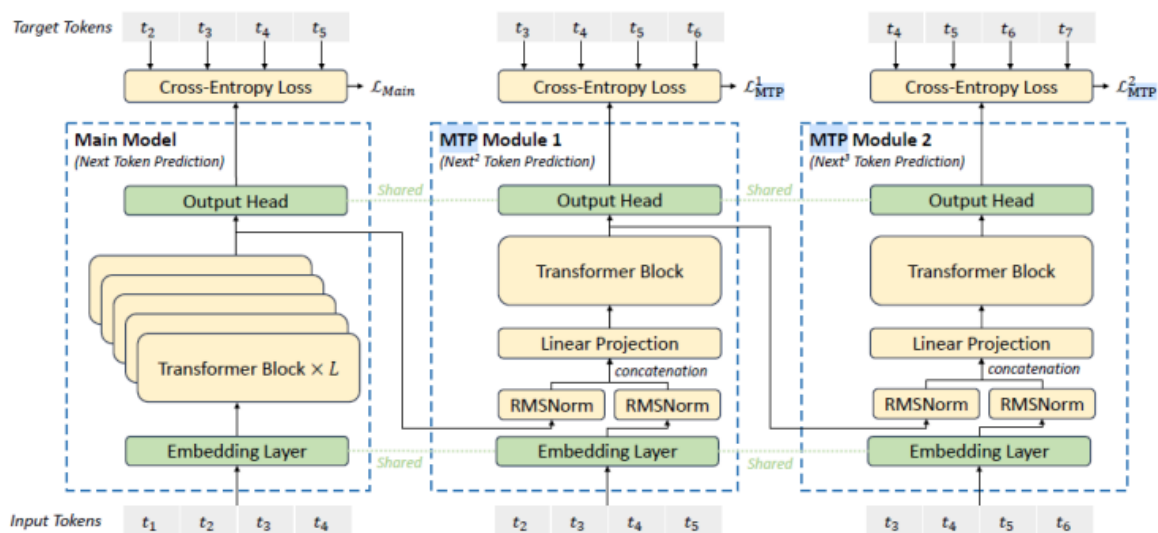
Experts (专家)，包含 256 个专家，分为两类：

- Routed Expert：被路由器选择的专家
- Shared Expert：共享的专家，用于捕捉全局特性

Top-Kr策略：

- 在 N_c 个专家中选出响应最优的 K_r 个专家
- 限制跨节点通信范围，优化效率
- 从 256 个专家中选 9 个 (8 + 1)

3.8 DeepSeek-V3模型结构创新 - MTP



《DeepSeek-V3 Technical Report》

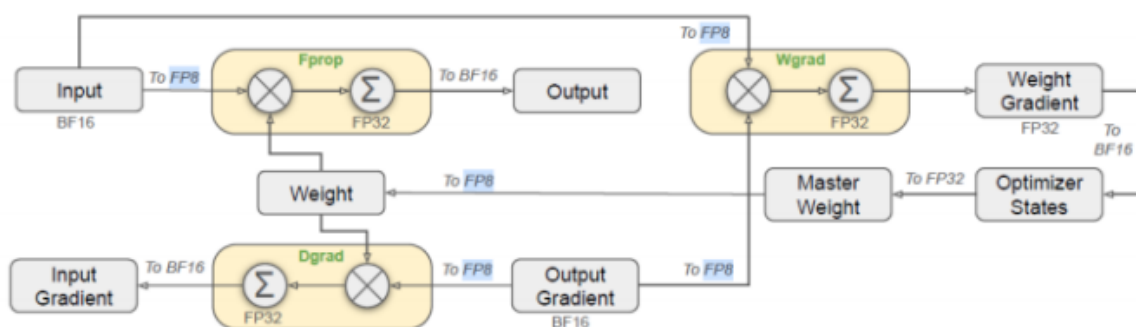
● 传统预测推理速度慢，逐个生成 token，预测一个 token 需要大量跟显存交互，加上单个 token 的预测倾向于局部捕捉最优，整体可能效果不好。

● 核心实现思路：在解码阶段，让模型一次性通过多个顺序模块预测多个未来的 token。具体在训练阶段，一次生成多个后续的 token，可以一次性学习多个位置的 label，提升样本的有效利用率，提升训练效率。该技术可以用在推理阶段，一次生成多个 token，实现推理速度成倍增加。

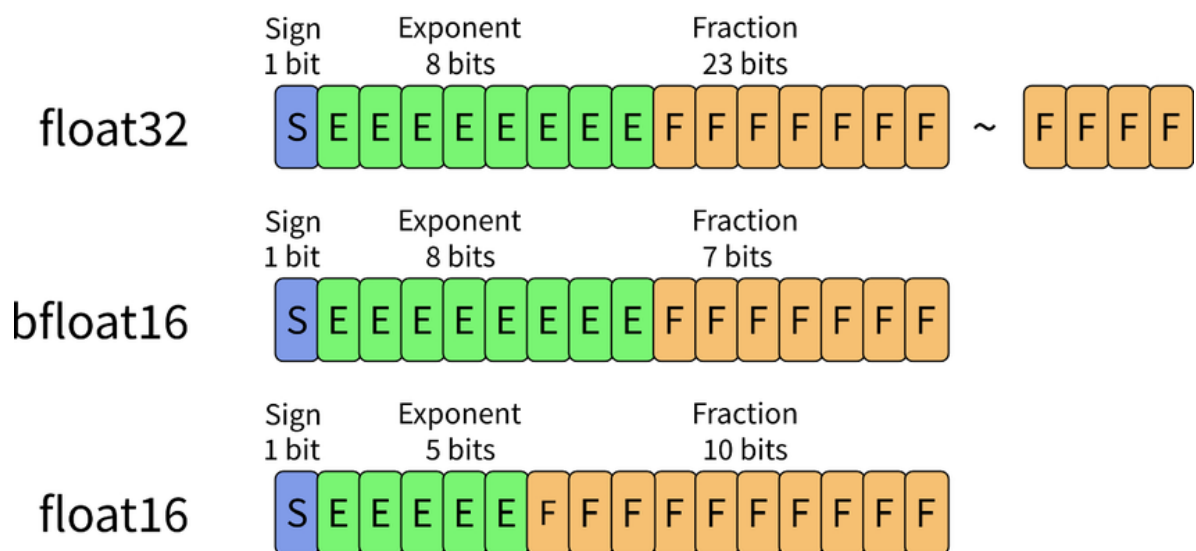
3.9 DeepSeek-V3工程创新 – FP8低位宽训练

● 混合精度策略：

- 计算密集型（前向传播、激活反向、权重反向等）算子均采用FP8计算来提升训练速度。
- 精度敏感型（向量层、输出层、门控模块、注意力模块等）仍然保持BF16/FP32精度，确保精度损失保持在可接受范围内。
- 优化器动量采用BF16，主要权重参数和梯度保持FP32格式。



float32 与 bfloat16 与 float16 的区别：

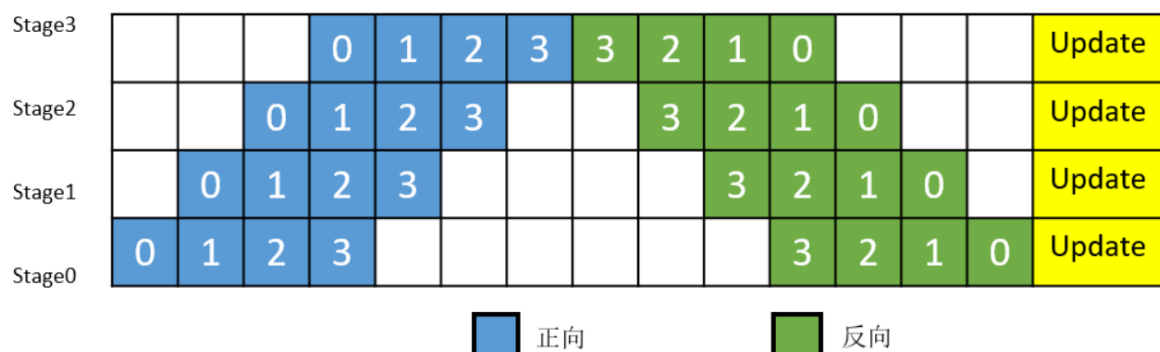


float16 (Brain Floating Point 16) 是由Google提出的。

- Google的推动：
 - Google开发bfloat16的主要目的是为了加速其TPU（张量处理单元）上的深度学习计算。
 - Google发现，在深度学习中，对数值的表示范围要求较高，而对精度要求相对较低。
 - 因此，他们设计了bfloat16，它与float32具有相同的指数位，从而保持了较大的表示范围，但尾数位较少，精度较低。
- 应用场景：
 - bfloat16在深度学习领域得到了广泛应用，特别是在需要处理大规模数据和进行快速计算的场景中。
 - 它在Google的TPU和其他一些硬件平台上得到了支持。
- 主要特点：
 - 与float32具有相似的表示范围。
 - 精度比float16更低。
 - 能够有效加速深度学习计算。

3.10 DeepSeek-V3工程创新 - DualPipe

● 在传统的流水线并行中（Pipeline），每个计算节点执行一部分计算（如前向传播、反向传播等），计算完成后，需要将结果传输给下一个GPU。如果每次计算后必须等待通信完成才能继续下一次计算，跨节点通信未完成，下一阶段的节点只能等待，导致计算资源闲置，这段空闲的时间片段称为流水线气泡（Pipeline Bubbles）。



● DualPipe:

- 每个计算块被划分为四个功能模块：注意力机制、全节点数据分发、MLP 处理、全节点数据整合。
- 使用双向流水线及流水线重排，有效融合前向和后向计算的通信阶段，减少流水线阻塞实现计算与通信充分重叠。



CSDN @致Great

<https://github.com/deepseek-ai/DualPipe>

DualPipe 简介:

- DualPipe 是一种双向流水线并行算法，由 DeepSeek-AI 在 DeepSeek-V3 技术报告中提出。
- 它实现了正向和反向计算-通信阶段的完全重叠，并减少了流水线气泡。
- 核心目标是提高分布式训练的效率，尤其是在大型模型训练中。

核心概念:

- 双向流水线并行：
 - 传统流水线并行是单向的，数据从前向后流动。
 - DualPipe 引入了反向流水线，数据从后向前流动，与正向流水线并行执行。
 - 这种双向设计允许计算和通信的高度重叠。
- 流水线气泡 (Pipeline Bubbles) :
 - 在流水线并行中，由于数据依赖关系，某些 GPU 可能需要等待数据，导致空闲时间，称为流水线气泡。
 - DualPipe 旨在减少这些气泡，提高 GPU 利用率。
- 计算-通信重叠：
 - DualPipe 的关键优势在于它能够将计算和通信操作重叠执行，进一步提高效率。

DualPipeV:

- DualPipeV 是 DualPipe 的一种变体，使用“cut-in-half”过程（由 Sea AI Lab 提出）从 DualPipe 派生而来。
- 它采用 V 形调度，进一步提高了效率。
- DualPipeV 减少了所需要的 GPU 的数量。

流水线气泡和内存使用比较:

方法	气泡	设备参数	设备激活值	设备数量
1F1B	$(PP-1)(F+B)$	$1\times$	PP	PP
ZB1P	$(PP-1)(F+B-2W)$	$1\times$	PP	PP
DualPipe	$(PP/2-1)(F\&B+B-3W)$	$2\times$	PP+1	PP
DualPipeV	$(PP/2-1)(F\&B+B-3W)$	$2\times$	PP+1	PP/2

- PP：流水线阶段数（偶数）。
- F：正向块的执行时间。
- B：完整反向块的执行时间。
- W：权重反向块的执行时间。
- F&B：两个相互重叠的正向和反向块的执行时间。

DualPipe 是一种先进的流水线并行算法，它通过引入双向流水线和计算-通信重叠，显著提高了分布式训练的效率。DualPipeV 是其高效的变体。该项目提供了示例代码和详细的技术说明，方便开发者使用。

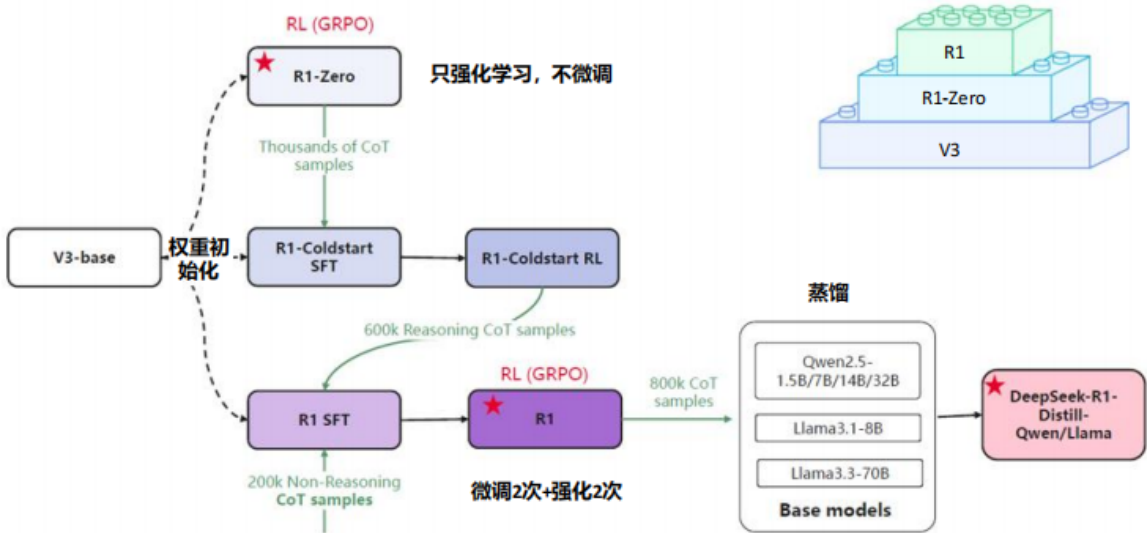
3.11 DeepSeek-R1

● 2025年1月20日，DeepSeek正式发布DeepSeek-R1模型，并同步开源模型权重。DeepSeek-R1在仅有极少标注数据的情况下，极大提升了模型推理能力。



● 广义上的DeepSeek-R1不是一个单一的模型，还包括了R1的初始阶段模型DeepSeek-R1-Zero，以及几个基于R1蒸馏的较小的大模型。

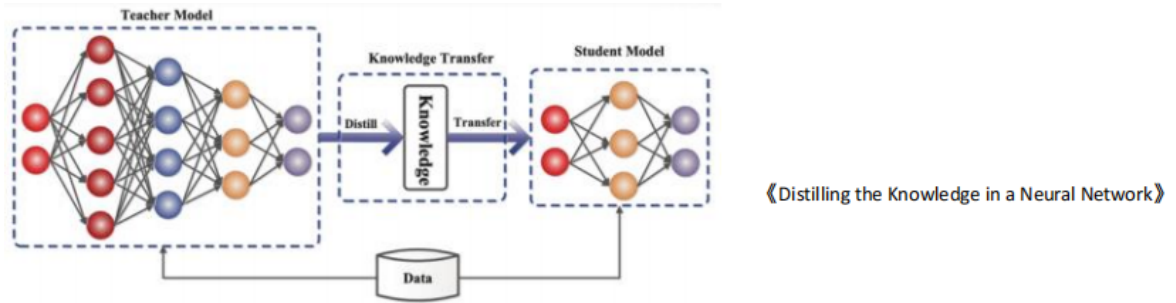
3.12 DeepSeek-R1训练流程



● **R1-Zero**: 抛开传统的SFT、MCTS、PRM，而是在基模上应用GRPO算法进行强化学习，使模型能够自主探索CoT来推理解决复杂问题。

3.13 DeepSeek模型蒸馏

● 模型蒸馏（Knowledge Distillation）是一种将大型复杂模型（教师模型，比如：DeepSeek-R1 671B）的知识迁移到小型高效模型（学生模型，比如：Qwen 7B）的技术。



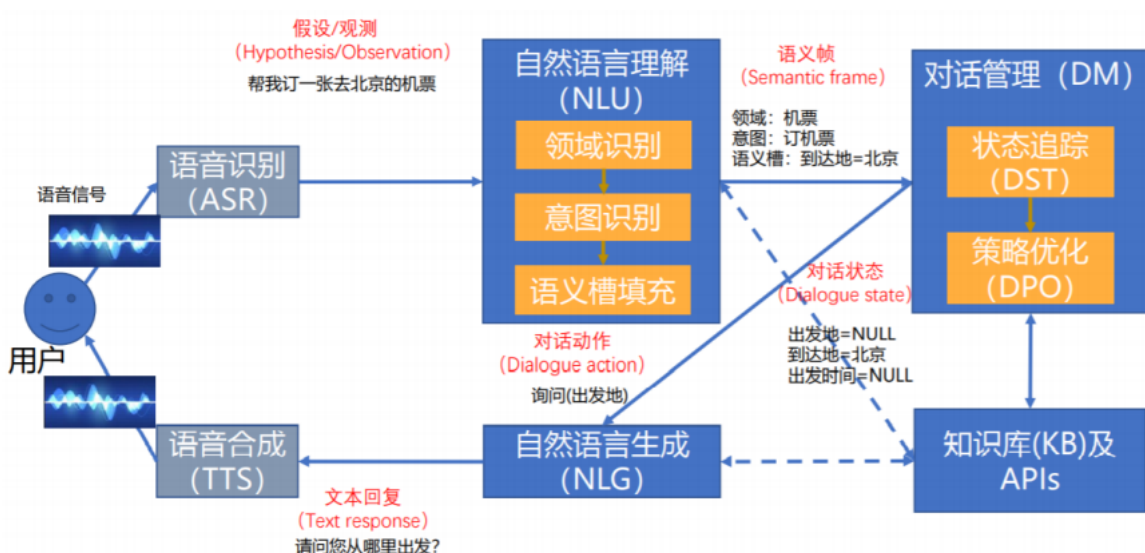
- DeepSeek-R1生成80万条数据作为监督数据。
- 对开源小模型如Qwen、Llama系列进行监督微调（SFT），而不使用强化学习。
- 32B/70B模型的性能接近OpenAI o1-mini。例如，DeepSeek-R1-DistillQwen 32B 在AIME 2024上达到72.6%的Pass@1。

第四章 自然语言处理应用系统

4.1 对话系统

	任务型 Task	聊天 Chat	知识问答 Knowledge	推荐 Recommendation
目的	完成任务或动作	闲聊	知识获取	信息推荐
领域	特定域（垂直类）	开放域	开放域	特定域
以话轮数评价	越少越好	话轮越多越好	越少越好	越少越好
应用	虚拟个人助理	娱乐、情感陪护	客服、教育	个性化推荐
典型系统	Siri、Cortana	小冰	IBM Watson	阿里小蜜

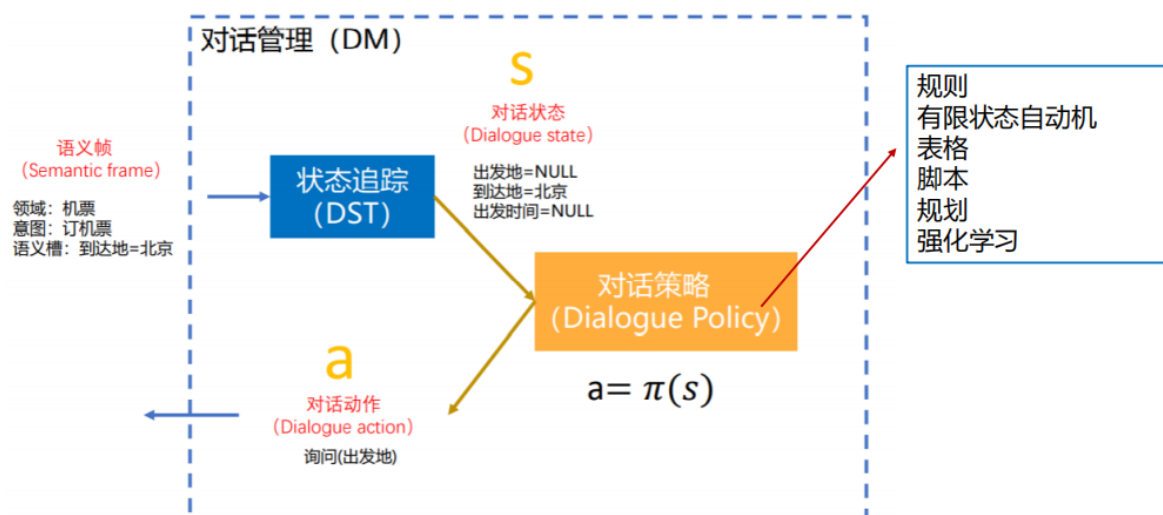
4.2 任务型对话系统架构



4.3 对话系统关键技术 - 自然语言理解

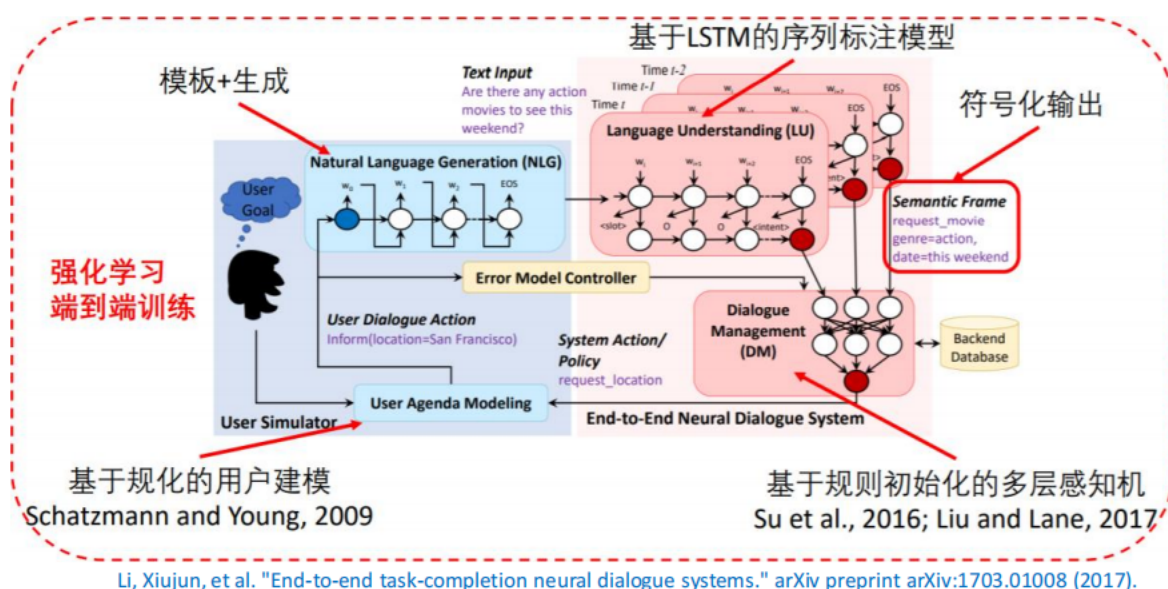


4.4 对话系统关键技术 - 对话管理



4.5 对话系统关键技术 – 端到端对话系统

- 各个部分都是独立的神经网络模型



第五章 缩略语

- LLM: Large Language Model
- MLM: Masked Language Model
- ELMO: Embeddings from Language Model
- NSP: Next Sentence Prediction